

Spring Boot

프레임 워크 개요



Spring



Boot



Security

스프링 boot API

<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>

<https://start.spring.io/>

≡ 프레임 워크 개요

≡ 프레임 워크

개발자들이 공통적으로 사용하는 코드와 구조를 재사용하며, 일관된 구조를 제공해주는 일종의 "골격" 또는 "틀"입니다.

주요 특징

재사용성

자주 사용되는 코드와 기능을 미리 구현해 놓았기 때문에, 개발자는 이를 재사용함으로써 개발 시간을 절약할 수 있습니다.

일관성

개발의 일관된 구조를 제공하여, 코드가 체계적으로 유지될 수 있게 합니다.

이는 팀 내 여러 개발자가 협업할 때 특히 유용합니다.

확장성

보통 기본 기능 외에도 플러그인이나 확장 모듈을 통해 기능을 추가할 수 있는 구조로 되어 있어, 필요에 따라 애플리케이션을 확장할 수 있습니다.

효율성

개발에 필요한 여러 가지 기능을 제공해 주기 때문에, 초기 설정이나 환경 구축 등에 드는 시간을 절약할 수 있습니다.

프레임 워크 개요

프레임 워크

개발자들이 공통적으로 사용하는 코드와 구조를 재사용하며, 일관된 구조를 제공해주는 일종의 "골격" 또는 "틀"입니다.

프레임 워크 예시

프레임워크는 개발자들이 특정 문제를 해결하는 데 있어 시간을 절약하고, 코드의 일관성과 품질을 유지하는 데 중요한 역할을 합니다.

항목	설명
웹 개발	Django(파이썬 기반) Spring(자바 기반) Ruby on Rails(루비 기반)
모바일 앱 개발	Flutter(다트 기반) React Native(자바스크립트 기반)
게임 개발	Unity(C# 기반) Unreal Engine(C++ 기반)

≡ 프레임 워크 개요

≡ 스프링 프레임 워크

스프링은 자바 플랫폼을 위한 오픈 소스 애플리케이션 프레임워크입니다.

스프링은 자바 개발을 보다 쉽고 효율적으로 할 수 있도록 도와주는 도구들을 제공합니다.
연관성이 있는 기능들을 모아서 일관성 있는 방식으로 사용할 수 있도록 구성되어 있습니다.

스프링 프레임 워크의 특징

경량 컨테이너

자바 객체 라이프 사이클 관리

의존성 주입 (DI)

스프링 컨테이너가 객체를 주입

AOP(Asspect-Oriented Programming)

핵심 비즈니스 로직과 각 관심사 분리

트랜잭션 관리

데이터 액세스 기술 지원: JDBC, JPA, Hibernate 등 지원

테스트 지원

단위 테스트, 통합 테스트 등 다양한 테스트 지원

보안

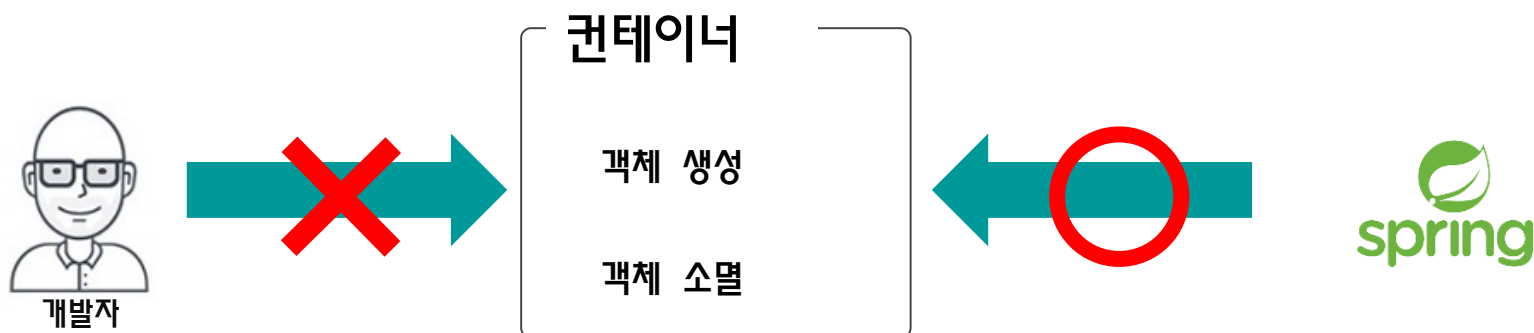
보안 기능 제공, 시큐리티(Spring Security)

프레임 워크 개요

스프링과 스프링 부트

스프링 컨테이너는 경량 컨테이너입니다.

스프링 컨테이너는 자바의 객체 라이프 사이클을 관리합니다.



프레임 워크 개요

제어의 역행(IoC)

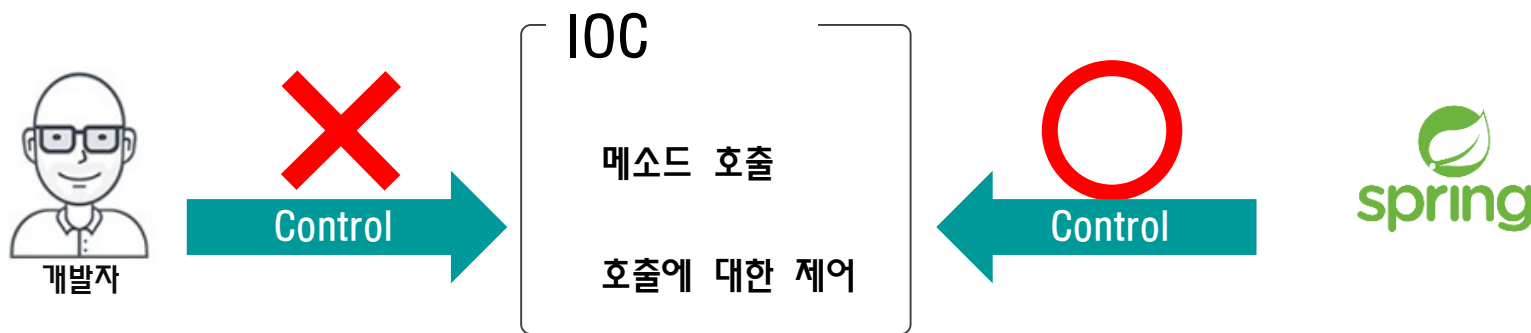
OOP에서 프로그램의 제어 흐름을 개발자가 아닌 프레임워크나 컨테이너가 맡는 것을 의미합니다.

제어의 역행(Inversion of Control, IoC)라고 합니다.

개발자가 직접 객체의 생성과 생명 주기를 관리하는 것이 아니라 외부 요소에 의해 객체의 제어가 역전되는 개념을 말합니다.

제어의 역행은 주로 의존성 주입(Dependency Injection, DI)을 통해 구현됩니다.

의존성 주입은 객체 간의 의존 관계를 외부에서 결정하고 주입하는 것을 말하며, 이를 통해 객체 간의 결합도를 낮추고 유연하고 확장 가능한 코드를 작성할 수 있습니다.



≡ 프레임 워크 개요

≡ 스프링과 스프링 부트

스프링과 스프링 부트의 차이점에 대하여 살펴 봅니다.

스프링 프레임워크

Spring은 자바 기반의 오픈소스 애플리케이션 프레임워크로서, 엔터프라이즈급 애플리케이션을 개발할 때 사용됩니다.

Spring Framework가 제공하는 기능

IoC(Inversion of Control)

DI(Dependency Injection)

AOP(Aspect-Oriented Programming),

MVC(Model-View-Controller) 등등

스프링 부트의 특징

Spring Boot는 Spring Framework를 기반으로 한 플랫폼으로서, 더 쉽게 스프링 애플리케이션을 개발하고 실행할 수 있도록 도와줍니다.

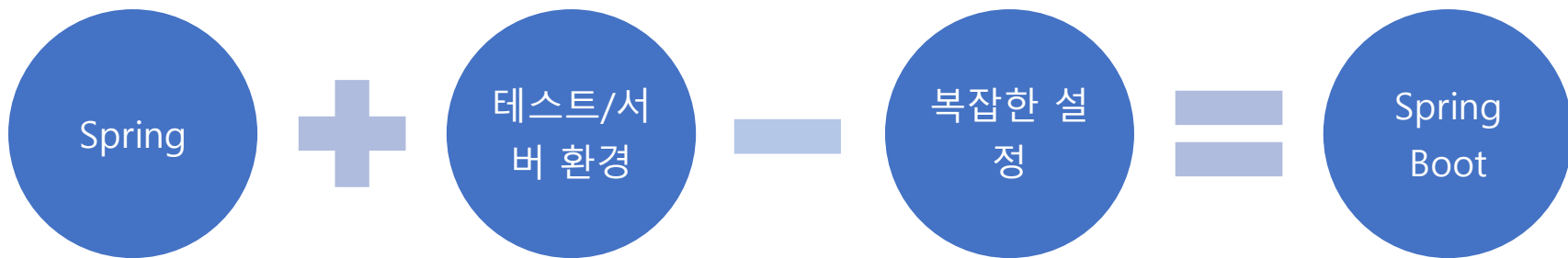
Spring Boot는 애플리케이션의 설정과 구성을 자동화하고, 내장형 웹서버를 제공하여 별도의 웹서버 없이 애플리케이션을 실행할 수 있도록 합니다.

Spring이 기본 프레임워크이며, Spring Boot는 Spring Framework를 이용하여 애플리케이션을 보다 쉽게 개발, 구성, 실행할 수 있는 도구입니다.

프레임 워크 개요

스프링과 스프링 부트

스프링 부트는 최소한의 설정 정보로, 실행 버튼 클릭만으로 독립 실행 애플리케이션을 지향합니다.



스프링 프레임워크

스프링 DI, security, jdbc, MVC 등의 설정을 위한 코드를 개발자가 직접 작성해야 합니다.
개발을 하기 위한 XML 설정이 너무 복잡하여 초반에 너무 많은 시간을 소비합니다.

스프링 부트의 특징

Spring Framework에 비하여 단순화 되었습니다.
Tomcat, 제티와 같은 웹 애플리케이션을 자체 내장하고 있습니다.
‘Spring Boot Starter’으로 빌드 구성을 단순화했습니다.
가급적 XML 설정 없이 단순한 자바 수준의 설정 기법을 제공해 줍니다.
JAR를 이용하여 자바 옵션만으로 배포가 가능합니다.

사전 지식

자바/웹 프로그래밍/Spring Framework/데이터 베이스에 대한 기본적인 이해가 필요합니다.

Spring Boot

프로젝트 생성하기



Spring



Boot



Security

≡ 프로젝트 생성하기

≡ 쇼핑몰 프로젝트 생성하기

for IntelliJ

spring initializer 사이트를 활용하여 프로젝트를 생성하도록 합니다.

사이트 소개

스프링 애플리케이션을 쉽게 생성할 수 있도록 도와주는 웹 기반 도구입니다.

스프링 프로젝트의 초기 설정을 자동화시켜, 필요한 설정을 빠르고 간편하게 완료하고 바로 개발에 집중할 수 있도록 지원해 줍니다.

프로젝트 생성하기

쇼핑몰 프로젝트 생성하기

for IntelliJ

spring initializer 사이트를 활용하여 프로젝트를 생성하도록 합니다.

Project 프로젝트 타입 설정

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ **Java** ☐ Kotlin ☐ Groovy

☒ **Maven**

Spring Boot Spring Boot 버전 선택

☐ 3.4.0 (SNAPSHOT) ☐ 3.4.0 (M3) ☐ 3.3.5 (SNAPSHOT)

☐ 3.2.11 (SNAPSHOT) ☒ **3.2.10**

Project Metadata 프로젝트 메타 데이터 설정

Group	com.shopping
Artifact	shopping
Name	shopping
Description	Shopping project for Spring Boot
Package name	com.shopping
Packaging	☒ Jar ☐ War
Java	☐ 23 ☐ 21 ☒ 17

빌트 툴 : Maven
언어 : Java
스프링 부트 : 적정 버전 선택
패키징 : Jar
의존성 : 파일 참조

<https://start.spring.io/> 사이트에서 다음과 같이 설정 후 단축키 Ctrl + B를 눌러서 의존성을 추가합니다. Project Metadata에서 Artifact 이름으로 zip 형식의 압축 파일을 만들어 줍니다.

항목	설명
Spring Boot DevTools	개발 편의 기능 제공 (코드 변경 시 자동 재시작, 캐시 비활성화, LiveReload 지원). 개발 단계에서 생산성을 높여줌.
Lombok	반복적인 코드(게터, 세터, 생성자, toString 등)를 어노테이션으로 자동 생성. IDE 플러그인 설치 필요.
Spring Data JPA	ORM(Object Relational Mapping) 기반 데이터 접근 계층을 간단히 구현. JPA 인터페이스와 리포지토리 방식으로 DB 작업을 쉽게 처리.
Oracle Driver	오라클 데이터베이스와 연결하기 위한 JDBC 드라이버. JPA/Hibernate와 함께 사용 가능.
Thymeleaf	서버 사이드 템플릿 엔진. HTML과 자연스럽게 결합되어 뷰 렌더링에 사용됨.
Spring Web	REST API 및 웹 애플리케이션 개발 기본 모듈. Spring MVC 기반의 컨트롤러와 라우팅 제공.
Spring Web Services	SOAP 기반의 웹 서비스 개발을 지원. 주로 레거시 시스템이나 엔터프라이즈 환경에서 사용.
h2	가볍고 빠른 인메모리 데이터베이스. 개발·테스트 단계에서 자주 사용, 별도 설치 없이 실행 가능.

프로젝트 생성하기

쇼핑몰 프로젝트 생성하기

for IntelliJ

spring initializer 사이트를 활용하여 프로젝트를 생성하도록 합니다.

다음과 같은 항목들을 추가한 후 하단의 **GENERATE** 버튼을 눌러서 종료하도록 합니다.

그러면, 관련 설정 파일을 이용하여 압축 파일을 생성해주며, 이 파일을 다운로드 할 수 있도록 해줍니다.

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Boot DevTools

DEVELOPER TOOLS

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Lombok

DEVELOPER TOOLS

Java annotation library which helps to reduce boilerplate code.

Spring Data JPA

SQL

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

Oracle Driver

SQL

A JDBC driver that provides access to Oracle.

Thymeleaf

TEMPLATE ENGINES

A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes.

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

H2 Database

SQL

Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

Spring Web Services

WEB

Facilitates contract-first SOAP development. Allows for the creation of flexible web services using one of the many ways to manipulate XML payloads.

MySQL Driver

SQL

MySQL JDBC driver.

GENERATE CTRL + G

EXPLORE CTRL + SPACE

SHARE...

≡ 프로젝트 생성하기

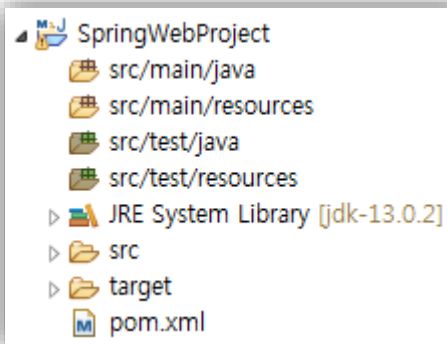
≡ 스프링 프로젝트의 기본 폴더 구조

생성된 스프링 프로젝트의 기본 구조는 다음과 같습니다.

스프링 프로젝트를 생성하게 되면 다음과 같은 폴더 구조가 만들어 집니다.

생성이 되지 않은 항목이 있는 경우 수동으로 생성해도 상관이 없습니다.

명시된 폴더 구조는 권장 사항이지 절대적인 내용은 아닙니다.



항목	설명
src/main/java	자바 관련 소스 코드들이 저장되는 폴더입니다.
src/main/resources	프로퍼티나 xml 파일 등 관련된 리소스가 저장되는 폴더입니다.
src/main/webapp	웹 애플리케이션과 관련된 폴더로써, webapp은 웹 페이지의 최상위 폴더의 이름입니다. jsp 파일이나 WEB-INF 폴더 등을 작성하는 영역입니다. WEB-INF 폴더는 Apache Tomcat이 반드시 읽어 들이는 항목이므로 무조건 작성해 두는 것이 좋습니다.
src/test/java	테스트 관련 자바 소스 파일이 작성 되는 폴더입니다.
src/test/resources	테스트 과정에서 필요한 리소스 파일을 생성하는 폴더입니다.

Spring Boot

설정하기



Spring



Boot



Security

≡ `application.properties` 파일

애플리케이션의 다양한 설정과 구성을 정의할 수 있는 **환경 설정용 파일**입니다.

application.properties

Spring에서 참조하는 여러 설정 옵션을 정의하는 데 사용되며, 애플리케이션이 실행될 때 이를 기반으로 환경 설정을 적용합니다.

예를 들면 서버 설정, 데이터베이스 연결, 로깅 수준 등을 간단히 구성할 수 있습니다.

프로젝트를 생성하게 되면 자동으로 생성이 되어 있으며, 파일의 위치는 `src/main/resources`입니다.

참조 사이트

<https://docs.spring.io/spring-boot/docs/current/reference/html/application-properties.html>

주요 역할 및 기능

항목	설명
애플리케이션 설정	Spring Boot 애플리케이션의 동작 방식을 정의합니다. 예를 들어, 서버 포트, 데이터베이스 연결 정보, 로깅 레벨 등의 설정을 포함할 수 있습니다.
데이터베이스 설정	데이터베이스 연결 정보를 정의하여 Spring Data JPA, JDBC 등과 같은 데이터 액세스 계층을 구성할 수 있습니다.
로깅 설정	애플리케이션의 로깅 레벨과 로그 파일 위치 등을 설정할 수 있습니다.
메시지, 국제화(i18n)	애플리케이션에서 사용하는 메시지와 국제화 관련 설정도 정의할 수 있습니다.
기타 커스터마이징	스프링에서 제공하는 다양한 기능을 application.properties 파일을 통해 커스터마이징할 수 있습니다. 예를 들어, 세션 설정, 캐시 설정, 스케줄러 설정 등이 가능합니다.

≡ Auto import 옵션 설정

for IntelliJ

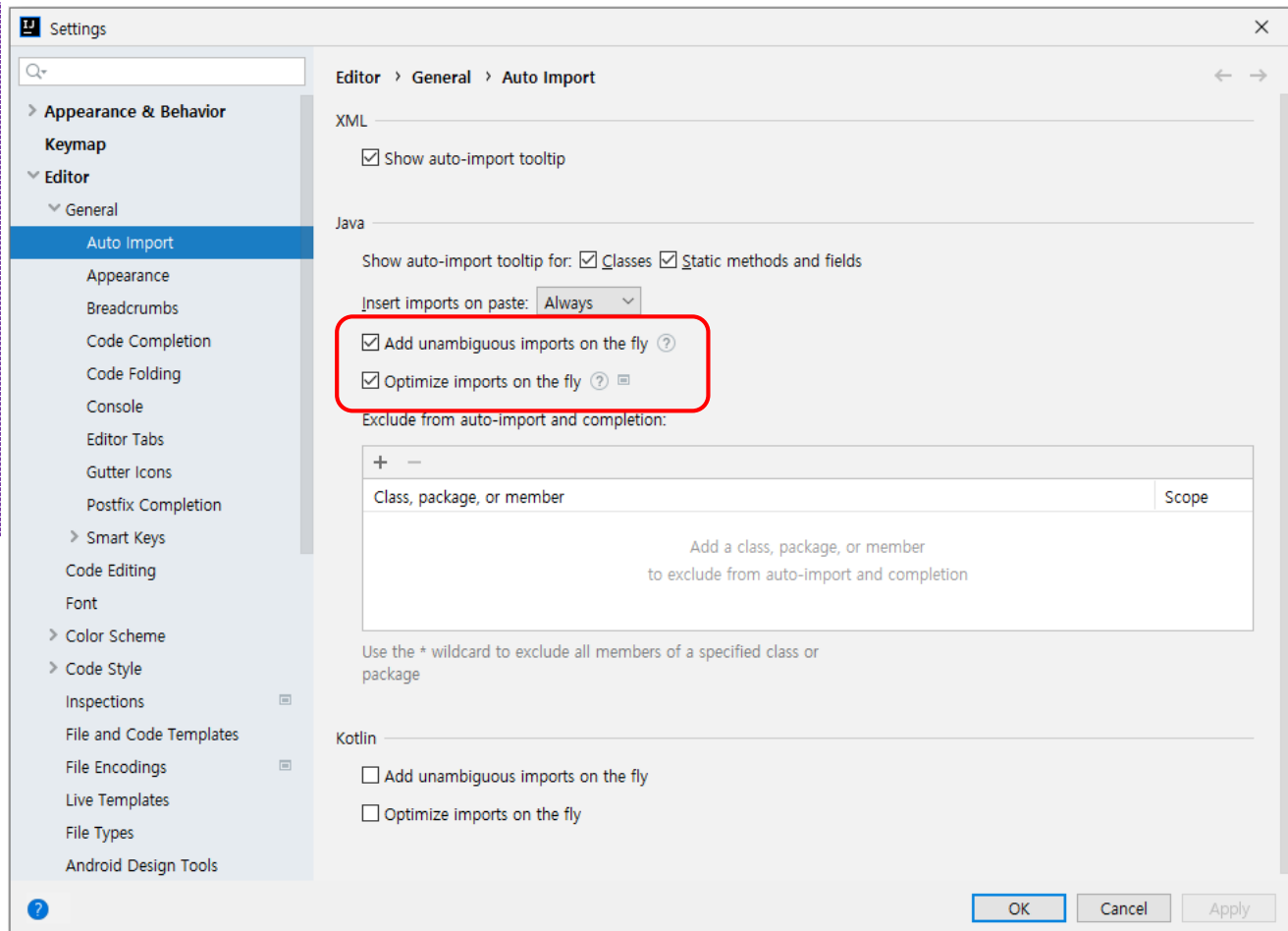
IntelliJ에서 자동 임포트를 수행하기 위하여 다음과 같이 설정하도록 합니다.

Auto import 기능

사용할 또는 사용되지 않는 클래스나 인터페이스에 대하여 자동으로 import 구문을 넣거나/빼주는 기능입니다.

"Add unambiguous import on the fly" 기능은, 코드 작성 중에 클래스 또는 패키지에 대한 import 구문을 자동으로 import 해주는 기능입니다.

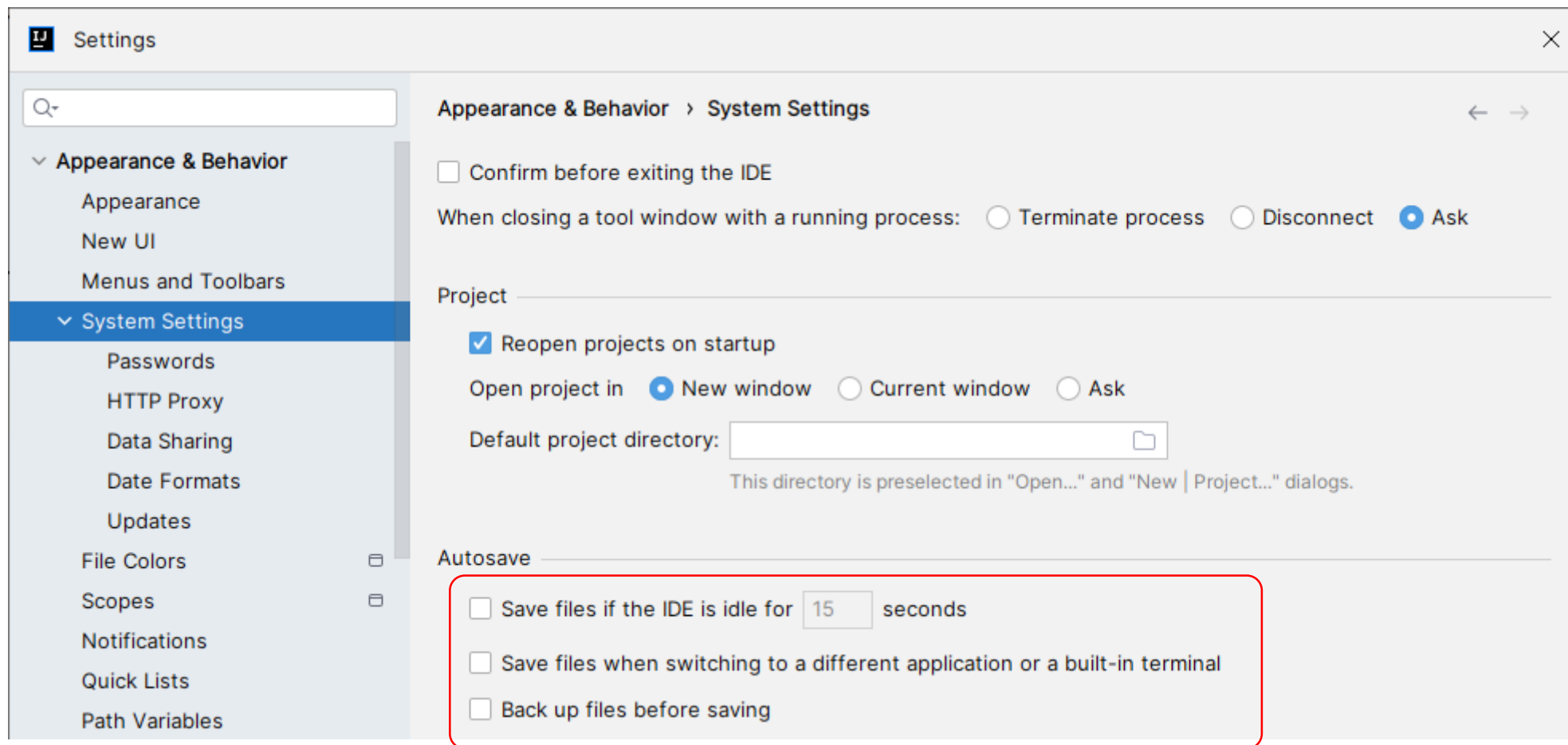
"Optimize imports on the fly"는 코드 작성 중에 자동으로 사용하지 않는 import 문을 제거해주는 기능입니다.



☰ 설정하기

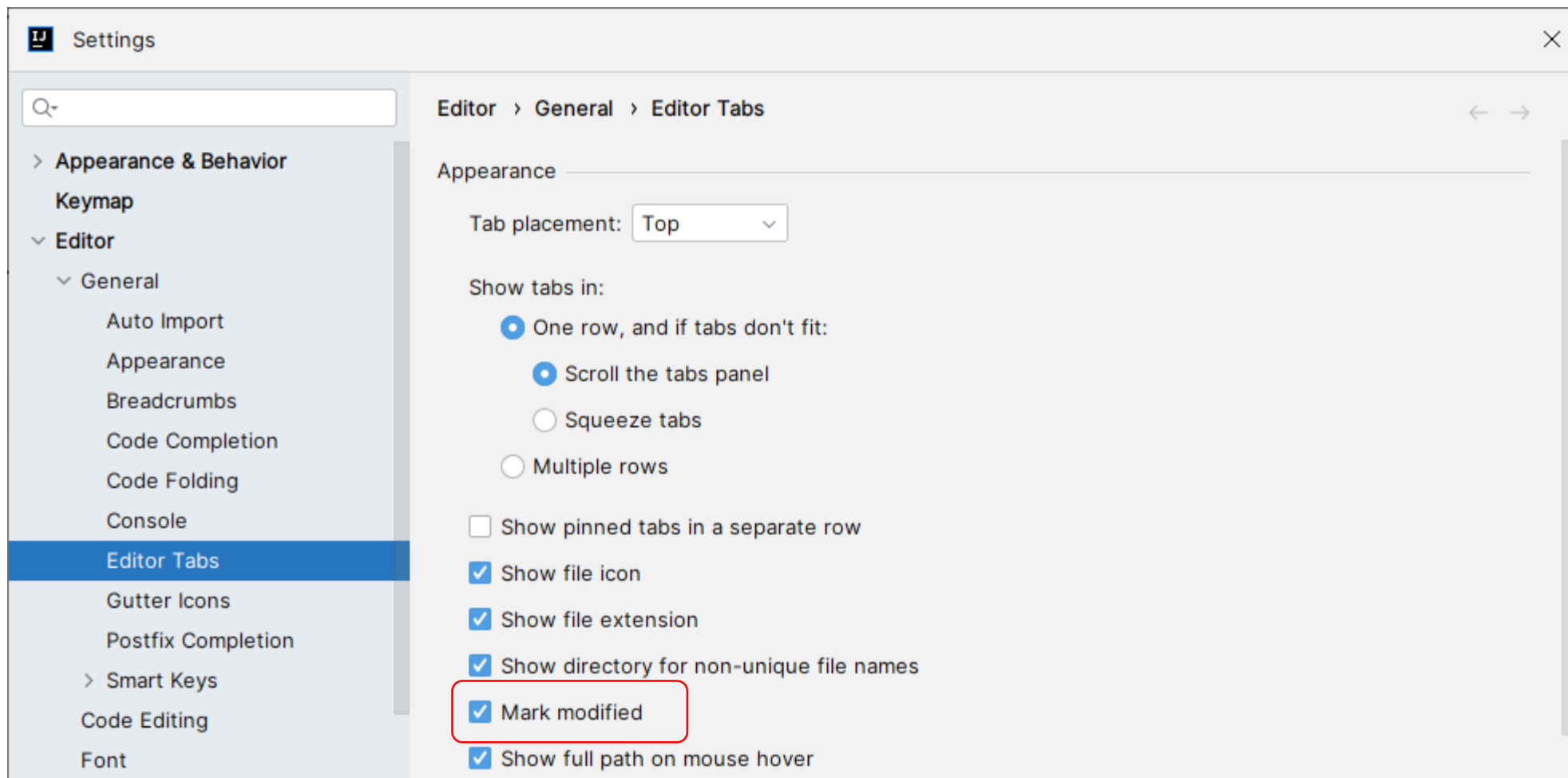
☰ 자동 저장 기능 해제

인텔리제이는 파일 수정시 자동으로 저장이 되는 데, 이러한 기능을 해제하고자 하는 옵션입니다.



≡ 자동 저장 기능 해제

인텔리제이는 파일 수정시 자동으로 저장이 되는 데, 이러한 기능을 해제하고자 하는 옵션입니다.



Spring Boot

메이븐(Maven)



Spring



Boot

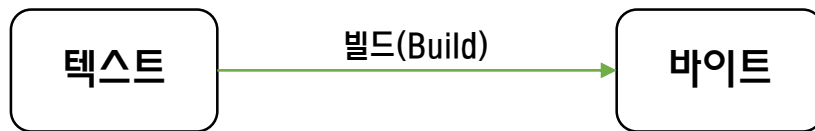


Security

빌드(Build)

텍스트 형식의 소스 코드를 컴퓨터가 실행할 수 있는 **바이트 형태의 소프트웨어 가공물로 변환**하는 과정 일체를 의미합니다.

예를 들어서 자바 소스 코드나 여러 개의 파일(xml, jpg, jar)을 자바 가상 머신이나 Apache Tomcat 등과 같은 WAS가 인식할 수 있는 구조로 패키징시키는 과정이라고 보시면 됩니다.



Build Tool

빌드 도구(Build Tool)란 프로젝트를 생성하거나, 테스트를 위한 빌드, 애플리케이션 배포 등의 작업을 위한 전용 프로그램을 지칭합니다

초기에는 지속적으로 **늘어나는 라이브러리의 추가**, **라이브러리의 버전 동기화의 어려움**을 해소하고자 등장하였고, java를 빌드 도구로 초기에는 **앤티(Ant)**를 많이 사용하였습니다.

이후에 나온 빌드 도구들은 **메이븐(Maven)**, **그레이들(Gradle)** 등이 있습니다.

메이븐(Maven)

자바용 프로젝트 관리 도구로써 Apache Ant의 대안으로 만들어졌습니다.

주요 특징

프로젝트의 전체적인 라이프 사이클을 관리해 줍니다.

프로젝트의 작성/컴파일 등 프로젝트 라이프 사이클에 포함되는 각 테스트를 지원해줍니다.

설정 파일(pom.xml)에 관련된 내용을 정의해 놓으면 필요한 모든 라이브러리 파일들을 자동으로 다운 받아서 처리해 줍니다.

간단한 설정을 통한 배포 관리가 가능하며, 라이브러리에 대한 의존 관계를 관리해 줍니다.

maven 관련 참조 사이트) <http://maven.apache.org>

메이븐(Maven)

Maven으로 할 수 있는 것들은 다음과 같은 항목들이 있습니다.

Maven으로 할 수 있는 것들



pom.xml

Maven이 프로젝트의 빌드 및 관리하기 위해 필요한 설정 정보들을 저장하고 있는 파일입니다.

자바 프로젝트를 Maven Project 형식으로 작성하게 되면 최상위 폴더에 pom.xml 파일이 생성됩니다.

이 파일은 "Project Object Model"의 줄인 말입니다.

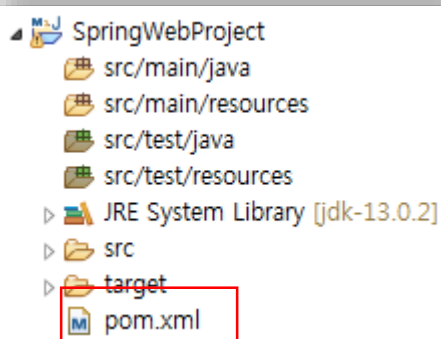
프로젝트 관리 및 빌드에 필요한 환경 설정, 버전 및 설정 관리, 빌드 환경, 라이브러리 저장소 및 의존성 등등 정보들을 기술합니다.

프로젝트의 세부 메타 데이터 정보를 포함하고 있습니다.

pom.xml 파일에서 가능한 일

다음과 같은 항목들에 대한 설정을 담당하고 있는 파일입니다.

항목	설명
프로젝트 정보	프로젝트에 대한 구조와 내용을 설명하고 있는 파일입니다. 프로젝트의 이름이나, 개발자의 목록 정보, 라이선스 등의 정보를 기술합니다.
빌드 설정	프로그램의 소스, 리소스, 생명 주기별 실행할 Plug-In 등등 Build와 관련된 여러 설정을 기술합니다.
빌드 환경	사용자 환경 별로 달라질 수 있는 Profile 정보를 기술합니다.
Pom 연관 정보	의존하는 프로젝트(모듈), 상위 프로젝트, 포함되고 있는 하위 모듈 정보를 기술합니다.



pom.xml

<dependency> 라는 태그를 사용하여 필요한 라이브러리를 지정할 수 있습니다.

예를 들면, querydsl이라는 라이브러리를 사용하기 위해서는 다음과 같이 설정하면 됩니다.

```
<dependency>
  <groupId>com.querydsl</groupId>
  <artifactId>querydsl-jpa</artifactId>
  <version>5.0.0</version>
</dependency>
```

라이브러리는 jar 파일의 형식이며, 다음과 같이 3개의 영역으로 나뉘어지고, 이것을 조합하여 파일 이름을 만들어 줍니다.

com.querydsl.querydsl-jpa.5.0.0.jar

항목	설명
Group Id	프로젝트에 속하는 그룹 식별 값, 회사, 본부, 단체 등을 의미합니다. 일반적으로 package 형식으로 계층을 표현합니다. 조직이나 프로젝트의 Id를 의미합니다.
Artifact Id	프로젝트 결과물의 식별자, 프로젝트나 모듈을 의미하는 값입니다. 성과물인 소프트웨어 Id(프로젝트 이름)를 의미합니다. 참고로 메이븐은 한 개의 모듈을 아티팩트라는 단위로 관리합니다.
Version	결과물의 버전을 의미합니다.

메이븐 라이브러리 저장소

메이븐은 라이브러리를 원격 저장소에서 로컬 저장소로 다운로드 받습니다.

저장소(Repository)란 흩어져 있는 정보나 시스템 개발에 관련된 정보를 모아 두어서 공유할 수 있도록 한 데이터의 저장소를 의미합니다.

"메이븐 중앙 저장소"로부터 스프링에서 제공하는 모든 라이브러리를 가져 와서 사용하는 것이 아니라 개발자가 직접 사용하고자 하는 라이브러리 항목만 "로컬 저장소"로 가져 오면 됩니다.

항목	설명
중앙 저장소(리포지터리)	https://repo.maven.apache.org/maven2/
	https://repo1.maven.org/maven2
로컬 저장소(리포지터리)	내 컴퓨터 [사용자홈]/.m2/repository/groupId/artifactId/version

내 컴퓨터의 [사용자홈] 내의 .m2 폴더는 메이븐 프로젝트를 만들게 되면 자동으로 생성됩니다.

[사용자홈] 폴더의 이름은 cmd 명령어를 입력해 보면 다음과 같이 확인할 수 있습니다.

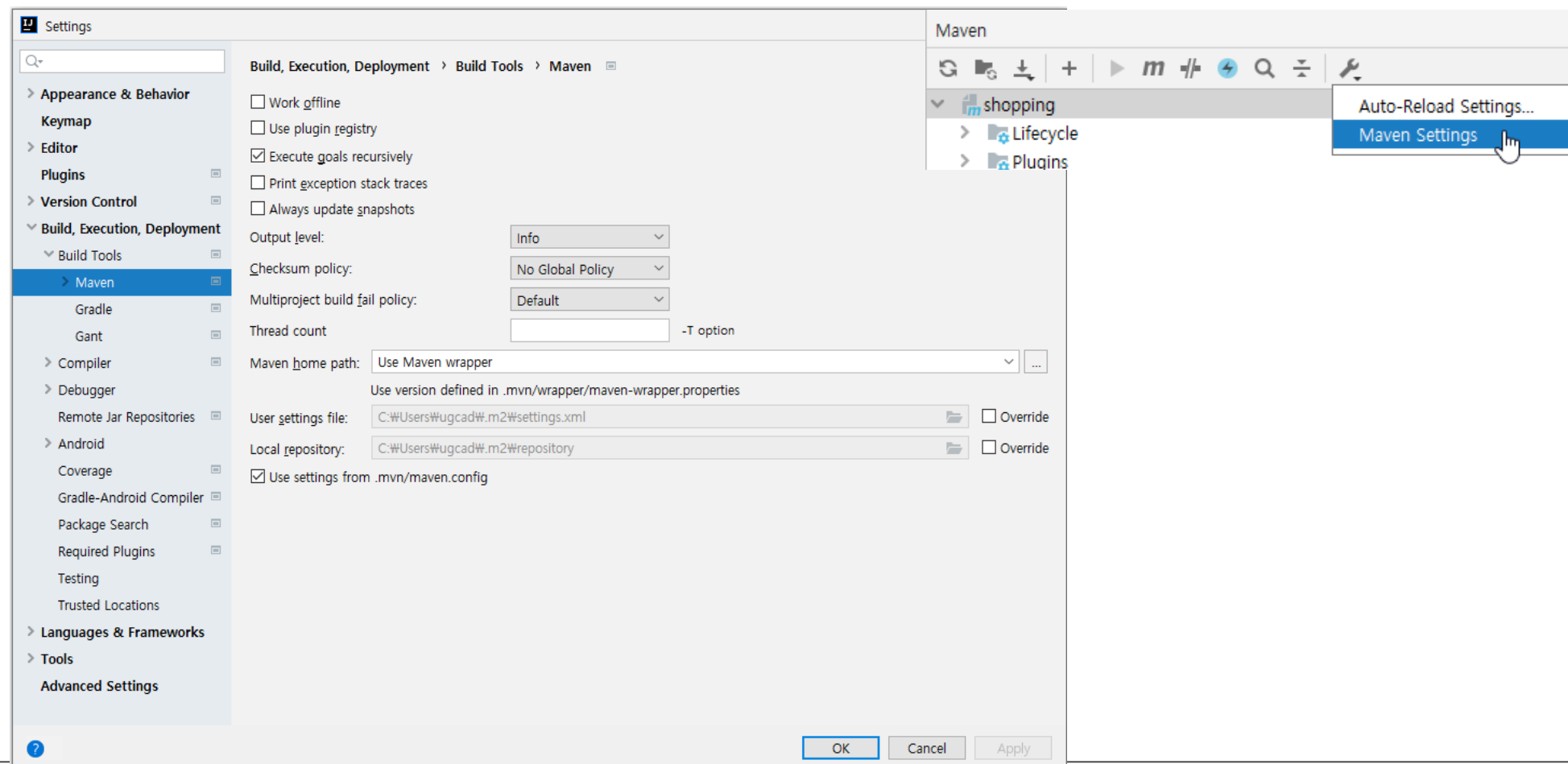
메이븐 라이브러리 저장소

for IntelliJ

인텔리 제이에서 maven 저장소를 확인하는 방법은 다음과 같습니다.

메이븐은 원격 저장소에서 필요한 라이브러리들을 로컬 저장소에 다운로드를 받아 줍니다.

로컬 저장소는 Build Tools Settings 메뉴를 클릭하면 확인할 수 있습니다.



라이브러리 다운로드

for IntelliJ

pom.xml 이라는 파일에 라이브러리를 작성해주면 메이븐이 스스로 네트워크를 통하여 파일을 다운로드 해줍니다.

관련된 jar 파일을 자동으로 다운로드 하여 로컬 저장소에 넣어 줍니다.

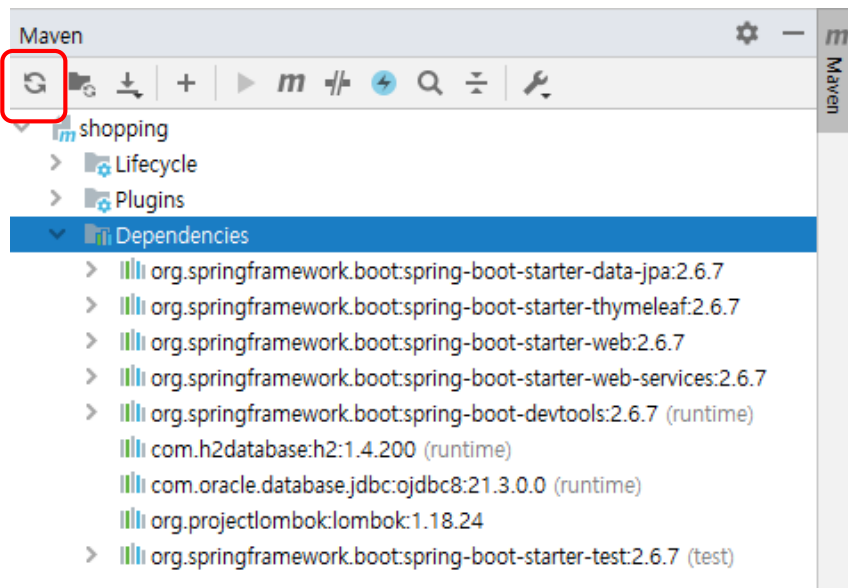
Maven의 의존성

인텔리제이의 오른쪽 상단에 있는 [Maven]이라는 항목을 클릭합니다.(Maven Tool View라고 합니다.)

Maven Tool View는 사이트 <https://www.jetbrains.com/help/idea/maven-projects-tool-window.html>를 참조하세요

의존성(Dependencies) 항목을 보면 spring initializer 사이트에서 선택했던 항목들을 확인할 수 있습니다.

Dependencies 파일 목록을 갱신하려면 새로 고침 버튼을 클릭하면 됩니다.



Spring Boot

JUnit



Spring



Boot



Security

제이유닛(JUnit)

Java 프로그래밍 언어를 위한 테스트 프레임워크로, 단위 테스트를 간단히 작성하고 실행하는 데 사용됩니다.

JUnit은 자바 언어용으로 개발된 유닛 테스트 프레임워크입니다.

JUnit은 테스트 주도 개발(TDD)을 지원하며, 개발자가 코드의 정확성과 품질을 유지할 수 있도록 도와줍니다.

개발자들은 JUnit을 사용하여 자신이 작성한 자바 소스 코드의 각각의 단위(함수, 메소드 등)가 예상대로 동작하는지를 테스트할 수 있습니다.

JUnit은 테스트 코드 작성을 지원하며, 자동화된 테스트를 수행할 수 있습니다.

이를 통해 개발자들은 프로그램을 변경할 때마다 수동으로 테스트를 실행할 필요 없이, 각각의 단위가 정상적으로 작동하는지를 확인할 수 있습니다.

또한, JUnit은 실패한 테스트 케이스를 식별하여 개발자들이 빠르게 수정할 수 있도록 도와줍니다.

JUnit의 특징

항목	설명
assertion(단정문)	JUnit은 다양한 assertion 메소드를 제공하여 테스트 케이스의 수행 결과를 판별해 알려 줍니다. 예를 들어, assertEquals(), assertTrue(), assertFalse() 등을 사용하여 예상되는 값과 실제 값을 비교할 수 있습니다.
어노테이션	@Test, @Before, @After 등의 어노테이션을 사용하여 테스트 메소드와 테스트 실행 순서를 정의할 수 있습니다.
단위 테스트	메소드 단위로 테스트를 작성할 수 있어, 개별 기능을 독립적으로 검증할 수 있습니다.
-	테스트 결과를 Test 클래스로 남김으로써 테스트에 대한 이력(history)을 저장할 수 있습니다. System.out.println()로 매번 사람이 동작 결과를 확인해야 하는 테스트 및 디버깅 과정을 쉽게 해줍니다.

제이유닛(JUnit)

Java 프로그래밍 언어를 위한 테스트 프레임워크로, 단위 테스트를 간단히 작성하고 실행하는 데 사용됩니다.

JUnit의 annotation

어노테이션	설명
@Test	해당 메소드를 테스트 대상으로 지정합니다.
@DisplayName	JUnit5에서 추가된 어노테이션으로 테스트 코드를 실행할 때 명시한 '텍스트'의 이름을 출력해 줍니다.

JUnit의 assert 메소드들

메소드	설명
assertEquals(A, B)	객체 A(저장하려고 요청한 값)와 B(실제 저장된 데이터)가 동일한지 확인해 줍니다.
assertArrayEquals(A, B)	배열 A와 B가 일치하는지 확인해 줍니다.
assertSame(A, B)	객체 A와 B가 같은 객체인지 확인해 줍니다.
assertTrue(A)	조건 A가 참인지 확인해 줍니다.
assertNull(A)	객체 A가 Null인지 확인해 줍니다. ↔ assertNotNull(A)

assertEquals 오류 예시

```
org.opentest4j.AssertionFailedError:
```

```
Expected : 하하하
```

```
Actual   : 호호호
```

```
<Click to see difference> ← 두 데이터의 차이점을 확인할 수 있습니다.
```

제이유닛(JUnit)

Java 프로그래밍 언어를 위한 테스트 프레임워크로, 단위 테스트를 간단히 작성하고 실행하는 데 사용됩니다.

build.gradle

JUnit 5에서는 `@Test` 어노테이션을 사용하기 위해서는 JUnit의 의존성을 추가해야 합니다.

하지만, 특정 IDE(예: IntelliJ IDEA)에서는 JUnit 라이브러리를 자동으로 포함해 주는 경우가 있습니다.

따라서 프로젝트의 `build.gradle` 파일에 명시적으로 JUnit 코드를 작성하지 않더라도 IDE가 JUnit 라이브러리를 제공해 주기 때문에 `@Test` 어노테이션을 사용할 수 있는 것입니다.

```
dependencies {  
    // 다른 의존성들...  
    implementation 'org.springframework.boot:spring-boot-starter-test' // for @SpringBootTest Annotation  
    implementation 'org.junit.jupiter:junit-jupiter:5.10.0' // JUnit 5  
}  
  
tasks.named('test') {  
    useJUnitPlatform()  
}
```

Spring Boot

Spring DI



Spring



Boot



Security

의존(Dependency)

하나의 객체가 다른 객체를 사용하거나, 다른 객체에 의존하여 동작하는 상태를 의미합니다.

의존(Dependency)



발생 가능한 시점

Person 객체가 Avante 객체를 인스턴스 변수로 가지는 경우

Person 객체가 Avante 객체가 가진 메소드를 호출하여 기능을 수행하고자 하는 경우

Person 객체가 Avante 객체를 외부에서 데이터를 받아오거나 리소스를 사용하는 경우

소스 코드 예시

다음 예시는 Person 클래스가 Avante 클래스의 객체 정보를 필요로 하는, 즉 의존하는 예시입니다.

```
public class Person {  
    private Avante avante ;  
  
    public Person() {  
        this.avante = new Avante() ;  
    }  
}
```

주입(Injection)

하나의 객체가 다른 객체 또는 값을 생성자 또는 setter 메소드를 사용하여 입력 받는 것을 의미합니다.

주입(Injection)

다음 예시는 Person 클래스 내의 멤버 변수들을 생성자를 통하여 외부에서 주입 받고 있는 예시입니다.

```
public class CarMain01 {  
    public static void main(String[] args) {  
        // 생성자를 이용하여 Constructor Injection을 수행하고 있습니다.  
        Person bean = new Person("김철수", 25, "남자");  
        System.out.println(bean.toString());  
    }  
}
```

```
@ToString  
public class Person {  
    private String name ;  
    private int age ;  
    private String gender ;  
  
    public Person(String name, int age, String gender) {  
        this.name = name ;  
        this.age = age ;  
        this.gender = gender ;  
    }  
}
```

의존성 주입(Dependency Injection)

객체 간의 의존 관계를 느슨하게 만들어 코드의 유연성과 테스트 용이성을 향상시키는 디자인 패턴입니다.

의존성은 일반적으로 클래스 간의 관계를 설명할 때 사용되며, 객체 지향 프로그래밍에서 중요한 개념 중 하나입니다.

객체가 필요로 하는 의존성을 외부에서 주입 받는 방식으로 다음과 같은 주입 방식이 존재합니다.

의존성 주입의 기대 효과

결합도 감소(Loose Coupling)

코드의 유연성과 재사용성 증가

단위 테스트 용이성

코드 가독성 향상

특정 구현체에 종속되지 않는 코드 작성

주입의 종류

항목	설명
생성자 주입(Constructor Injection)	클래스의 생성자의 매개 변수로 의존성 정보를 전달시켜 객체를 생성합니다.
세터 주입(Setter Injection)	클래스의 setter() 메소드를 통해 의존성을 주입하는 방식입니다.
필드 주입(Field Injection)	@Autowired 어노테이션을 사용하여 멤버 변수에 직접 의존성을 주입하는 방식입니다.

강한 결합과 약한 결합

강한 결합과 약한 결합은 객체 지향 프로그래밍에서 객체 간의 의존성 관계의 정도를 나타내는 개념입니다.

객체의 의존 관계에서 "강한 결합"이란, 어떠한 객체가 다른 객체에 직접적으로 의존하는 경우를 의미합니다.

즉, 의존 관계에 대한 코딩을 변경하면 의존 중인 클래스 역시 코딩 변경이 이루어져야 합니다.

클래스에 어떤 변동 사항이 생기면, 코드를 수정해야 하는 범위가 넓어질 가능성이 매우 높습니다.

강한 결합의 문제점

코드의 재사용성 저하

유지 보수성 저하

코드의 유연성과 확장성 제한

강한 결합 예시

차량 시승을 예로 들어 봅니다.

Person 클래스의 멤버 변수 타입으로 Avante 클래스가 존재하는 것을 확인할 수 있습니다.

여기서 문제는 다음과 같습니다.

Avante 클래스가 없으면 Person 클래스를 정의할 수 없습니다.

따라서, "Person 클래스가 Avante 클래스에 의존성이 존재"함을 알 수 있습니다.

소스 코드

```
public class Person {  
    private Avante avante ;  
  
    public Person() {  
        this.avante = new Avante() ;  
    }  
  
    public drive() {  
        this.avante.drive();  
    }  
}
```

강한 결합과 약한 결합

강한 결합과 약한 결합은 객체 지향 프로그래밍에서 객체 간의 의존성 관계의 정도를 나타내는 개념입니다.

강한 결합 예시

Person 클래스 안의 Avante 클래스를 Sonata 클래스로 변경하는 경우를 생각해 봅시다.

예시에서는 3군데 코드를 수정해야 합니다.

코드의 변화 내용을 살펴 보면, 멤버 변수 타입(클래스) 하나만 바꿨을 뿐인데 Person 클래스에 있는 대다수의 코드가 변경됩니다.

그렇기 때문에 "강한 결합"(강한 의존성)이라고 하는 것이며, 이는 유지 보수를 하는 측면에서 굉장히 좋지 않은 케이스라고 할 수 있습니다.

```
public class Person {  
    private Avante avante ;  
  
    public Person() {  
        this.avante = new Avante() ;  
    }  
  
    public drive() {  
        this.avante.drive();  
    }  
}
```



```
public class Person {  
    private Sonata sonata ;  
  
    public Person() {  
        this.sonata = new Sonata() ;  
    }  
  
    public drive() {  
        this.sonata.drive();  
    }  
}
```

인터페이스를 활용한 의존 관계 설정

인터페이스를 사용하는 프로그래밍을 이용하여 의존 관계를 설정해 봅니다.

약한 결합을 위해서 인터페이스를 사용해야 하는 이유는 다음과 같습니다.

항목	설명
추상화와 구체화 분리	인터페이스는 객체의 동작을 정의하는 추상화된 형태로써, 구체적인 구현 세부 사항과는 분리되어 있습니다. 인터페이스를 사용하면 특정 객체가 다른 객체에 대해 필요로 하는 동작을 정의할 수 있습니다. 객체 간의 상호 작용은 인터페이스를 통해 이루어집니다.
유연성과 확장성	인터페이스를 사용하면 구체적인 구현에 대한 의존성을 최소화할 수 있습니다. 객체가 인터페이스에 의존하면서, 실제로 사용할 구현체를 교체하거나 다양한 구현체를 사용할 수 있습니다. 이는 시스템의 유연성과 확장성을 높이는데 도움을 줍니다. 구현체 클래스 생성, 기존 구현체 코드 변경에도, 인터페이스를 사용하는 객체에는 영향을 주지 않고 상호 작용할 수 있습니다.
다형성과 대체 가능성	인터페이스를 사용하면 다형성을 활용할 수 있습니다. 여러 개의 구현체가 동일한 인터페이스를 구현할 수 있으므로, 객체 간의 대체 가능성을 제공합니다. 이는 코드의 재사용성과 유지 보수성을 향상시킵니다. Avante 대신 Sonata 사용이 가능했었습니다.
단위 테스트 용이성	인터페이스를 사용하면 단위 테스트 작성이 용이해집니다. 의존성을 인터페이스로 주입하면, 테스트 시에는 모킹(mocking)된 구현체를 주입하여 테스트할 수 있습니다. 이는 의존성을 격리시켜 테스트의 의도를 명확히 할 수 있게 합니다.

인터페이스를 활용한 의존 관계 설정

인터페이스를 사용하는 프로그래밍을 이용하여 의존 관계를 설정해 봅니다.

자바의 인터페이스는 구현체 클래스들이 준수해야 할 강제적인 규칙을 정의하는 역할을 합니다.

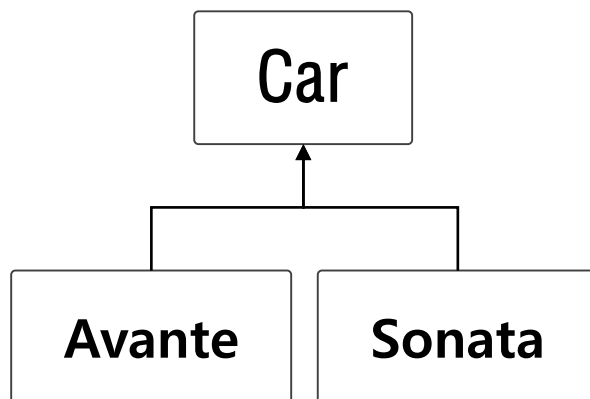
차량 시승 예시에서 차량 유형은 2가지(Avante, Sonata)가 있다고 가정합니다.

이런 경우 해당 클래스(Avante, Sonata)들이 의존 관계를 구현하는 인터페이스(Car)에 의존한다면, 해당 의존 관계의 구현을 변경하더라도 의존 중인 클래스를 변경할 일이 없습니다.

이러한 설계 기법을 "인터페이스를 사용하는 프로그래밍"이라고 합니다.

이러한 접근 방법은 해당 인터페이스와 의존 중인 클래스와 의존 관계 사이에 약한 결합(느슨한 결합)을 만들어 줍니다.

의존 관계 클래스가 구현하는 인터페이스를 "의존 관계 인터페이스"라고 합니다.



인터페이스를 활용한 의존 관계 설정

인터페이스를 사용하는 프로그래밍을 이용하여 의존 관계를 설정해 봅니다.

약한 결합은 객체 간의 의존성을 최소화하여 독립성과 유연성을 높이는 것을 의미합니다.

객체에 대한 의존성은 인터페이스나 추상 클래스에 의존하며, 구체적인 구현에 대한 의존성은 최대한 제거됩니다.

이렇게 함으로써, 코드의 재사용성과 유지 보수성을 향상시키는데 도움을 줍니다.

약한 결합은 객체들을 독립적으로 변경하고 재구성할 수 있게 해주며, 유연하고 확장 가능한 시스템을 구축하는데 중요한 역할을 합니다.

약한 결합이 되면

객체 간의 독립성과 유연성과 좋아짐
코드의 재사용성과 유지 보수성 양호
코드의 유연성과 확장성이 좋아짐

```
public class Person {  
    private Car car ;  
  
    public Person(Car car) {  
        this.car = car ;  
    }  
  
    public drive() {  
        this.car.drive();  
    }  
}
```



```
Car car = new Avante();  
또는  
Car car = new Sonata();  
  
Person person = new Person(car) ;
```


스프링을 활용한 의존 관계 설정

스프링을 활용한 의존 관계 설정 방법에 대하여 살펴 봅니다.

어노테이션(Annotation)

우선 어노테이션이 무엇인지 살펴 보도록 하겠습니다.

Spring에서 어노테이션(Annotation)은 메타 데이터를 제공하는 데 사용이 됩니다.

즉, 코드에 대한 추가 정보를 제공하고, Spring이 해당 코드를 어떻게 처리해야 하는지 알려 줍니다.

Spring에서 가장 일반적으로 사용되는 어노테이션은 다음과 같습니다.

이 외에도 다양한 어노테이션이 존재하며, 이를 통해 Spring 애플리케이션의 구조와 동작을 세밀하게 제어할 수 있습니다.

파일	설명
@Component	해당 클래스가 Spring의 컴포넌트임을 나타냅니다.
@Controller	해당 클래스가 Spring의 컨트롤러임을 나타냅니다.
@Service	해당 클래스가 Spring의 서비스임을 나타냅니다.
@Repository	해당 클래스가 Spring의 레파지토리(데이터 액세스 계층)임을 나타냅니다.
@Autowired	해당 필드나 생성자 파라미터 등이 자동으로 주입되어야 함을 나타냅니다.
@RequestMapping	해당 메소드가 어떤 요청 경로와 매핑되는지 나타냅니다.

스프링을 활용한 의존 관계 설정

`@Bean`은 명시적으로 스프링 컨테이너에 빈(Beans)으로 등록시켜 주는 어노테이션입니다.

스프링이 해당 어노테이션을 찾아서 bean 객체로 생성해 줍니다.

항목	설명
목적	명시된 메소드의 리턴 값을 빈 객체로 생성해줍니다.
설정 위치	메소드 선언 부
특징	이 어노테이션을 포함하는 클래스에는 @Configuration 어노테이션이 반드시 명시되어 있어야 합니다.

사용 예시

case01) 메소드 이름이 객체 이름이 되는 경우

```
Member hohoho = new Member("김철수", 20);
```

```
@Configuration
public class SomeConfig{
    @Bean
    public Member hohoho() {
        return new Member("김철수", 20);
    }
}
```

case02) name 속성의 값이 객체 이름이 되는 경우

```
Member hahaha = new Member("김철수", 20);
```

```
@Configuration
public class SomeConfig{
    @Bean(name="hahaha")
    public Member hohoho() {
        return new Member("김철수", 20);
    }
}
```

스프링을 활용한 의존 관계 설정

`@Configuration`은 Java 기반의 설정 클래스를 선언하고자 할 때 사용되는 어노테이션입니다.

`@Configuration` 어노테이션이 명시된 클래스는 스프링의 설정 정보를 포함하고 있는 것으로 인식됩니다.

"Java Config"란 스프링 프레임워크에서 제공하는 자바 기반의 설정 방식을 의미합니다.

이전에 사용한 XML 기반의 설정 파일을 대체하는 형태로 사용되는 기법입니다.

이를 통해 코드 기반의 설정을 사용하여 스프링 애플리케이션을 구성할 수 있습니다.

어노테이션	설명
의미	어떠한 클래스에 이 어노테이션을 명시하게 되면 해당 클래스 파일을 설정 정보 파일로 사용하겠습니다.'라는 의미가 됩니다.
설정 위치	클래스 선언 부

주된 용도

항목	설명
Bean 정의	클래스 내부에서 <code>@Bean</code> 어노테이션을 사용하여 스프링 빈을 정의할 수 있습니다. <code>@Bean</code> 어노테이션으로 생성된 객체는 스프링 컨테이너가 빈으로 등록시킵니다.
외부 설정	스프링 설정 파일로서의 역할을 수행할 수 있습니다. <code>@PropertySource</code> 를 이용하여 외부 프로퍼티 파일을 로드할 수 있고, <code>@Value</code> 어노테이션을 사용하여 프로퍼티 값을 주입할 수 있습니다.
조건부 설정	<code>@Conditional</code> 어노테이션과 함께 사용하여 조건에 따라 설정을 적용하거나 제외할 수 있습니다. 이를 통해 특정 조건에 따라 동적으로 설정을 변경할 수 있습니다.

스프링을 활용한 의존 관계 설정

@Configuration은 Java 기반의 설정 클래스를 선언하고자 할 때 사용되는 어노테이션입니다.

사용 예시

다음 예시는 Avante 객체 avante01을 생성하는 구문입니다.

```
Car avante01 = new Avante("아반떼 빨간차", 111, "외관이 이뻐요^^");
```

라는 구문으로 객체 생성을 하고, 스프링이 avante01 객체를 관리합니다.

동일한 방식으로 carlist라는 리스트 컬렉션에 여러 대의 자동차가 포함됩니다.

```
@Configuration
public class CarConfig {

    @Bean("avante01")
    public Car makeAvante01(){
        Car mycar = new Avante("아반떼 빨간차", 111, "외관이 이뻐요^^");
        return mycar;
    }

    @Bean("carlist")
    public List<Car> carList(){
        List<Car> lists = new ArrayList<>();
        lists.add(this.makeAvante01());
        lists.add(this.makeSonata01());
        lists.add(this.makeAvante02());
        return lists;
    }
}
```

Spring Boot

Spring AOP



Spring



Boot



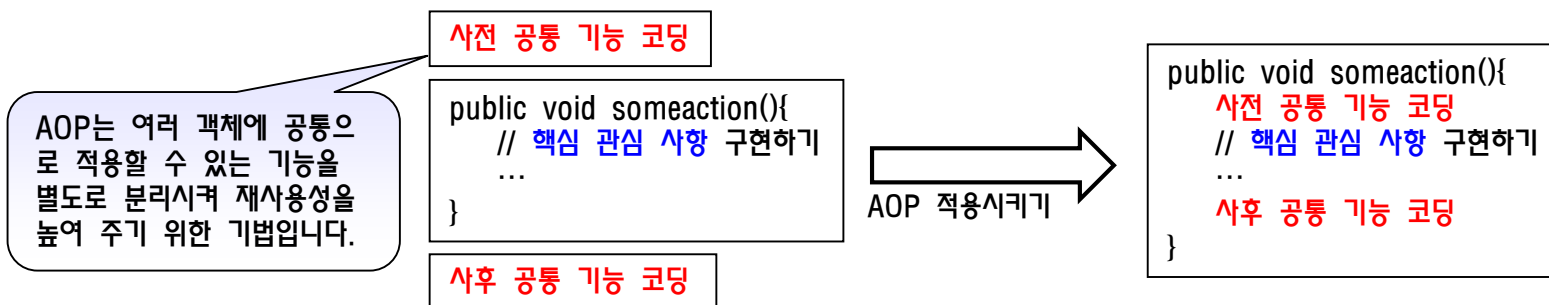
Security

Spring AOP

AOP(Aspect-Oriented Programming)

소프트웨어 개발에서 비즈니스 로직과 횡단 관심사를 분리하기 위한 프로그래밍 패러다임입니다.

문제를 해결하기 위한 핵심 관심 사항(core concern, 핵심 로직)과 전체에 적용되는 공통 관심 사항(cross-cutting concern, 공통 기능)을 기준으로 프로그래밍함으로써 공통 모듈을 여러 코드에 쉽게 적용할 수 있도록 도와 줍니다.



스프링 AOP의 특징

- [공통 관심 사항]을 구현한 코드를 [핵심 관심 사항]을 구현한 코드 안에 자동으로 삽입합니다.
- [핵심 관심 사항]을 구현한 코드에서 [공통 관심 사항]을 직접적으로 호출하지 않습니다.
- [핵심 관심 사항]을 구현한 코드를 컴파일하거나, 컴파일된 클래스를 로딩하거나, 로딩된 클래스의 객체를 생성할 때 AOP가 적용되어 핵심 로직 구현 코드 안에 [공통 관심 사항]이 삽입됩니다.
- [공통 관심 사항]이 변경되더라도 [핵심 관심 사항]을 구현한 코드를 변경할 필요가 없습니다.
- [공통 관심 사항] 코드를 변경한 뒤 [핵심 관심 사항] 구현 코드에 적용만 하면 됩니다.

Spring AOP

AOP 용어

AOP와 관련된 용어들에 대하여 살펴 보겠습니다.

항목	설명
Core-concerns	핵심 관심 사항 (상품 주문 하기, 게시물 등록하기 등등)입니다. 즉, 핵심적인 비즈니스 로직이 여기에 해당됩니다.
Aspect	여러 객체에 공통으로 적용되는 공통 관심 사항(Cross-cutting-concerns) 을 의미합니다. 예를 들어 [로그인 검사]나 [사용 권한 검사] logic 등이 여기에 속합니다. 즉 여러 개의 클래스에서 공통적으로 사용되는 소스 모듈 을 의미합니다.
Advice	Aspect가 수행해야 할 어떠한 작업 목록을 의미합니다. Aspect(공통 관심 사항)을 언제 (~하기 전에 ,~한 직후에) 핵심 로직에 적용시킬 것인가 ?
Joinpoint	Advice를 적용할 수 있는 모든 요소들을 의미합니다. 메소드의 호출, 변수(필드) 값의 수정, 예외의 발생 등이 Joinpoint에 해당될 수 있습니다. 각각의 구체적인 적용 지점을 Pointcut 이라고 합니다.
Pointcut	Joinpoint를 구성하는 각각의 구체적인 적용 지점(메소드 실행, 예외 처리, 속성 변경)을 말합니다. 스프링에서는 AspectJ Pointcut Expression(표현식)을 사용하여 Pointcut을 표현 합니다. 스프링에서 핵심 관심 사항을 표현한 클래스(Target Object) 내의 특정 메소드를 표현합니다. Aspect가 Advice할 Joinpoint의 영역을 좁히는 일을 말합니다.
Weaving	Advice를 핵심 로직 코드에 적용하는 것을 의미합니다. 공통 코드를 핵심 로직 코드에 삽입하는 것을 의미합니다.

Spring AOP

JoinPoint와 PointCut

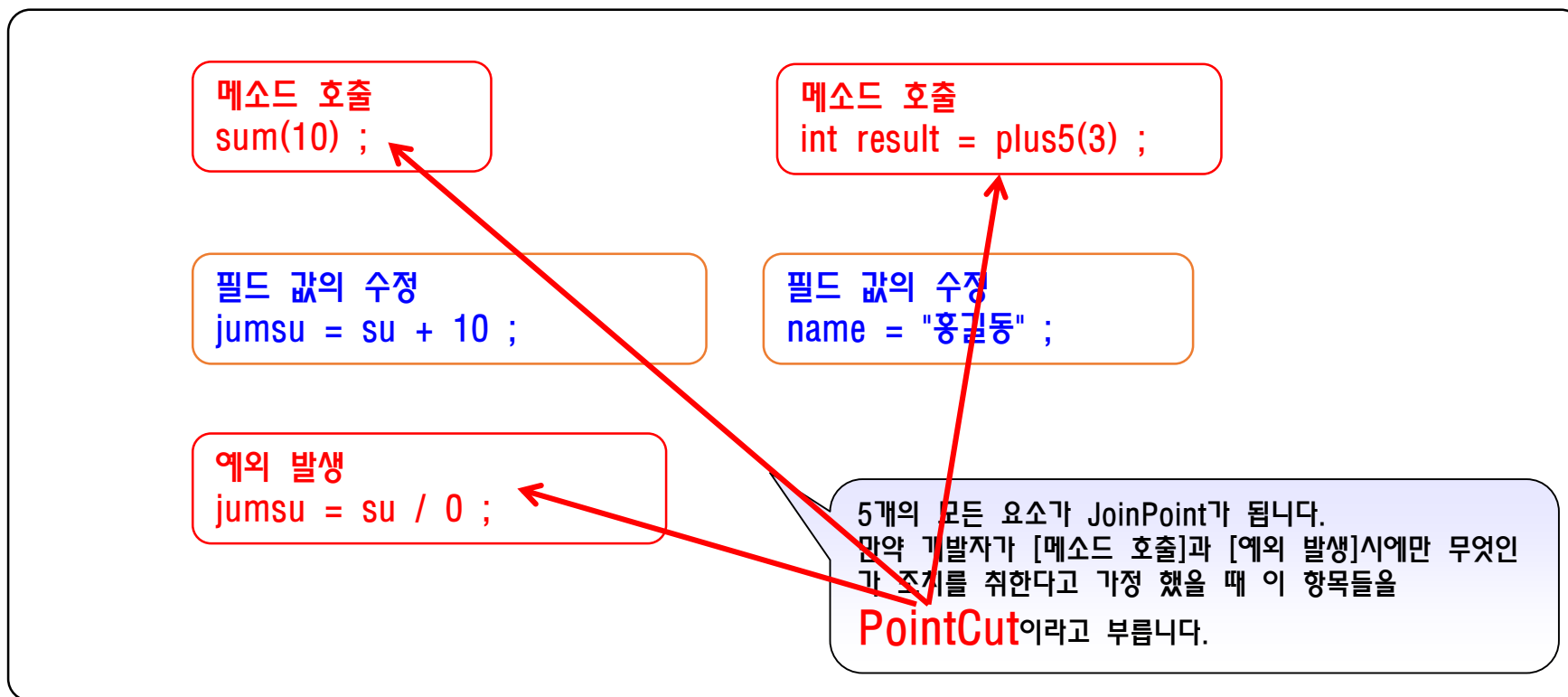
JoinPoint와 PointCut 용어에 대하여 살펴 보겠습니다.

Joinpoint란 Advice를 적용할 수 있는 모든 요소들을 의미합니다.

메소드의 호출, 변수(필드) 값의 수정, 예외의 발생 등이 Joinpoint에 해당될 수 있습니다.

각각의 구체적으로 적용하고자 하는 지점을 Pointcut이라고 부릅니다.

그림에서 Joinpoint는 총 5개가 되고, Pointcut은 3개가 됩니다.



Spring AOP

AspectJ Pointcut Expression

AspectJ는 Pointcut을 명시할 수 있는 다양한 명시자를 제공해 주고 있습니다.

항목	설명
execution 명시자	Advice를 적용할 메소드를 명시할 때 사용합니다.
기본 형식	execution(수식어패턴? 리턴타입패턴 패키지패턴?이름패턴 (파라미터패턴) 수식어_패턴) 생략이 가능한 부분입니다.(public, protected 등이 올 수 있습니다.) 리턴_타입_패턴) 리턴 타입을 명시합니다. 클래스_이름_패턴, 이름_패턴) 클래스 이름 및 메소드 이름을 패턴으로 명시합니다. 파라미터_패턴) 매칭될 파라미터에 대해서 명시합니다.
특징	각 패턴은 '*'를 이용하여 모든 값을 표현할 수 있습니다. '..'의 의미는 0개 이상이라는 의미입니다.

항목	설명
execution(public void set*(..))	리턴 타입이 void이고 메소드 이름이 set으로 시작하고, 파라미터가 0개 이상인 메소드를 호출합니다.
execution(* mypkg.core.*.*())	mypkg.core 패키지의 파라미터가 없는 모든 메소드를 호출합니다.
execution(*.mypkg.core..*.*(..))	mypkg.core 패키지, 하위 패키지에 있는 파라미터가 0개 이상인 메소드를 호출합니다.
execution(Integer mypkg.WriteBoard.write(..))	리턴 타입이 Integer인 WriteBoard 인터페이스의 write() 메소드를 호출합니다.
execution(* get*(*))	이름이 get으로 시작하고 1개의 파라미터를 갖는 메소드를 호출합니다.
execution(* get*(*, *))	이름이 get으로 시작하고 2개의 파라미터를 갖는 메소드를 호출합니다.
execution(* read*(Integer, ..))	메소드 이름이 read로 시작하고, 첫 번째 파라미터 타입이 Integer이며, 1개 이상의 파라미터를 갖는 메소드를 호출합니다.

Spring AOP

@Component

개발자가 직접 정의한 클래스를 스프링 빈(Spring Bean)으로 자동 등록하는 데 사용됩니다.

@Component is an annotation to register objects in Spring.

```
Animal a = new Animal();  
Animal b = new Animal();
```

@Component

```
public class Animal {  
}
```

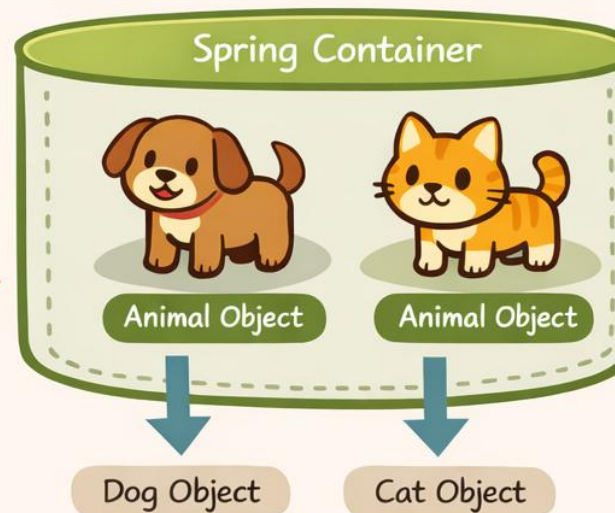


a = new Animal()



b = new Animal()

Automatically
Registers!



In Java, when you create objects, new **Animal** objects are created for 'a' and 'b' separately.

Using **@Component**, Spring automatically creates and registers "Animal" objects in the container.

Spring AOP

@Component

개발자가 직접 정의한 클래스를 스프링 빈(Spring Bean)으로 자동 등록하는 데 사용됩니다.

주요 특징

항목	설명
빈 등록의 핵심 어노테이션	스프링이 자동으로 빈을 생성하고 관리할 수 있도록 알려 주는 역할을 합니다. 빈으로 등록된 클래스는 스프링의 컨테이너에 의해 관리됩니다.
DI(의존성 주입) 지원	@Component로 등록된 클래스를 다른 빈이 필요로 하는 경우, 스프링 컨테이너가 이를 자동으로 주입해 줍니다.
클래스 스캔을 통한 빈 등록	스프링의 컴포넌트 스캔(Component Scan)에 의해 탐지됩니다. @SpringBootApplication, @Configuration과 같은 어노테이션에 포함된 @ComponentScan이 클래스 경로에서 이 어노테이션을 감지합니다.
스테레오타입 어노테이션	스프링의 관리 하에 빈으로 등록되어야 한다는 것을 명시하는 가장 일반적인 스테레오타입 어노테이션입니다. 이 외에도 다음과 같은 항목들이 존재합니다. * @Service: 비즈니스 로직을 구현하는 클래스에 사용 * @Repository: 데이터 액세스 계층을 나타내는 클래스에 사용 * @Controller: @Component에 "웹 요청 처리" 기능이 추가된 어노테이션
커스터마이징 가능	빈의 이름을 직접 지정할 수 있습니다. 기본적으로 클래스명 첫 글자를 소문자로 변환한 이름으로 빈이 등록되지만, 다음과 같이 이름을 명시적으로 설정할 수 있습니다. 예시 @Component("customBeanName") public class MyComponent { // ... }

@Aspect을 이용한 AOP 설정

어노테이션 @Aspect을 이용한 AOP 설정 방법입니다.

클래스가 AOP의 한 단위인 `Aspect`임을 선언합니다.

항목	설명
@After	After Advice로 사용할 메소드에 적용하는 어노테이션입니다. 대상 객체의 메소드를 실행하는 도중에 예외가 발생했는지의 여부와 상관 없이 메소드 실행 후 의 공통 기능을 실행합니다. try ... catch ... finally 절의 finally 블록과 비슷한 개념입니다.
@Before	Before Advice를 구현할 때에 사용되는 어노테이션입니다. 대상 객체의 메소드 호출 전 에 공통 기능을 실행하는 어노테이션입니다. AspectJ의 Pointcut 표현식이나 @Pointcut 어노테이션이 적용된 메소드 이름이 올 수 있습니다. JoinPoint 타입의 파라미터를 첫 번째 파라미터로 가질 수 있습니다.
@Around	Around Advice를 구현할 때에 사용되는 어노테이션입니다. 대상 객체의 메소드 실행 전, 후 또는 예외 발생 시점 에 공통 기능을 실행하는 어노테이션입니다. 주의 사항 Around Advice를 구현한 메소드는 org.aspectj.lang.ProceedingJoinPoint를 반드시 첫 번째 파라미터로 지정해야 합니다. 그렇지 않을 경우 예외가 발생합니다.

Spring AOP

@Aspect을 이용한 AOP 설정

어노테이션 @Aspect을 이용한 AOP 설정 방법입니다.

항목	설명
@AfterThrowing	After Throwing Advice로 사용할 메소드에 적용하는 어노테이션입니다. 대상 객체의 메소드를 실행하는 도중 예외가 발생한 경우에 공통 기능을 실행합니다. Advice를 구현한 메소드 내에서 Advice 대상 메소드가 발생시킨 예외 객체에 접근하려면 throwing속성을 이용해서 예외 객체를 전달 받을 파라미터의 이름을 지정해주면 됩니다. 특정 타입의 예외에 대해서만 처리하고 싶다면, Throwable이나 Exception이 아닌 처리하고 싶은 타입을 파라미터로 지정하면 됩니다. 예를 들어서 수행하고자 하는 파일이 발견되지 않는 경우에만 처리하고 싶다면 public void afterThrowing(FileNotFoundException ex){ ... } 의 형태로 처리하면 됩니다.
@AfterReturning	After Returning Advice로 사용할 메소드에 적용하는 어노테이션입니다. 대상 객체의 메소드가 예외 없이 실행한 이후에 공통 기능을 실행하고자 하는 경우에 사용하는 어노테이션입니다.
참고 사항	@Around 어노테이션을 제외한 나머지 Advice를 표시하는 어노테이션들에 대하여 대상 객체 및 호출되는 메소드에 대한 정보나 전달되는 파라미터에 대한 정보가 필요한 경우 org.aspectj.lang.JoinPoint를 파라미터로 명시하면 됩니다.

Spring Boot

Lombok 라이브러리



Spring



Boot

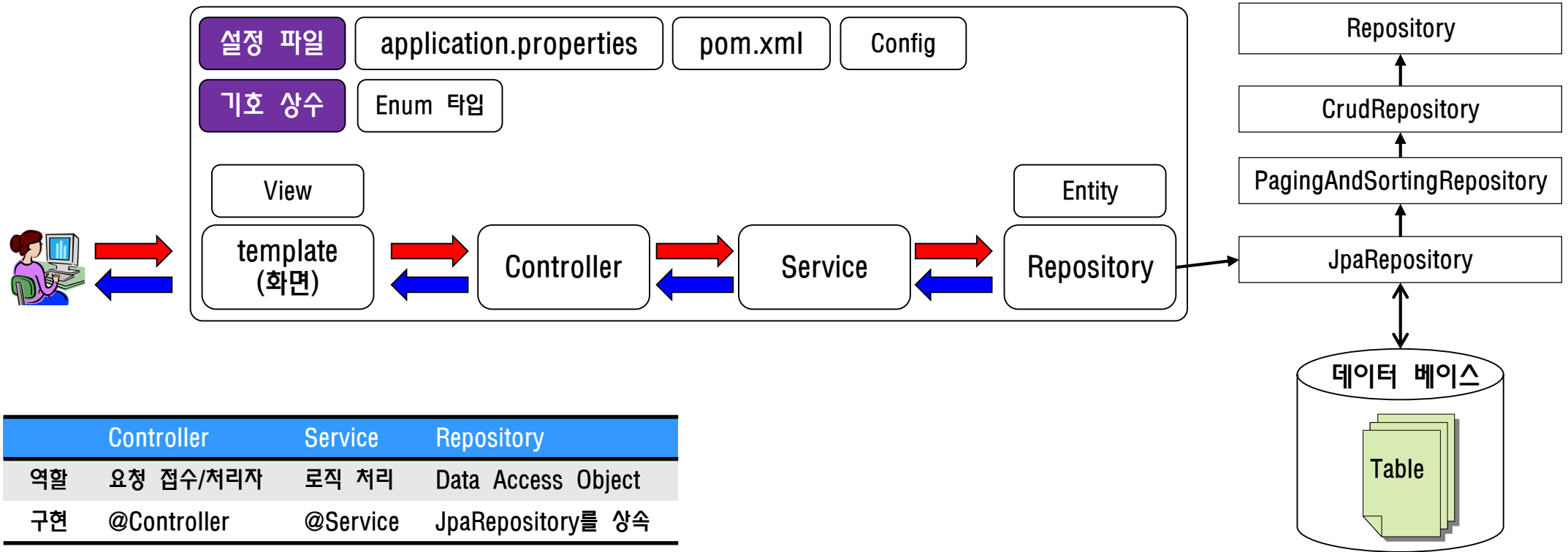


Security

≡ Lombok 라이브러리

≡ 프로젝트 구성 요소

스프링 부트와 연관된 프로젝트 구성 요소는 다음과 같은 항목들이 존재합니다.



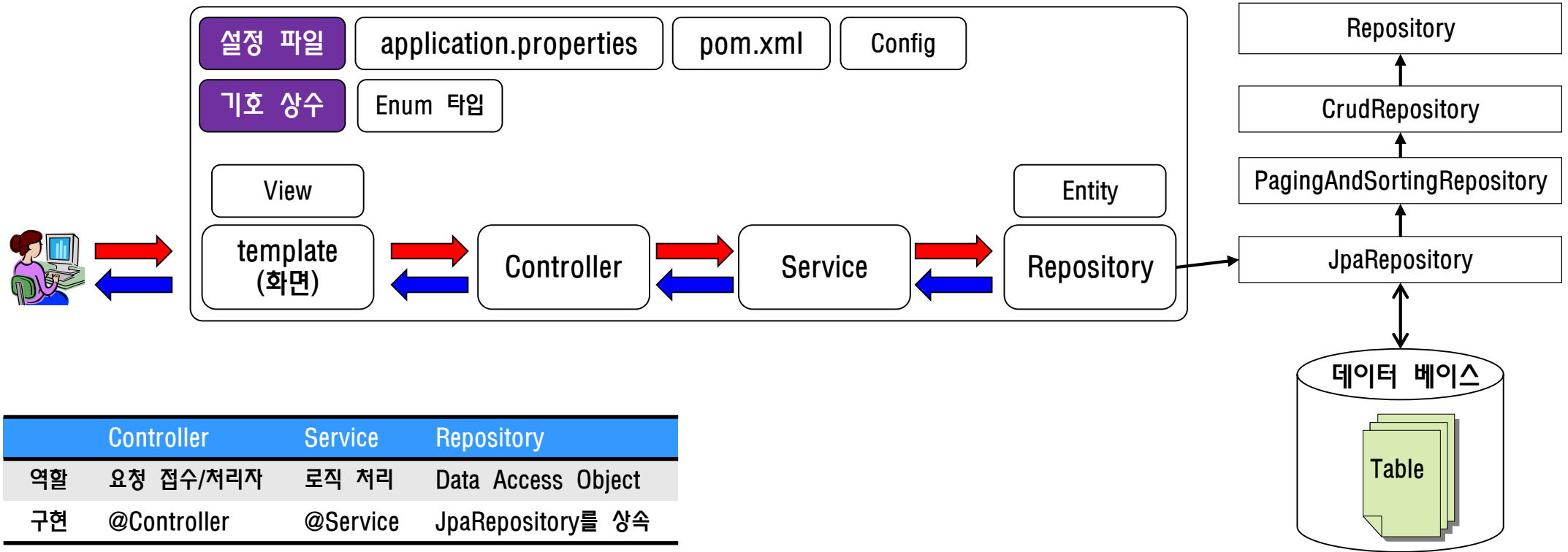
	Controller	Service	Repository
역할	요청 접수/처리자	로직 처리	Data Access Object
구현	@Controller	@Service	JpaRepository를 상속

	Entity	View	Config	Enum 타입	application.properties	pom.xml
역할	JPA와 연동	화면(form)과 연동	자바로 만든 설정 파일	기호 상수	서비스에 필요한 설정 파일	메이븐 용 설정 파일

≡ Lombok 라이브러리

≡ 프로젝트 구성 요소

스프링 부트와 연관된 프로젝트 구성 요소는 다음과 같은 항목들이 존재합니다.



	Controller	Service	Repository
역할	요청 접수/처리자	로직 처리	Data Access Object
구현	@Controller	@Service	JpaRepository를 상속

	Entity	View	Config	Enum 타입	application.properties	pom.xml
역할	JPA와 연동	화면(form)과 연동	자바로 만든 설정 파일	기호 상수	서비스에 필요한 설정 파일	메이븐 용 설정 파일

어노테이션(`@Controller)

특정 웹 페이지에 대한 요청과 응답을 처리해주는 역할을 수행합니다.

Lombok 라이브러리에 대한 실습을 수행하기 전에 우선 @Controller 어노테이션을 살펴 보도록 하겠습니다.

항목	설명
목적	MVC 패턴에서 Controller 역할을 합니다. 사용자의 웹 요청(request)과 응답(response)을 처리한 다음 지정된 뷰에 모델 객체를 넘겨주는 역할을 수행합니다. 프레젠테이션(Presentation) 레이어 역할을 합니다.
설정 위치	클래스 선언부
역할	주로 로직을 처리해주는 Service 객체가 반환해준 결과물을 Client에게 다시 전달해 줍니다.

≡ 어노테이션 `@RestController`

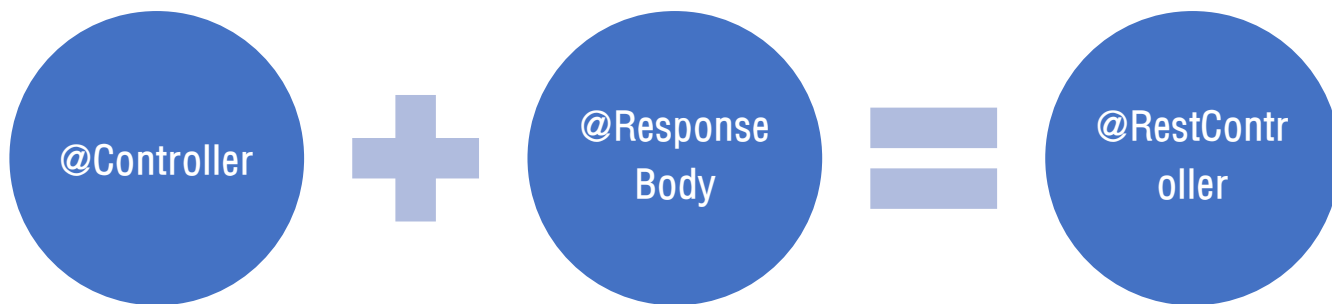
@RestController는 스프링 프레임워크에서 제공하는 어노테이션 중 하나로, RESTful 웹 서비스를 개발할 때 사용됩니다.

이 어노테이션은 컨트롤러 클래스에 사용되며, 해당 클래스가 REST API의 엔드 포인트를 제공하는 것을 나타냅니다.

@RestController 어노테이션이 붙은 클래스는 JSON, XML 등의 형식으로 데이터를 반환할 수 있으며, 일반적으로 HTTP 요청 메소드(GET, POST, PUT, DELETE 등)와 매핑되는 메소드를 정의합니다.

@RestController 어노테이션은 @Controller와 @ResponseBody 어노테이션을 조합한 것으로 볼 수 있습니다.

이전에는 @Controller와 @ResponseBody 어노테이션을 함께 사용하여 RESTful 웹 서비스를 개발했지만, @RestController 어노테이션을 사용하면 이 두 어노테이션을 대신할 수 있습니다.



≡ 어노테이션 @GetMapping

http 요청 방식 중에서 **get 방식**에 대한 요청을 처리해주는 어노테이션입니다.

항목	설명
설정 위치	메소드 선언 부

Lombok 라이브러리에 대한 실습을 수행하기 전에 우선 @GetMapping 어노테이션을 살펴 보도록 하겠습니다.

@GetMapping 어노테이션은 HTTP GET 요청을 처리해주는 메소드를 맵핑(@RequestMapping) 시켜 주는 어노테이션입니다.

다음 예시에서 test() 메소드는 Get 방식으로 <http://localhost:8989/fruit/>를 요청했을 때 실행되는 컨트롤러 메소드입니다.

≡ `@PathVariable`

웹 서비스 구축시 URI 경로의 변수 값을 메소드 파라미터로 매핑하는 데 사용됩니다.

클라이언트가 전달한 URL 경로의 특정 부분을 파라미터로 받아 사용할 수 있습니다.

`@RequestMapping`이나 `@GetMapping`, `@PostMapping`과 같은 핸들러 메소드와 함께 자주 사용됩니다.

항목	설명
설정 위치	클래스 선언 부
특징	경로 변수와 메소드 파라미터 이름이 같으면 이름을 생략할 수 있으며, 필요시 이름을 명시할 수 있습니다. 여러 개의 경로 변수를 사용하거나 `required` 속성을 조정할 수 있습니다. 경로 변수 값은 자동으로 타입 변환이 되며, Spring이 인코딩을 처리해줍니다.
속성 이름	설명
value	URI에서 어떤 부분을 메소드 파라미터로 매핑할지를 지정합니다.
required	해당 경로 변수가 필수인지 여부를 결정하는 데 사용됩니다. 값이 True이면 반드시 입력해 주어야 하는 필수 속성입니다.

```
@RestController
@RequestMapping("/api/books")
public class BookController {
    @GetMapping("/{id}")
    public Book getBookById(@PathVariable("id") Long bookId) {
        return bookService.findById(bookId);
    }
}
```

☰ Lombok 라이브러리

☰ Lombok 라이브러리

컴파일시 getter/setter/toString() 등의 반복적인 코드를 자동으로 생성해주는 라이브러리입니다.

Lombok 이란?

Java 기반의 프로그램에서 Bean, VO(Value Object), DTO(Data Transfer Object) 등을 보다 쉽게 작성하기 위해 사용되는 라이브러리입니다.

Getter, Setter, ToString 등을 자동으로 생성해주므로 코드에 대한 가독성이 좋습니다.

여러 명이 개발 중인 경우 모든 팀원들이 lombok을 반드시 설치해야 합니다.

```
package com.shopping.view;

import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class SomeExample {
    private Long id;
    private String imageUrl;
    private Integer price;

    // getter, setter 구현하지 않아도 자동으로 생성됩니다.
}
```

≡ Lombok 라이브러리

≡ Lombok 라이브러리

컴파일시 getter/setter/toString() 등의 반복적인 코드를 자동으로 생성해주는 라이브러리입니다.

관련 Annotation

자료형	설명
@Getter/@Setter	해당 클래스에 선언된 변수들의 getter, setter 메소드를 생성해줍니다. 생성된 메소드들은 기본적으로 public입니다. AccessLevel를 통하여 접근제한자를 설정해줄 수 있습니다. 예시) @Setter(AccessLevel.PRIVATE)
@ToString	toString() 메소드를 자동으로 생성합니다. exclude 매개 변수를 통하여 제외할 필드를 지정할 수 있습니다. @ToString(exclude = "member") java.lang.StackOverflowError 발생시 exclude 매개 변수를 이용하여 예외를 발생시키지 않도록 합니다. callSuper 매개 변수 값을 true로 할 경우 상속 받은 클래스의 정보까지 출력이 됩니다.(Default = false)
@NonNull	변수에 선언을 해주면, 해당값은 반드시 값이 입력되어야 합니다. Setter에 null값이 들어오면 NullPointerException예외를 일으킵니다.
@NoArgsConstructor	parameter가 없는 기본 생성자를 생성해 줍니다. AccessLevel을 통하여 접근제한자를 지정할수 있습니다.
@EqualsAndHashCode	equals()와 hashCode() 메소드를 생성해 줍니다.

주의 사항

@Getter/@Setter을 사용하게 되면 굳이 구현할 필요가 없는 변수에도 강제로 생성하게 됩니다
Lombok을 사용하는 경우 리팩토링이 힘들어지는 부분도 있으니 잘 고려하여 사용해야 합니다.

≡ Lombok 라이브러리

≡ Lombok 라이브러리

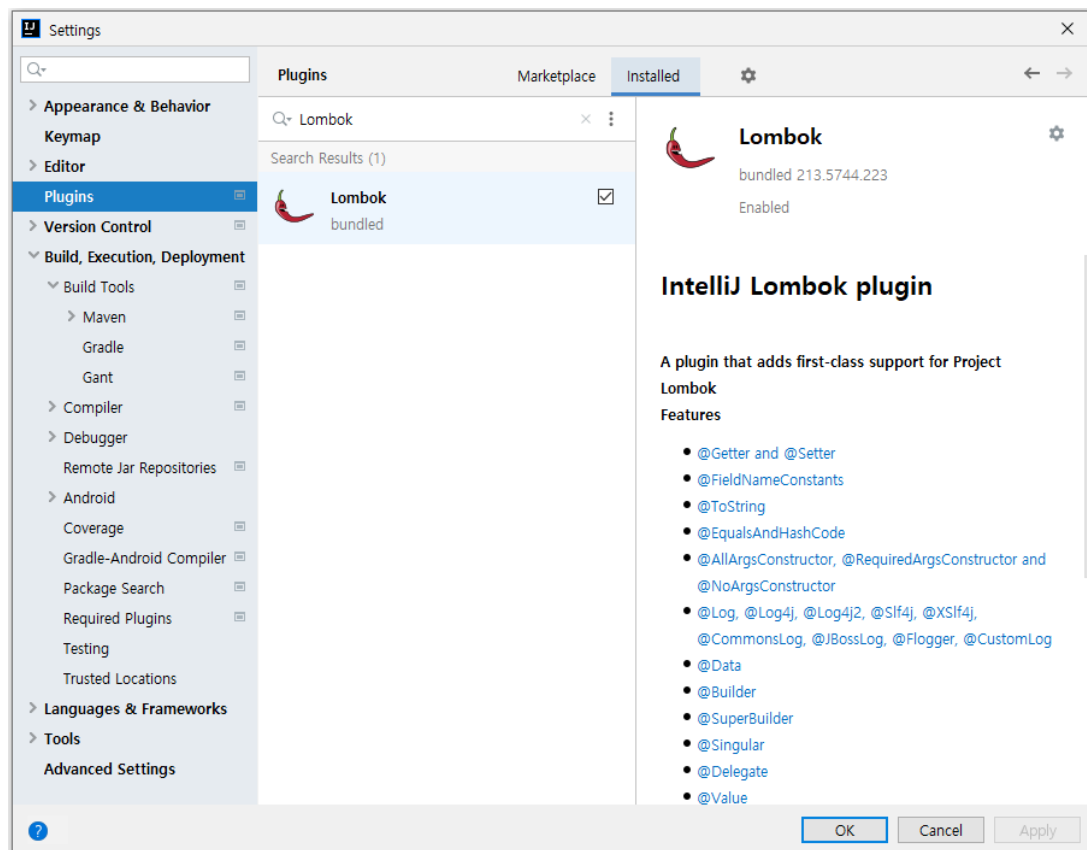
for IntelliJ

settings 메뉴의 Plugins 탭을 클릭합니다.

Marketplace 탭에서 설치하고자하는 플러그인을 찾아서 설치를 진행하면 됩니다.

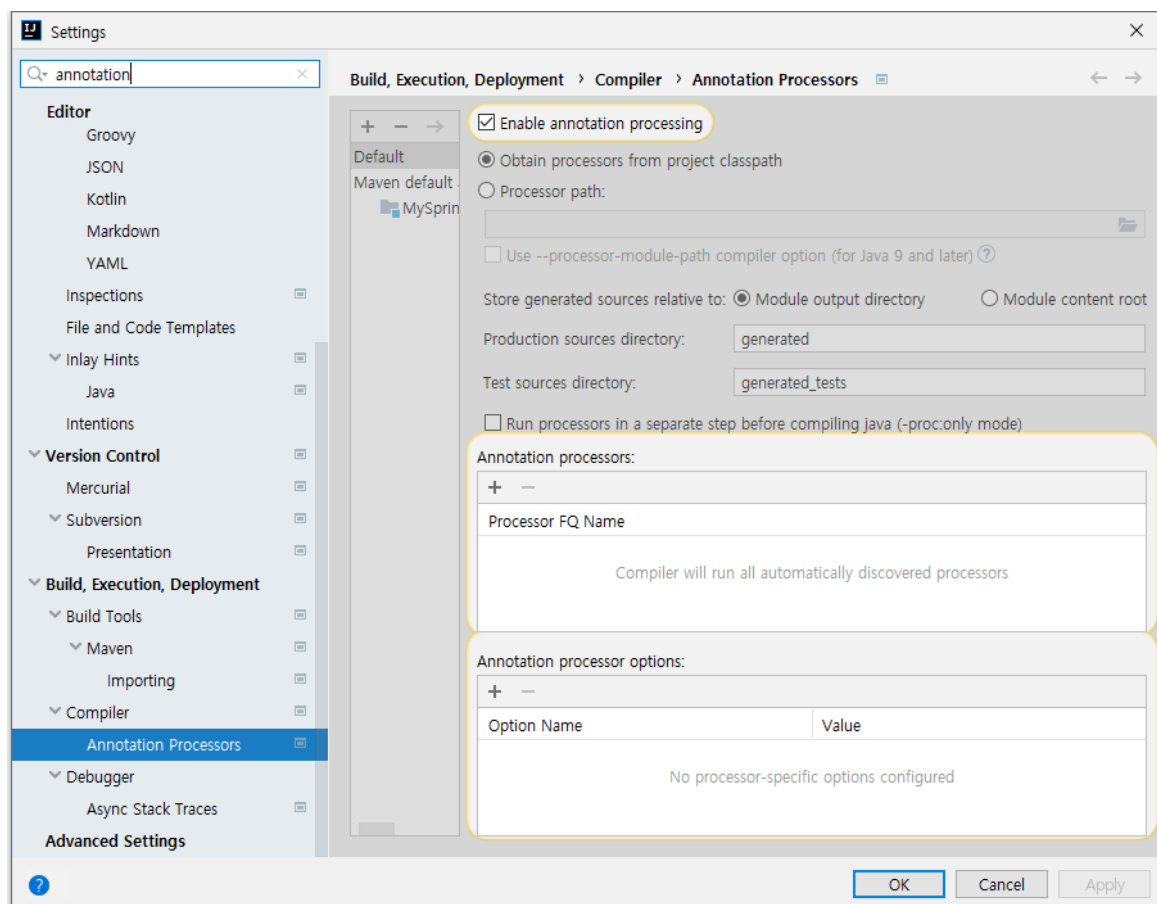
Installed 항목을 클릭하면 이미 설치되어 있는 플러그인 목록을 보여 줍니다.

Lombok은 기본 값으로 설치가 되어 있습니다.



settings 메뉴에서 annotation processing 기능을 활성화하도록 합니다.

롬복과 같은 외부 라이브러리가 잘 동작하려면 annotation processing을 활성화시켜 주어야 합니다.
Annotation Processor에서 체크 박스 'Enable annotation processing'를 on 상태로 변경합니다.



≡ Lombok 라이브러리

≡ Lombok 라이브러리

for eclipse

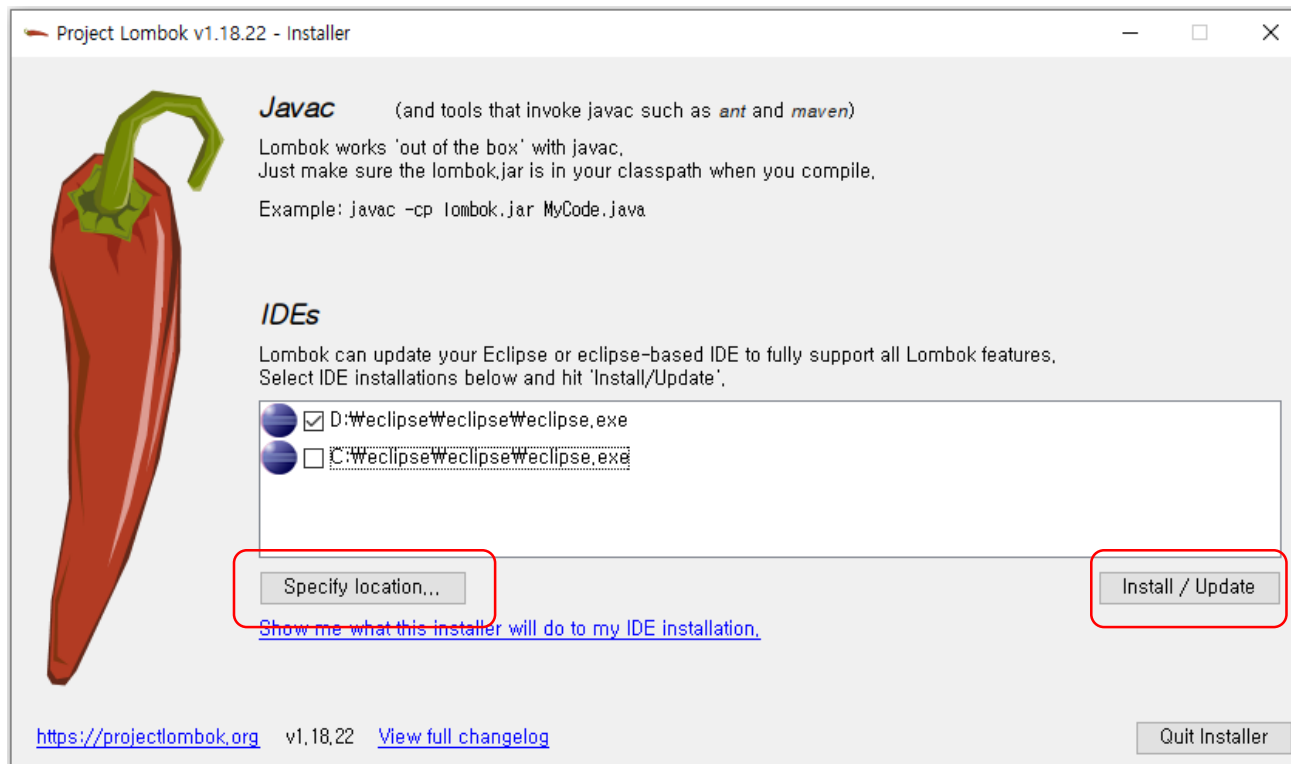
이클립스에서는 Lombok 라이브러리를 다음과 같이 설치합니다.

이클립스를 종료하고 <https://projectlombok.org/download>에 접속하여 'Lombok 라이브러리' 파일을 다운로드 합니다.

다운로드 받은 파일(lombok.jar)을 이클립스가 있는 폴더에 복사합니다.

cmd 창에서 해당 폴더로 이동한 후, 다음과 같은 명령어를 실행하면 아래와 같은 그림이 로딩됩니다.

```
java -jar lombok.jar
```



이클립스 설치 경로 내에 설정 정보 파일 eclipse.ini에 다음과 같이 작성하고 저장합니다.

```
-vm  
C:\Program Files\Java\jdk-11.0.11\bin\javaw.exe
```

≡ Lombok 라이브러리

≡ Lombok 라이브러리

for IntelliJ

pom.xml 파일에 다음과 같이 의존성을 추가하면 됩니다.

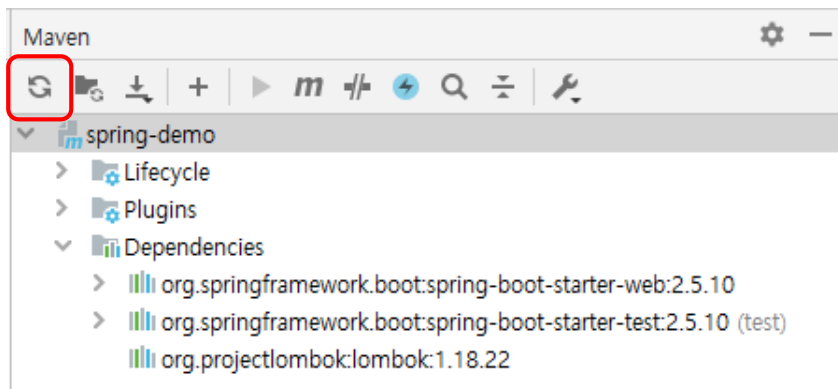
메이븐 사용

Lombok 관련 의존성을 다음과 같이 추가하고, Lombok 설치를 마무리하기 위해서는 IntelliJ를 다시 시작합니다.
재실행되고 난후 Maven Dependencies에 Lombok 라이브러리가 추가되었는지 확인합니다.

```
<dependency>  
  <groupId>org.projectlombok</groupId>  
  <artifactId>lombok</artifactId>  
</dependency>
```

주의 사항

pom.xml 파일을 작성한 다음 Maven Dependencies에서 "Reload All Maven Projects" 버튼을 눌러 주어야 합니다.



≡ Lombok 라이브러리

≡ Lombok 라이브러리

for IntelliJ

build.gradle 파일에 다음과 같이 의존성을 추가하면 됩니다.

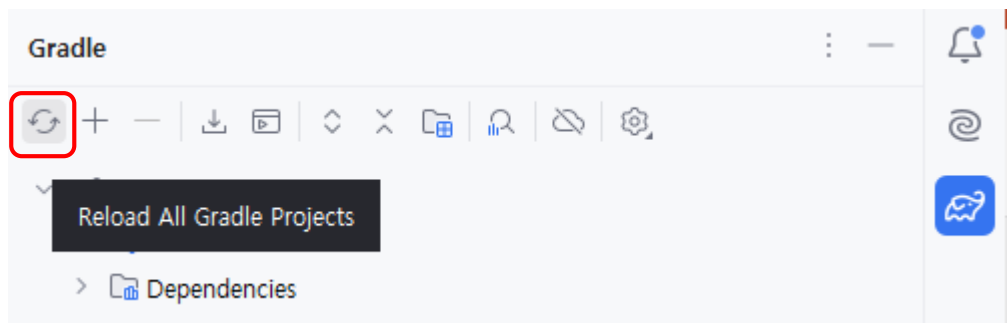
그레이들 사용

Lombok 관련 의존성을 다음과 같이 추가합니다.

```
dependencies {  
    // 이전 다른 설정들  
    compileOnly 'org.projectlombok:lombok'  
}
```

주의 사항

build.gradle 파일을 작성한 다음 그림과 같이 "Reload All Gradle Projects" 버튼을 눌러 주어야 합니다.



Spring Boot

Spring Data JPA



Spring



Boot



Security

Spring Data JPA

ORM(Object Relational Mapping)

자바의 객체 지향 기술과 관계형 database를 서로 연결(매핑)해주는 추상화된 기술입니다.

ORM

ORM은 객체 지향 프로그래밍의 객체(Object)와 관계형 데이터 베이스의 테이블(Table)이 유사하다는 것에 착안하여 만들어진 개념입니다.

Java Class

```
class Some{  
    String id ;  
    String name ;  
    int age ;  
    String gender ;  
}
```

ORM

Table

id	name	age	gender
kim9	김구	20	남자
maria	조마리아	30	여자

자바 객체와 RDBMS의 데이터를 자동으로 매핑(연결)해주는 것을 의미합니다.

- 객체 지향 프로그래밍은 클래스를 사용하고, 관계형 데이터베이스는 테이블을 사용합니다.
- 객체 모델과 RDBMS의 모델 간에 불일치가 존재할 수 있습니다.
- ORM을 통해 객체 간의 관계를 바탕으로 SQL을 자동으로 생성하여 불일치를 해결해 줍니다.

Object 필드와 데이터베이스 데이터를 서로 매핑(맺어줌)시킵니다.

- 객체를 사용하여 간접적으로 데이터베이스 데이터를 다루는 개념입니다.
- 데이터 영속성을 위한 API(Persistent API)의 한 종류입니다.

영속성(Persistence)

영속성(永續性)이란 데이터가 사라지지 않고 오랫동안 지속되는 현상을 의미합니다.

영속성을 갖지 않는 데이터는 휘발성인 메모리에서만 존재하다가 프로그램을 종료하면 모두 유실합니다.

메모리에 존재하는 데이터를 파일 시스템, 관계형 데이터베이스 등을 활용하여 영구적으로 저장하여 영속성을 부여해야 합니다.

데이터 베이스에 저장하는 방식

- 1) JDBC 사용하기(java에서 사용)
- 2) Spring JDBC(JdbcTemplate 등을 사용한)
- 3) Persistence Framework (Hibernate(하이버 네이트), Mybatis 등등)

Persistence Framework

JDBC 프로그래밍의 복잡함을 없애고, 간단한 작업만으로 데이터베이스와 연동되는 시스템을 빠르게 개발할 수 있습니다.

SQL Mapper와 Objet-Relational Mapping 두 종류로 구분할 수 있는데, JPA, Hibernate, Mybatis 등이 여기에 해당합니다.

Spring Data JPA

JPA(Java Persistence Api)

JPA는 데이터 베이스 관련 작업을 대신 처리해주는 추상화된 계층의 구현 스펙입니다.

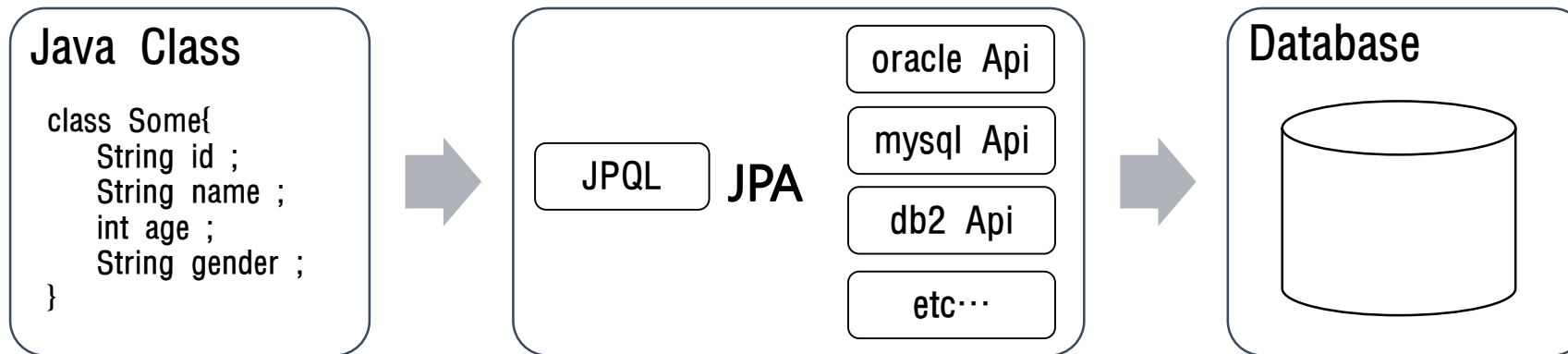
JPA 소개

JPA는 ORM이라는 기법을 자바에서 구현해 놓은 기술입니다.

Java 기술을 이용하여 데이터를 관리/유지 시켜주는 기법을 하나의 스펙으로 정리해 놓은 표준 Spec입니다.

이전 사용되는 EJB라는 기술 스펙에서 Entity Bean이라는 데이터를 처리해주는 스펙을 정해준 것이 JPA의 효시라고 볼수 있습니다.

데이터 베이스와 연관된 처리를 JPA 계층에서 처리하므로 좀 더 추상화된 형태의 코드를 제작하는 것으로 개발이 가능합니다.



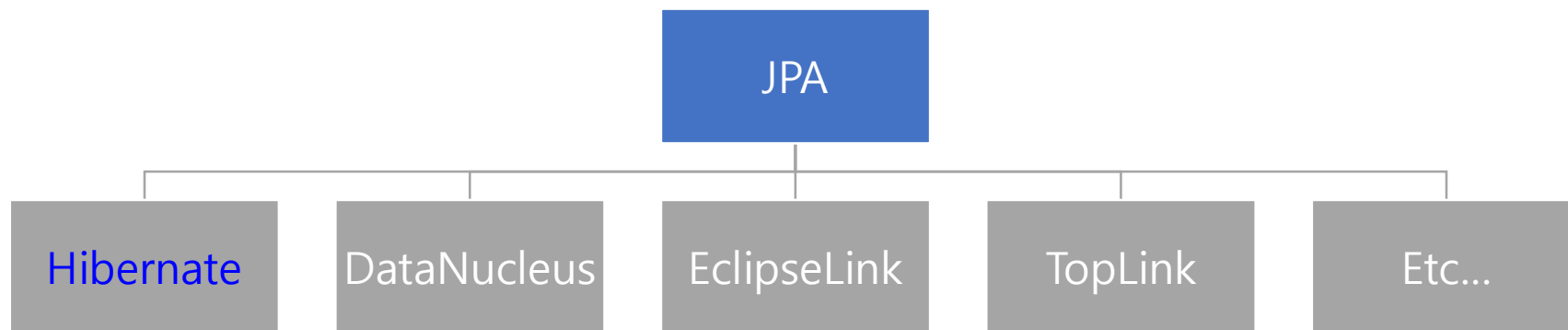
Spring Data JPA

JPA의 구현체

JPA는 자바의 Interface 형태로 설계가 되어 있습니다.

이러한 인터페이스를 구현해 놓은 구현체 클래스는 다음과 같은 항목들이 있습니다.

따라서, 코딩을 하고자 하는 경우에는 구현체 클래스를 반드시 사용해야 합니다.



☰ Spring Data JPA

☰ JPA의 장단점

데이터 베이스 관련 개발에서 다음과 같은 장점과 단점을 가집니다.

관계형 데이터 베이스와 객체 지향 프로그래밍의 객체 사이에서 도래하는 불일치를 해결하기 위해 ORM 기술이 만들어 졌습니다.

ORM 기술을 구현하기 위해 만들어진 프레임워크는 Hibernate, TopLink, CoCobase 등이 존재합니다.

차후 이러한 ORM 기술을 구현한 프레임워크에 대한 표준화가 이루어지면서 구현된 개념이 바로 JPA입니다.

선수 지식

ORM, 객체 지향 프로그래밍, database 개념등이 필요합니다.

JPA의 장점

특정 데이터 베이스에 종속적이지 않습니다.

데이터 베이스 설계 중심의 코드보다는 객체 지향적인 설계가 가능합니다.

테이블과 연관된 클래스만 수정하면 되므로 생산성 향상을 기대할 수 있습니다.

JPA의 단점

복잡한 쿼리 구문은 오히려 속도를 저하시킬 수 있습니다.

객체 간의 매핑이 잘못되는 경우 오히려 성능 저하를 발생시킬수 있습니다.

기존의 데이터 베이스에 비하여 새롭게 학습해야 할 내용이 많아서 시간이 오래 걸릴 수 있습니다.

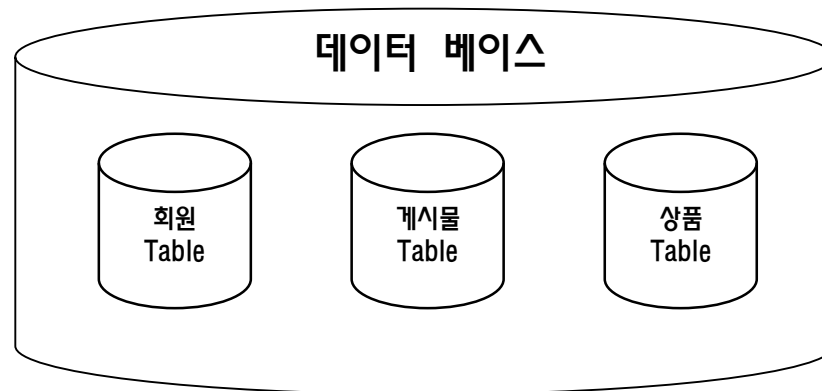
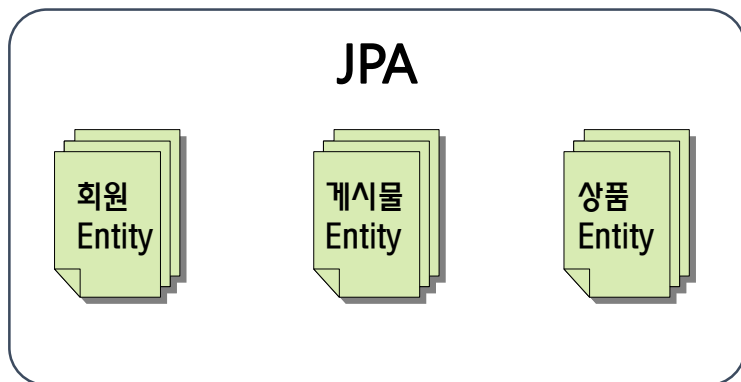
특정 데이터 베이스의 좋은 기능(성능 검사, 튜닝 등)들을 활용할 수 없을 수 있습니다.

Spring Data JPA

엔터티(Entity)

엔터티는 데이터 베이스에서 관리가 필요한 어떠한 대상(target)입니다.

Entity는 데이터 베이스에서 흔히 말하는 Table을 의미합니다.
다음 예시에서 회원, 게시물, 상품 등을 Entity Types라고 합니다.



Entity 관리자

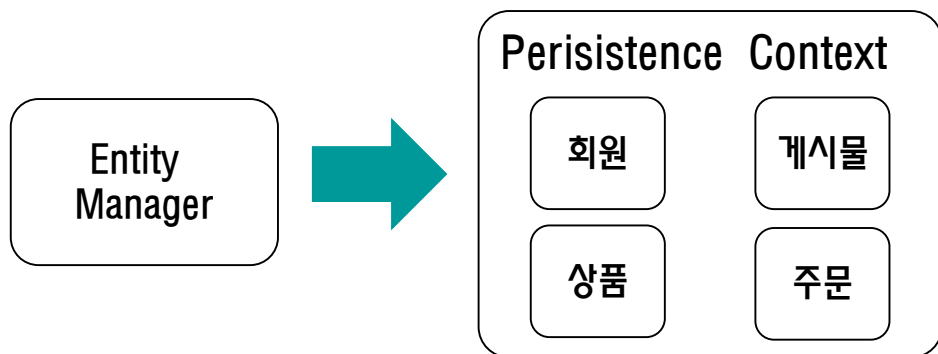
Entity Manager는 Entity들의 생명 주기(life cycle)를 관리해주는 관리자 역할을 합니다.

Entity들을 영속성 컨텍스트(Persistence Context)에서 관리 합니다.

영속성(Persistence) Context

Entity 클래스에서 만들어지는 Entity를 영구적으로 저장하고 관리해주는 환경을 의미합니다.

즉, Entity를 모아 놓고 관리해주는 공간 또는 유타리입니다.



Entity Manager 메소드

항목	설명
find()	영속성 컨텍스트에서 Entity를 검색합니다. 존재하지 않는 경우 데이터 베이스에서 읽어 와서 영속성 컨텍스트에 저장합니다.
persist()	Entity를 영속성 컨텍스트에 저장시킵니다.
remove()	Entity를 영속성 컨텍스트에 삭제시킵니다.
flush()	영속성 컨텍스트에 저장된 내용을 데이터 베이스에 반영시킵니다.
clear()	영속성 컨텍스트의 내용을 비우고, 상태를 초기화합니다.

Entity 관리자

Entity들의 생명 주기(life cycle)를 관리해주는 관리자 역할을 합니다.

Entity의 상태(state)

Persistence Context와 관련하여 Entity는 다음과 같이 4가지의 상태를 가질 수 있습니다.

항목	설명
비영속(new/transient)	Entity 객체가 만들어져서 아직 저장되지 않은 상태로, 영속성 컨텍스트와 전혀 관계가 없는 상태입니다.
영속(managed)	Entity가 영속성 컨텍스트에 저장되어, 영속성 컨텍스트가 관리할 수 있는 상태입니다.
준영속(detached)	Entity가 영속성 컨텍스트에 저장되어 있다가 분리된 상태로, 영속성 컨텍스트가 더 이상 관리하지 않는 상태입니다.
삭제(removed)	Entity를 영속성 컨텍스트와 데이터베이스에서 삭제됩니다.

영속성 컨텍스트의 특징

영속성 컨텍스트는 Entity를 식별자 값(@Id로 테이블의 기본키와 매핑한 필드 값)으로 구분합니다.

그렇기 때문에 영속 상태는 식별자 값이 반드시 있어야 합니다.

② 영속성 컨텍스트에 Entity를 저장하면 바로 데이터베이스에 저장되는 것이 아니라, 1차 캐시에 Entity를 생성하고, 쓰기 지연 SQL 저장소에 쿼리문을 생성해서 저장합니다.

이렇게 쌓인 쿼리문은 flush()가 실행될 때 데이터베이스에 반영이 됩니다.

Spring Data JPA

Dialect(방언)

application.properties 파일에 사용될 방언에 대하여 살펴 보겠습니다.

Dialect 참조 사이트

설정된 데이터 베이스의 종류(Vendor)에 맞는 sql 구문을 만들어 주기 위한 방언을 의미합니다.

DBMS Vendor별 용어

SQL 구문의 표준은 ANSI SQL입니다.

하지만 DBMS Vendor(공급 업체)마다 자신들만의 독자적인 기능을 만들기 위하여 조금씩 특화된 SQL 구문이 존재할 수 있습니다.

다음은 Vendor마다 조금씩 다른 개념들을 표로 정리한 내용입니다.

Oracle	Ms-Sql	My-Sql
pl/sql	t-sql	stored program
sequence	row idntity	auto increment

JPA는 SQL 구문을 작성하기 위하여 사용하는 DBMS가 어떤 Vendor인지 알 필요가 있습니다.

JPA에게 DBMS를 사용하는 알려 주는 방법이 바로 방언을 설정하는 방법입니다.

JPA에 Dialect를 설정할 수 있는 추상화 방언 클래스를 제공하고 설정된 방언으로 각 DBMS에 맞는 구현체를 제공합니다.

Dialect 참조 사이트

<https://docs.jboss.org/hibernate/orm/5.3/javadocs/org/hibernate/dialect/package-summary.html>

데이터 베이스 초기화 전략

JPA와 Hibernate를 사용할 때, 데이터베이스 스키마 생성 및 업데이트를 제어하는 옵션입니다.

파일 이름 resources\application.properties

spring.jpa.hibernate.ddl-auto 옵션은 JPA와 Hibernate를 사용할 때, 데이터베이스 스키마 생성 및 업데이트를 제어하는 옵션입니다.

일반적으로 개발 환경에서는 `update`나 `create-drop`을 사용하고, 운영 환경에서는 데이터 손실 방지를 위해 `none`이나 `validate`를 사용하는 것이 권장됩니다.

옵션	설명
none	Hibernate가 데이터베이스 스키마에 대하여 아무것도 수행하지 않습니다.(기본값) 스키마 관리가 수동으로 이루어져야 할 때 사용합니다.
create	애플리케이션이 실행될 때 기존 데이터베이스 스키마를 삭제(drop)하고 생성된 DDL을 실행해 줍니다. 이때 기존 데이터는 모두 삭제가 됩니다.
create-drop	`create` 옵션과 동일하지만, 애플리케이션이 종료될 때 데이터베이스 스키마를 삭제합니다. in-memory DB인 경우 이 옵션이 기본 값입니다.
update	엔티티의 변경 사항에 따라 데이터베이스 스키마를 업데이트합니다. 이미 존재하는 테이블을 수정할 수 있지만, 데이터 손실이 발생할 수 있는 구조적 변경은 처리하지 않습니다.
validate	애플리케이션의 엔티티와 데이터베이스 테이블 구조가 정상적으로 매핑이 되었는 지를 확인합니다. 일치하지 않으면 예외가 발생하지만, 스키마 수정은 하지 않습니다.

Spring Data JPA

SQL 쿼리 로그 관리

JPA와 Hibernate ORM을 사용시 SQL 쿼리 로그를 관리하는 데 관련된 설정들입니다.

파일 이름

resources\application.properties

옵션	설명
spring.jpa.properties.hibernate.show_sql=true	Hibernate가 실행하는 SQL 쿼리를 콘솔에 출력하도록 합니다. true로 설정하면, 애플리케이션이 실행할 때 모든 SQL 쿼리를 로그에 출력합니다.
spring.jpa.properties.hibernate.format_sql=true	콘솔에 출력되는 SQL 쿼리를 가독성이 좋게 포맷팅합니다. true로 설정하면, SQL 쿼리를 여러 줄에 걸쳐 들여쓰기와 함께 표시하여 가독성을 높입니다.
logging.level.org.hibernate.type.descriptor.sql=trace	SQL 쿼리의 바인드 파라미터 (PreparedStatement에서 ?로 표현되는 부분)를 로그에 출력합니다. trace 레벨로 설정하면, Hibernate가 쿼리에 어떤 값을 바인드하는지 자세히 확인할 수 있습니다.
spring.jpa.hibernate.ddl-auto=create	이 설정은 애플리케이션이 시작할 때 데이터베이스 테이블을 어떻게 처리할지를 결정합니다. create로 설정하면, 애플리케이션이 시작될 때마다 기존 데이터베이스 테이블을 삭제하고 새로 생성합니다. 이는 개발 중 데이터베이스 스키마를 자주 변경할 때 유용하지만, 실 운영 환경에서는 주의해야 합니다.
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect	이 설정은 Hibernate가 사용하는 데이터베이스 방언 (Dialect)을 지정합니다. MySQL8Dialect는 MySQL 8.x 버전에 맞춘 방언으로, Hibernate가 MySQL 데이터베이스에 최적화된 SQL을 생성할 수 있게 해줍니다. MySQL 이외에도 PostgreSQL, Oracle 등의 다른 데이터베이스를 사용할 경우 해당하는 방언을 설정해야 합니다.

Spring Data JPA

엔터티 매핑 어노테이션

엔터티는 데이터 베이스의 테이블에 대응해주는 클래스를 의미합니다.

엔터티 매핑 관련 어노테이션

항목	설명
@Entity	클래스를 엔터티로 만들어 주는 역할을 합니다.
@Table	엔터티와 매핑을 진행할 테이블 이름(name 속성)을 지정해 줍니다.
@Id	테이블의 기본 키를 의미합니다. 문자열인 경우 해당 어노테이션으로 기본 키를 지정할 수 있습니다.
@Column	필드와 컬럼 매칭을 수행합니다. nullable = false이면 테이블 설계시 반드시 not null이어야 합니다.
@GeneratedValue	숫자 형식의 키 값을 생성하는 전략을 명시하는 데, strategy 속성을 사용하여 기본 키를 부여하는 방식을 결정해 줍니다.
@CreateDate	Entity가 생성 후 저장될 때의 시간을 자동으로 저장해 줍니다.
@LastModifiedDate	Entity의 값을 변경할 때의 시간을 자동으로 저장해 줍니다.
@CreationTimestamp	insert 시점의 시간을 자동으로 저장해 줍니다.
@UpdateTimestamp	update 시점의 시간을 자동으로 저장해 줍니다.
@Lob	CLOB(Character Large Object), BLOB(Binary Large Object) 타입 매핑을 해줍니다.
@Enumerated	enum 타입과 관련된 변수를 매핑해 줍니다. ORDINAL과 STRING 방식 중에서 주로 STRING을 많이 사용합니다.

상품 엔터티 설계하기

엔터티는 데이터 베이스의 테이블에 대응해주는 클래스를 의미합니다.

기본 키 생성 전략

Java의 JPA(Java Persistence API)에서 엔터티 클래스의 기본 키(primary key)를 자동으로 생성하고자 할 때 사용됩니다.

`javax.persistence.GeneratedValue` 패키지에 포함 되어 있습니다.

데이터베이스에서 사용하는 기본 키(primary key) 값을 자동으로 생성하고 할당하는 전략을 정의합니다.

기본 키를 생성하는 전략에는 다음과 같은 항목들이 있습니다.

생성 전략

`strategy` 속성은 키 생성 전략을 지정하는 속성입니다.

이 속성은 `GenerationType` 열거형(enum)을 사용하여 설정할 수 있습니다.

생성 전략	설명
AUTO	기본 설정 값으로 각 데이터베이스에 따라 기본키를 자동으로 생성해 줍니다.
IDENTITY	기본키 생성을 데이터베이스에게 위임하는 방식으로, mysql에서 <code>AUTO_INCREMENT</code> 를 하여 기본키를 생성해주는 방식입니다.
SEQUENCE	데이터 베이스의 Sequence Object를 사용하여 데이터베이스가 자동으로 기본키를 생성해주는 방식입니다.
TABLE	키 생성을 위한 테이블을 사용하는 방식으로 <code>@TableGenerator</code> 이 필요합니다.

`generator` 속성은 시퀀스나 테이블 기반 전략을 사용할 때, 사용할 시퀀스나 테이블의 이름을 지정할 수 있는 속성입니다.

예를 들어, SEQUENCE 전략을 사용할 경우, `generator` 속성을 사용하여 시퀀스의 이름을 지정합니다.

☰ Spring Data JPA

☰ 엔터티 매핑 관련 어노테이션

각각의 어노테이션에 대한 속성들을 정리해 봅니다.

@Entity 어노테이션을 클래스에 선언하면 그 클래스는 JPA가 관리합니다.

따라서, 데이터 베이스의 테이블과 Class(VO, DTO)와 맵핑한다면 반드시 @Entity를 붙여 주어야 합니다.

@Entity가 붙은 클래스에는 다음 제약 사항이 필요합니다.

필드에 final, enum, interface, class를 사용할 수 없습니다.

생성자 중에서 기본 생성자는 반드시 필요합니다.

@Entity 어노테이션

속성	설명
name	Entity 이름을 지정하는 속성으로 기본 값은 클래스 이름을 사용합니다.

주의 사항

@Entity 어노테이션이 붙어 있는 클래스에 대한 변수들을 확인하여 Column을 만들어 줍니다.

이미 생성된 컬럼은 JPA가 자동으로 삭제해 주지 않으므로, 개발자가 직접 제거해 주어야 합니다.

엔터티 매핑 관련 어노테이션

@Table 어노테이션은 데이터베이스 테이블과 Java 클래스 간의 매핑을 정의하는 데 사용됩니다.

@Table 어노테이션

클래스 레벨에서 선언되며, 해당 클래스가 데이터베이스의 특정 테이블에 매핑된다는 것을 나타냅니다.

해당 어노테이션을 통해 다양한 속성을 설정할 수 있으며, 이를 통해 테이블의 이름, 스키마, 고유 제약 조건 등을 지정할 수 있습니다.

속성	설명
name	해당 클래스가 매핑될 데이터베이스 테이블의 이름을 정의합니다. 만약 name 속성을 지정하지 않으면 기본적으로 클래스 이름이 테이블 이름으로 사용됩니다.
catalog	데이터 베이스의 catalog를 맵핑합니다.
schema	테이블이 속한 데이터베이스 스키마(hr, scott 등등)를 지정합니다. 이를 통해 특정 스키마 아래에 있는 테이블에 클래스가 매핑되도록 할 수 있습니다.
uniqueConstraint	테이블에 고유 제약 조건을 추가할 때 사용합니다. 이 속성은 @UniqueConstraint 배열로, 하나 이상의 고유 제약 조건을 정의할 수 있습니다.
indexes	테이블의 인덱스를 정의할 수 있습니다. 이 속성은 @Index 배열로, 하나 이상의 인덱스를 설정할 수 있습니다.

엔터티 매핑 관련 어노테이션

엔터티 클래스의 필드를 데이터베이스 테이블의 컬럼과 매핑할 때 사용됩니다.

@Column 어노테이션

@Entity 어노테이션이 선언된 클래스 내에서 사용됩니다.

다양한 속성을 통해 컬럼의 특성을 세부적으로 정의할 수 있습니다.

클래스의 필드를 특정 테이블의 컬럼에 연결하여, ORM(Object-Relational Mapping)이 이루어지도록 합니다.

속성	설명
name	필드와 매핑되는 데이터베이스 테이블의 컬럼명을 지정합니다. 기본적으로 필드 이름과 동일한 이름의 컬럼이 생성되지만, name 속성을 통해 컬럼명을 변경할 수 있습니다.
nullable	컬럼에 NULL 값을 허용할지를 설정합니다. 기본값은 true로, null을 허용합니다.
length	문자열 타입의 컬럼에 대해 최대 길이를 설정합니다.(기본 값 : 255)
unique	해당 컬럼에 대해 유니크 제약 조건을 설정합니다. 기본값은 false로, 중복된 값을 허용합니다.
insertable 및 updatable	데이터베이스에 삽입 또는 갱신할 때 해당 컬럼을 포함할지를 결정합니다. 기본값은 둘 다 true입니다.
table	해당 필드를 특정 테이블의 컬럼에 매핑할 수 있습니다. 이는 엔터티가 여러 테이블과 연결될 때 유용합니다. 기본적으로 엔터티 클래스가 매핑된 테이블을 사용하지만, 이 속성을 사용하면 다른 테이블을 지정할 수 있습니다.
precision 및 scale	숫자 타입의 컬럼에 대해 정밀도(precision)와 소수점 자릿수(scale)를 설정합니다. precision은 전체 자릿수, scale은 소수점 이하 자릿수를 의미합니다.

Spring Data JPA

Repository 설계하기

Repository를 설명하기 전에 CRUD가 무엇인지 살펴 보도록 합니다.

CRUD(출처 위키 백과)

CRUD는 대부분의 컴퓨터 소프트웨어가 가지는 기본적인 데이터 처리 기능인 Create(생성), Read(읽기), Update(갱신), Delete(삭제)를 묶어서 일컫는 말입니다. 사용자 인터페이스가 갖추어야 할 기능(정보의 참조/검색/갱신)을 가리키는 용어로서도 사용이 됩니다.



이름	조작	SQL
Create	생성	INSERT
Read(또는 Retrieve)	읽기(또는 인출)	SELECT
Update	갱신	UPDATE
Delete(또는 Destroy)	삭제(또는 파괴)	DELETE

Spring Data JPA

Repository 설계하기

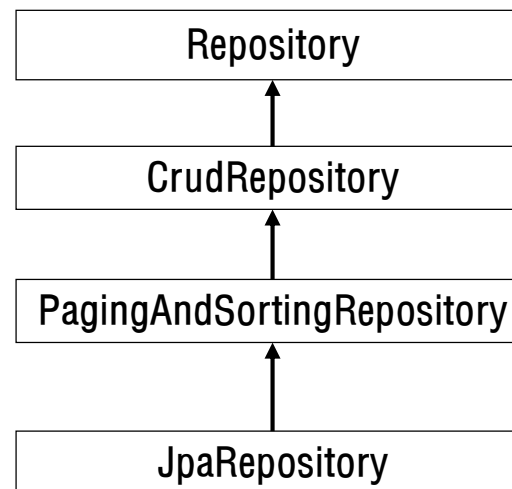
데이터베이스에 접근하여 CRUD를 수행해주는 데이터 접근을 위한 클래스/인터페이스를 의미합니다.

관련 개념

Dao(Data Access Object) 역할을 수행해주는 인터페이스 객체로써, JPA에서는 Repository 객체를 이용하여 Entity를 관리합니다. 일반적으로 CrudRepository 또는 JpaRepository 인터페이스를 상속 받아서 구현하며, 다음과 같은 메소드를 사용할 수 있습니다.

메소드	설명
<S extends T> save(S entity)	엔티티를 저장하거나 수정할 때 사용합니다.
void delete(T entity)	엔티티를 삭제합니다.
count()	엔티티의 개수를 반환해줍니다.
Iterable<T> findAll()	모든 엔티티를 조회합니다.
findById()	기본 키를 이용하여 1건을 조회합니다.
deleteById()	기본 키를 이용하여 데이터를 삭제합니다.
saveAndFlush()	강제로 flush를 호출하여 영속성 컨텍스트의 객체들을 데이터 베이스에 반영시킵니다.

Class 계층도



Spring Data JPA

SpringBootTest 어노테이션

테스트와 연관된 어노테이션을 살펴 보겠습니다.

테스트와 연관된 어노테이션

어노테이션	설명
@SpringBootTest	통합 테스트를 위하여 스프링 부트에서 제공하는 어노 테이션입니다.

주의 사항

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope> ← 어노테이션을 인식하지 못하므로 반드시 삭제해 주세요.
</dependency>
```

@Repository 어노테이션

항목	설명
목적	데이터 베이스 또는 외부 파일과 상호 작용하여 Input/Output 작업을 처리해 주는 클래스 역할을 합니다. '영속성'을 가지고 있는 영역(Database, File)과 연동하는 layer로써, 일반적으로 퍼시스턴스(Persistence) 레이어라고 합니다.
설정 위치	클래스 선언부

☰ Repository 설계하기

@Autowired은 의존 관계를 자동으로 설정해주는 어노테이션입니다.

의존성 주입(Dependency Injection)을 자동으로 처리해주는 역할을 합니다.

필드, 생성자, 메소드에 사용되며, 해당 객체에 필요한 의존성을 스프링이 자동으로 주입해줍니다.

이 어노테이션은 스프링 컨테이너가 관리하는 빈(Beans) 객체 중 적합한 타입의 빈을 찾아 자동으로 연결해줍니다.

항목	설명
목적	bean 객체간의 의존 관계를 자동 설정할 때 사용하며 타입을 이용하여 의존하는 객체를 삽입해 줍니다. 그러므로 해당 타입의 bean 객체가 존재하지 않거나 또는 2개 이상 존재할 경우 스프링은 예외를 발생시키게 됩니다.
설정 위치	생성자, 필드, 메소드(굳이 setter 메소드가 아니어도 됩니다.)

사용 예시

```
@SpringBootTest
class ProductRepositoryTest {
    @Autowired // 인스턴스 변수 productRepository는 기본 값이 null인데,
    ProductRepository productRepository; // 자동 주입을 통하여 의미있는 객체(notnull object)가 됩니다.

    @Test
    @DisplayName("상품 저장 테스트")
    public void createProductTest(){
        Product savedProduct = productsRepository.save(products);
    }
}
```


Spring Data JPA

쿼리 메소드

데이터베이스에 대하여 쿼리를 수행해 주는 메소드입니다.

JPA에서는 객체 지향 프로그래밍의 개념을 바탕으로 데이터베이스와의 상호 작용을 수월하게 해줍니다.

쿼리 메소드는 일반적으로 `Repository` 인터페이스에서 정의되며, Spring Data JPA와 함께 사용할 때 매우 유용합니다.

데이터베이스와의 상호 작용을 단순화하고, 코드의 가독성을 높이며, 유지보수를 용이하게 해주는 강력한 도구입니다.

주요 개념

항목	설명
메소드 이름으로 쿼리 생성	JPA는 메소드 이름을 분석하여 자동으로 쿼리를 생성합니다. 예를 들어, `findByLastName(String lastName)` 메소드는 `lastName` 속성이 주어진 값과 일치하는 모든 엔티티를 검색하는 쿼리를 생성합니다.
JPQL(Java Persistence Query Language)	JPQL은 JPA에서 사용하는 쿼리 언어로, 객체 지향 쿼리를 작성할 수 있게 해줍니다. 쿼리 메소드 대신 JPQL을 사용할 수도 있습니다.
커스텀 쿼리	@Query 어노테이션을 사용하여 복잡한 쿼리를 정의할 수 있습니다. 예를 들어, 특정 조건에 맞는 데이터를 선택하기 위해 JPQL을 사용할 수 있습니다.
Paging and Sorting	JPA의 쿼리 메소드는 페이징 및 정렬 기능을 지원합니다. Pageable이나 Sort 객체를 파라미터로 받아 처리할 수 있습니다.

사용 형식

find + (엔티티_이름) + By + 변수_이름

쿼리 메소드는 스프링 데이터 JPA에서 제공하는 핵심 기능입니다.

가장 많이 사용되는 문법은 find 키워드를 사용하는 기법입니다.

(엔티티_이름)의 생략이 가능하며 By 뒤에는 검색을 하고자 할때 사용되는 변수의 이름을 작성하면 됩니다.

리턴 타입은 크게 List<T>, Page<T>, Slice<T>의 3가지 종류가 있습니다.

Spring Data JPA

쿼리 메소드

다음 내용은 쿼리 메소드와 관련 sample과 snippet(작은 조각의 정보)입니다.

키워드	Sample	JPQL snippet(스니펫)
And	findByLastnameAndFirstname	... where x.lastname = ?1 and x.firstname = ?2
Or	findByLastnameOrFirstname	... where x.lastname = ?1 or x.firstname = ?2
Is, Equals	findByFirstname, findByFirstnames, findByFirstnameEquals	... where x.firstname = ?1
Between	findByStartDateBetween	... where x.startDate between ?1 and ?2
LessThan	findByAgeLessThan	... where x.age < ?1
LessThanEqual	findByAgeLessThanEqual	... where x.age <= ?1
GreaterThan	findByAgeGreaterThan	... where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual	... where x.age >= ?1
After	findByStartDateAfter	... where x.startDate > ?1
Before	findByStartDateBefore	... where x.startDate < ?1
IsNull, Null	findByAge(Is)Null	... where x.age is null
IsNotNull, NotNull	findByAge(Is)NotNull	... where x.age not null

Spring Data JPA

쿼리 메소드

다음 내용은 쿼리 메소드와 관련 sample과 snippet(작은 조각의 정보)입니다.

키워드	Sample	JPQL snippet(스니펫)
OrderBy	findByAgeOrderByLastnameDesc	... where x.age = ?1 order by x.lastname desc
Not	findByLastnameNot	... where x.lastname <> ?1
In	findByAgeIn(Collection<Age> ages)	... where x.age in ?1
NotIn	findByAgeNotIn(Collection<Age> ages)	... where x.age not in ?1
TRUE	findByActiveTrue()	... where x.active = true
FALSE	findByActiveFalse()	... where x.active = false
IgnoreCase	findByFirstnameIgnoreCase	... where UPPER(x.firstname) = UPPER(?1)

Spring Data JPA

쿼리 메소드

다음 내용은 쿼리 메소드와 관련 sample과 snippet(작은 조각의 정보)입니다.

like 구문에 대한 처리는 다음과 같은 항목들을 사용할 수 있습니다.

키워드	Sample	JPQL snippet(스니펫)
Like	findByFirstnameLike	... where x.firstname like ?1
StartingWith	findByFirstnameStartingWith	... where x.firstname like ?1(첨부 된 매개 변수 %)
EndingWith	findByFirstnameEndingWith	... where x.firstname like ?1(앞에 붙은 매개 변수 %)
Containing	findByFirstnameContaining	... where x.firstname like ?1(매개 변수 묶음 %)
NotLike	findByFirstnameNotLike	... where x.firstname not like ?1

☰ Spring Data JPA

☰ @Query 어노테이션

JPA 리포지토리 인터페이스에서 쿼리를 직접 정의할 수 있도록 해주는 어노테이션입니다.

쿼리 메소드의 가독성

쿼리 메소드를 사용하여 상품 데이터를 조회해 보았습니다.

쿼리 메소드는 조건의 개수가 적으면 쿼리 메소드를 유용하게 사용할 수 있습니다.

반면에, 조건식이 많아지게 되면 오히려 메소드 이름의 가독성이 낮아집니다.

즉, 쿼리 메소드는 간단한 쿼리 처리에는 유용하지만, 복잡한 쿼리를 다루기에는 적합하지 않습니다.

이러한 경우에는 SQL과 유사한 JPQL이라는 언어를 사용하면 복잡한 쿼리를 수행할 수 있습니다.

장점

- 간단한 쿼리 작업에는 유용

단점

- 복잡한 쿼리 작성에는 다소 부적합

☰ Spring Data JPA

☰ @Query 어노테이션

@Query 어노테이션을 사용하여 JPQL이라는 쿼리 언어를 사용할 수 있습니다.

JPQL

Java Persistence Query Language의 줄인 말입니다.

엔티티 객체를 대상으로 쿼리를 수행합니다.

SQL를 추상화하여 사용하기 때문에 특정 DBMS에 의존하지 않습니다.

접근 방식	문법	작업 대상
@Query Annotation	JPQL	Entity
데이터 베이스	SQL	Table

주의 사항

문자열을 잘못 입력하게 되면 실행하기 이전에 오류가 있다는 사실을 인식하기 어렵습니다.

jpql 구문의 **from 절에는 반드시 대소문자 구분하여 entity 이름을 적어 주어야** 합니다.

‘nativeQuery = true’ 옵션은 범용적이지 못하고, 특정 데이터 베이스에 종속되는 구문 작성시 필요합니다.

대소문자 구분 없이 Table 이름을 명시하면 합니다.

Spring Data JPA

@Query 어노테이션

사용자 정의 JPQL(Java Persistence Query Language) 또는 SQL 쿼리를 정의하는 데 사용됩니다.

사용법

항목	설명
기본 사용법	<pre>@Repository public interface UserRepository extends JpaRepository<User, Long> { @Query("SELECT u FROM User u WHERE u.email = ?1") User findByEmail(String email); }</pre>
파라미터 바인딩	쿼리 내에서 파라미터를 사용하여 값을 바인딩할 수 있습니다. `?1`, `?2` 등의 형식으로 인덱스를 지정하거나, `:name` 형식으로 명명된 파라미터를 사용할 수 있습니다. <pre>@Query("SELECT u FROM User u WHERE u.age > :age") List<User> findUsersOlderThan(@Param("age") int age);</pre>
SQL 쿼리 사용	JPQL뿐만 아니라 네이티브 SQL 쿼리도 사용할 수 있습니다. 이 경우 `nativeQuery` 속성을 `true`로 설정해야 합니다. <pre>@Query(value = "SELECT * FROM users WHERE email = ?1", nativeQuery = true) User findByEmailNative(String email);</pre>
복잡한 쿼리	조인, 서브 쿼리, 집계 함수 등 복잡한 쿼리를 작성할 수 있습니다. <pre>@Query("SELECT u FROM User u JOIN u.orders o WHERE o.total > :total") List<User> findUsersWithHighOrderTotal(@Param("total") double total);</pre>

☰ Spring Data JPA

☰ @Query 어노테이션

사용자 정의 JPQL(Java Persistence Query Language) 또는 SQL 쿼리를 정의하는 데 사용됩니다.

사용법

항목	설명
여러 결과 반환	<code>@Query</code>를 사용하여 여러 결과를 반환할 수 있습니다. 이 경우 <code>List<T></code> 또는 <code>Page<T></code>와 같은 컬렉션 타입으로 메소드를 정의할 수 있습니다. <pre>@Query("SELECT u FROM User u WHERE u.status = :status") List<User> findByStatus(@Param("status") String status);</pre>
정렬 및 페이징	쿼리와 함께 정렬 및 페이징을 적용할 수 있습니다. <pre>@Query("SELECT u FROM User u WHERE u.status = :status") Page<User> findByStatus(@Param("status") String status, Pageable pageable);</pre>

☰ Spring Data JPA

☰ @Query 어노테이션

@Query 어노테이션을 사용하여 JPQL이라는 쿼리 언어를 사용할 수 있습니다.

파라미터 바인딩 예시

어노테이션	설명
@Param	JPQL에 파라미터를 넘겨 주고자 할 때 사용합니다. 문자형 파라미터 형식은 ":파라미터이름"와 같이 맨 앞에 콜론을 붙여서 작성합니다. 숫자를 이용하는 경우 ":1"의 형식으로 인덱스 값을 넣어서 작성하면 됩니다.
사용 예시	<pre>// 로그인한 회원의 주문 데이터를 페이징 조건에 맞추어서 조회하되 최근 주문을 먼저 보여줍니다. @Query("select o from Order o " + "where o.member.email = :email " + "order by o.orderDate desc") List<Order> findOrders(@Param("email") String email, Pageable pageable);</pre>

Querydsl 개요

Java 애플리케이션에서 타입 안전한 SQL-like 쿼리를 작성할 수 있도록 지원하는 프레임워크입니다.

Querydsl은 Java 개발자에게 강력하고 유연한 쿼리 작성 도구를 제공하여, 데이터베이스와의 상호 작용을 보다 안전하고 효율적으로 만들어 줍니다.

복잡한 쿼리를 직관적으로 작성할 수 있어, 애플리케이션 개발에서 큰 도움이 되며, 특히 JPA, SQL, MongoDB, Lucene 등 다양한 데이터베이스에 대한 지원을 제공합니다.

참조 사이트) <http://querydsl.com/>

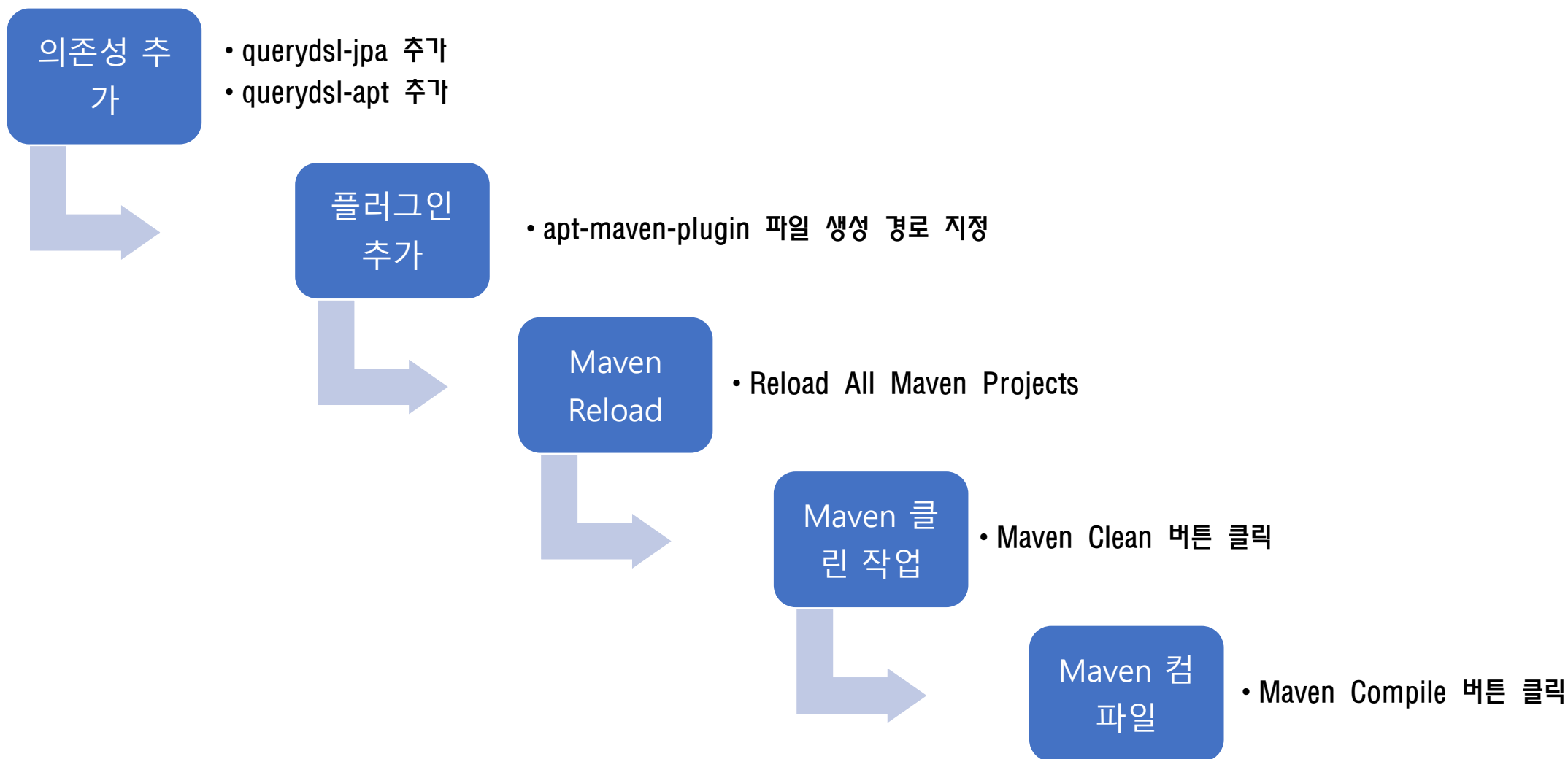
주요 개념

항목	설명
타입 안전성	SQL 구문을 문자열 형태가 아닌 자바 소스 코드 형태로 작성 을 하므로 컴파일 시점에 미리 오류를 파악할 수 있습니다. 쿼리에서 사용하는 엔티티와 필드가 Java 타입으로 정의되어 있어, IDE에서 자동 완성 및 리팩토링이 가능합니다.
DSL(Domain Specific Language)	쿼리 문장을 Java 코드로 표현하는 DSL을 제공합니다. 이는 SQL 쿼리를 Java 객체의 메소드 호출로 변환하여 쿼리를 작성할 수 있게 합니다.
다양한 데이터베이스 지원	JPA, SQL, MongoDB, Lucene 등 여러 데이터베이스에서 사용할 수 있는 모듈이 있습니다. 이를 통해 여러 유형의 데이터베이스에서 동일한 코드베이스를 사용할 수 있습니다.
동적 쿼리 생성	Querydsl을 사용하면 조건에 따라 쿼리를 동적으로 생성할 수 있습니다. 즉, 프로그램 실행 중에 필요에 따라 쿼리를 수정하거나 조합할 수 있습니다. 비슷한 쿼리를 재사용할 수 있으며 제약 조건 조립 및 가독성을 높일 수 있습니다.
모듈화	Querydsl은 각 데이터베이스에 대한 특정 모듈을 제공하여, 필요에 따라 원하는 기능을 선택하여 사용할 수 있습니다.
기타	IDE 사용시 자동 완성 기능이 되므로, 코드 생산성이 좋아집니다.

Spring Data JPA

Querydsl 설정 절차

Querydsl을 사용하기 위해서는 다음과 같은 순서대로 설정을 해주어야 합니다.



☰ Spring Data JPA

☰ 의존성 라이브러리 추가

Querydsl을 사용하기 위하여 다음과 같은 라이브러리를 pom.xml 파일에 추가해 줍니다.

Maven 의존성 추가

pom.xml 파일에 다음과 같이 Querydsl 의존성을 추가해 주어야 합니다.

만약 java.version 17 이상을 사용하는 SpringBoot 3.x 버전을 사용시 `<classifier>jakarta</classifier>` 코드를 추가해 주어야 합니다.

```
<dependency>
  <groupId>com.querydsl</groupId>
  <artifactId>querydsl-jpa</artifactId>
  <version>5.0.0</version>
</dependency>

<dependency>
  <groupId>com.querydsl</groupId>
  <artifactId>querydsl-apt</artifactId>
  <version>5.0.0</version>
  <classifier>jakarta</classifier>
</dependency>
```

<classifier> 속성

여기서 jakarta라는 classifier는, 해당 QueryDSL APT(Annotation Processing Tool)가 Jakarta EE 환경에서 사용되는 버전임을 나타냅니다.

<classifier> 속성은 이전의 Java EE에서 Jakarta EE로 명칭이 변경된 이후, Jakarta EE API를 사용하는 환경에 맞춘 특별한 artifact라는 의미로 사용되고 있습니다.

만약 Java EE 환경에서 사용해야 하는 경우라면, jakarta 대신 javax 등의 classifier를 사용하게 될 수 있습니다.

Spring Data JPA

Querydsl 관련 설치 및 설정

Querydsl을 사용하기 위하여 다음과 같은 라이브러리를 build.gradle 파일에 추가해 줍니다.

```
dependencies {  
    // 이전 설정들  
    // for QueryDSL  
    implementation 'com.querydsl:querydsl-jpa:5.0.0:jakarta' // QueryDSL JPA 모듈  
    annotationProcessor "com.querydsl:querydsl-apt:5.0.0:jakarta" // QueryDSL APT(Annotation Processing Tool) 모듈  
    annotationProcessor "jakarta.annotation:jakarta.annotation-api" // QueryDSL이 필요한 애노테이션을 처리  
    annotationProcessor "jakarta.persistence:jakarta.persistence-api" // QueryDSL이 JPA 엔티티를 처리하는 데 필요함  
}  
// for QueryDSL  
// 생성될 QueryDSL 타입 클래스가 저장될 디렉토리를 build/generated/querydsl으로 지정하고 있습니다.  
def querydslDir = layout.buildDirectory.dir("generated/querydsl").get().asFile  
  
// sourceSets는 프로젝트의 소스 파일 경로를 설정하는데, 기본 소스 세트(프로젝트의 주 코드)와  
// querydslDir 두 개의 디렉토리를 포함시켜, querydslDir에서 생성된 QueryDSL 클래스를 함께 사용할 수 있도록 합니다.  
sourceSets {  
    main {  
        java {  
            srcDirs = ['src/main/java', querydslDir]  
        }  
    }  
}  
// 애노테이션 프로세서가 생성하는 소스 파일의 경로를 설정합니다.  
// 이렇게 함으로써 QueryDSL이 생성한 클래스 파일들이 지정한 디렉토리에 저장되도록 합니다.  
tasks.withType(JavaCompile).configureEach {  
    options.annotationProcessorGeneratedSourcesDirectory = file(querydslDir)  
}
```

Gradle 의존성 추가

Spring Data JPA

Querydsl 관련 설치 및 설정

Q-타입 클래스는 데이터베이스 쿼리를 타입 안전하게 작성할 수 있도록 도와 주는 클래스입니다.

Q-타입 클래스 개요



Q-타입 클래스의 특징

Entity를 기반으로 Q가 붙는 클래스를 Qdomain 객체라고 합니다.
예를 들면 Product.java 클래스는 QProduct.java 클래스가 됩니다.
Querydsl은 Qdomain 객체를 사용하여 쿼리를 만들어 줍니다.
JPA 엔티티 클래스의 필드와 매핑되는 정적 필드를 포함하고 있습니다.
쿼리 작성시 매핑된 엔티티의 속성을 직접 참조할 수 있습니다.

Q-타입 클래스 설정

Querydsl은 Q-타입 클래스를 생성하기 위해 APT(Annotation Processing Tool)를 사용합니다.
IDE(인텔리J, 이클립스 등등)나 빌드 도구(maven, gradle 등등)에서 이 설정을 반드시 추가해야 합니다.

Spring Data JPA

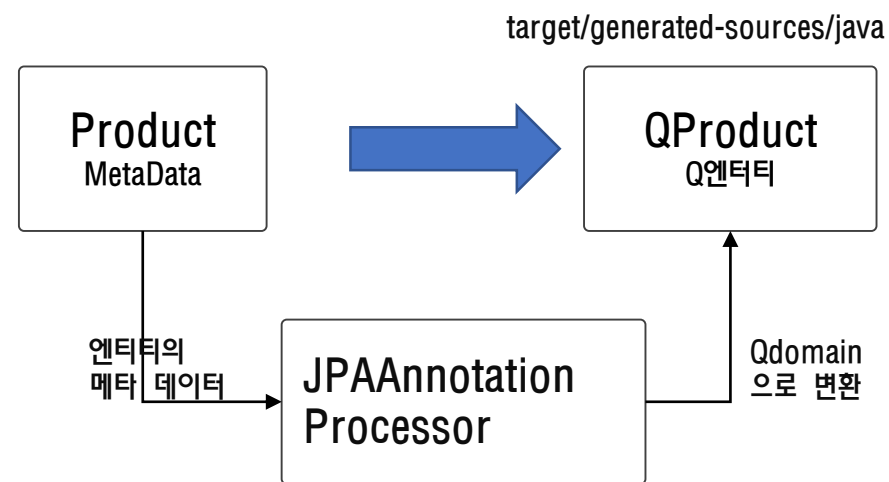
Querydsl 관련 설치 및 설정

Querydsl을 사용하기 위해서는 다음과 같은 설정을 해주어야 합니다.

Qdomain 생성 플러그인

pom.xml 파일에 Qdomain이라는 자바 코드를 생성하는 플러그인을 다음과 같이 추가해 줍니다.

```
<plugin>
  <groupId>com.mysema.maven</groupId>
  <artifactId>apt-maven-plugin</artifactId>
  <version>1.1.3</version>
  <executions>
    <execution>
      <goals>
        <goal>process</goal>
      </goals>
      <configuration>
        <outputDirectory>target/generated-sources/java</outputDirectory>
        <processor>com.querydsl.apt.jpa.JPAAnnotationProcessor</processor>
      </configuration>
    </execution>
  </executions>
</plugin>
```



Spring Data JPA

Querydsl 관련 설치 및 설정

추가된 의존성을 읽어 들이기 위하여 메이븐 프로젝트를 다시 로딩합니다.

좌측 상단의 버튼(Reload All Maven Projects)을 클릭하고 나서, 의존성 및 플러그인에 대하여 확인해 봅니다.

이제 maven compile 항목을 더블 클릭하도록 합니다.

작업할 내용

step01 : Reload All Maven Projects 버튼 클릭

step02 : maven - lifecycle - compile 더블 클릭

The image contains three screenshots from an IDE illustrating the steps to resolve the error:

- Left Screenshot:** Shows the Maven toolbar with the 'Reload All Maven Projects' button (a circular arrow icon) highlighted with a red box. Below, the 'shopping' project is expanded to show the 'Lifecycle' and 'Plugins' sections.
- Middle Screenshot:** Shows the 'Dependencies' list. The dependencies for 'com.querydsl:querydsl-jpa:5.0.0' and 'com.querydsl:querydsl-apt:5.0.0' are highlighted with a red box.
- Right Screenshot:** Shows the 'shopping' project expanded to the 'Lifecycle' section. The 'compile' goal is highlighted with a red box.

다음과 같이 오류가 발생하면 다음 페이지를 참조하도록 합니다.

Caused by: java.lang.ClassNotFoundException: javax.persistence.Entity

Spring Data JPA

Querydsl 관련 설치 및 설정

javax.persistence.Entity 오류 발생시 pom.xml 파일에 추가하도록 합니다.

java.lang.ClassNotFoundException: javax.persistence.Entity 오류는 Java Classpath에서 JPA(Java Persistence API) 관련 클래스를 찾지 못해 발생하는 문제입니다.

일반적으로 JPA 라이브러리가 프로젝트의 의존성으로 제대로 추가되지 않았거나, 런타임 환경에서 해당 클래스를 찾을 수 없는 경우에 발생할 수 있습니다.

persistence 관련

이전 실습에서 compile하는 동안 오류가 발생하였다면 다음과 같이 설정하고 다시 진행해 봅니다.

pom.xml 파일에 Qdomain이라는 자바 코드를 생성하는 플러그인을 다음과 같이 추가해 줍니다.

```
<dependency>
  <groupId>javax.persistence</groupId>
  <artifactId>javax.persistence-api</artifactId>
  <version>2.2</version> <!-- JPA API 버전 -->
</dependency>
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.1.Final</version> <!-- Hibernate 버전 -->
</dependency>
```

```
<dependency>
  <groupId>jakarta.persistence</groupId>
  <artifactId>jakarta.persistence-api</artifactId>
  <version>3.0.0</version>
</dependency>
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.1.Final</version>
</dependency>
```

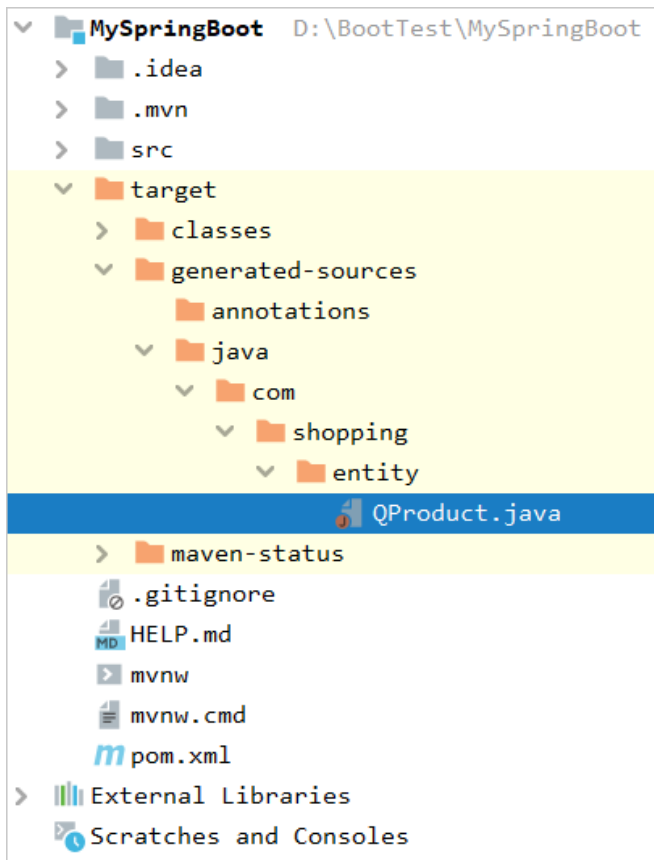
Spring Data JPA

Qdomain 생성 확인

Qdomain 객체가 생성이 되었는 지 확인하도록 합니다.

빌드가 완료되고 나면 target/generated-sources 폴더에 QProduct 클래스가 생성이 되었는 지 확인합니다.

QProduct 클래스에는 Product 클래스의 모든 컬럼들에 대하여 사용 가능한 operation을 호출할 수 있는 메소드들이 정의 되어 있습니다.



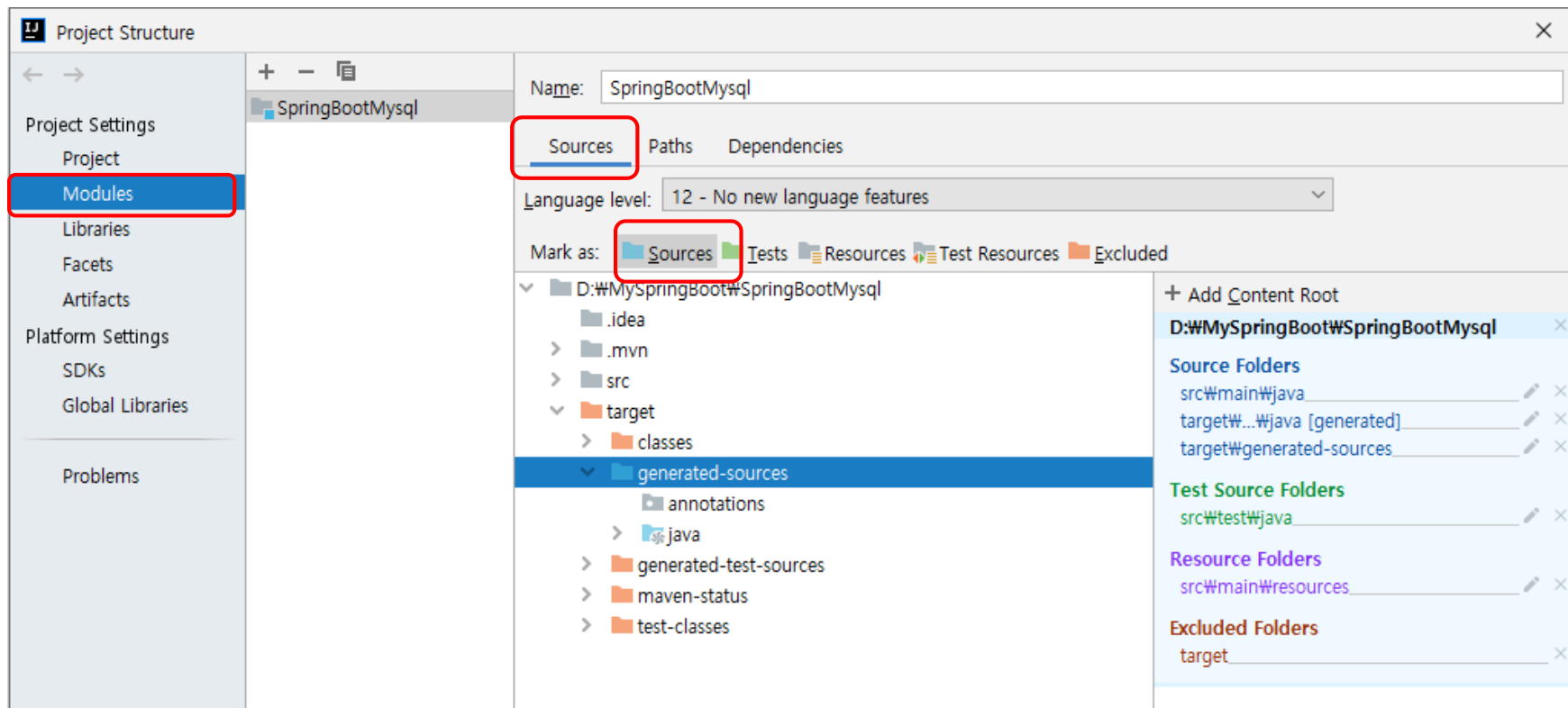
Spring Data JPA

Qdomain 생성 확인

인텔리J에서 Qdomain 객체 생성이 잘 되지 않는 경우는 다음과 같이 설정해 보도록 합니다.

[File]-[Project Structure]-[Module] 메뉴에서 target 폴더 아래의 generated-sources 폴더를 클릭한 다음 [Source] 버튼을 클릭하여 소스 코드로 인식할 수 있도록 처리합니다.

반영이 제대로 되지 않는 경우 [IntelliJ](#)를 다시 재시작하도록 합니다.



Spring Boot

Querydsl 관련 API



Spring



Boot



Security

≡ Querydsl 관련 API

≡ Querydsl 관련 API

QueryDsl에 사용할 수 있는 클래스 및 인터페이스 목록들을 정리해 봅니다.

클래스/인터페이스	설명
JPAQueryFactory	쿼리를 Build 시켜 주는 클래스로써, EntityManager와 JPQL를 사용하여 query 문장들을 처리해줍니다.
JPAQuery	JPQLQuery interface의 구현체로써, 쿼리 결과 집합을 구해주는 객체입니다.
@PersistenceContext	JPA에서, 영속성 컨텍스트를 주입(Injection)해주는 어노테이션입니다. 일반적으로 EntityManager를 주입하는 용도로 빈번하게 사용됩니다. 스프링의 Autowired 어노테이션과 유사한 개념입니다.
BooleanBuilder	쿼리 사용시 where 절의 조건식을 생성해주는 클래스입니다. BooleanBuilder is a cascading builder for Predicate expression 이 클래스는 Predicate 인터페이스를 상속 받고 있는 final 클래스입니다.
Page	쿼리 메소드의 리턴 타입 중의 하나입니다.
PageImpl	Page 인터페이스의 구현체입니다.
Pageable	JPA에서 페이징 처리를 편리하게 사용할 수 있도록 제공하는 인터페이스입니다.
PageRequest	Pageable 인터페이스를 상속 받는, 즉 Pageable의 구현체입니다. 부트 2.0 이후에는 PageRequest.of() 메소드를 사용합니다.
StringUtils	thymeleaf에서 사용되는 String 관련 식들을 검증하는 데 사용되는 final 클래스입니다. 예를 들어, 문자열 비교, 비어 있는 지 등을 검증할 때 사용할 수 있습니다.

Querydsl 관련 API

Querydsl 관련 API

JPAQuery 데이터 반환 메소드들을 정리해 봅니다.

JPAQuery는 QueryDSL을 사용할 때 자주 사용되는 클래스입니다.

JPAQuery를 사용하면 타입 안전하게 JPQL 쿼리를 작성하고, 다양한 메소드로 결과를 처리할 수 있습니다.

데이터 반환 메소드

메소드	설명
List<T> fetch()	조회되는 대상이 여러 개인 경우 list 컬렉션 정보를 반환해주고, 결과가 없으면 빈 리스트를 반환합니다.
T fetchOne()	조회 대상이 1건인 경우에 한하여 generic에 지정한 타입으로 반환됩니다. 결과가 없으면 null을 반환하고, 둘 이상의 결과가 있으면 예외(NonUniqueResultException)를 발생시킵니다.
T fetchFirst()	조회 건수가 여러 개인 경우에 대하여 1번째 건만 반환해줍니다. 내부 로직에서 return limit(1).fetchOne()으로 처리를 수행합니다.
Long fetchCount()	count 쿼리를 실행하여 데이터의 개수(count)를 반환해 줍니다.
QueryResults<T> fetchResults()	QueryResults는 페이징에 대한 정보와 데이터 목록을 가지고 있는 객체입니다. 페이징 처리를 위해 사용되는 메소드로, 조회한 리스트 목록과 전체 레코드 수를 함께 반환합니다.
long fetchCount()	쿼리의 결과 개수를 반환하며, 페이징이나 조건에 맞는 레코드 수를 알고 싶을 때 사용합니다.
stream()	쿼리 결과를 Java 8 Stream 형태로 반환합니다. 결과를 대량으로 처리하거나 스트리밍 방식으로 작업할 때 유용합니다.
iterate()	쿼리 결과를 CloseableIterator로 반환합니다. 주로 메모리 효율성을 높이기 위해 결과를 한 번에 로드하지 않고 필요할 때마다 처리하고자 할 때 사용됩니다.

Querydsl 관련 API

QueryResults 객체

`QueryResults` 객체는 페이지에 대한 정보와 데이터 목록을 가지고 있는 객체입니다.

쿼리 실행 결과를 처리하고자 할 때 사용하는 객체로, 특정한 조건이나 페이지가 적용된 결과를 캡슐화하여 제공하는데 사용됩니다.

쿼리 실행의 결과뿐만 아니라 메타 데이터를 포함하는 편리한 구조입니다.

예를 들면 결과 목록뿐만 아니라 전체 데이터의 개수, 페이지네이션 관련 정보 등을 같이 제공해 줍니다.

메소드	설명
List<T> <code>getResults()</code>	결과 집합을 List 컬렉션 형태로 반환해 줍니다.
long <code>getTotal()</code>	쿼리가 반환하는 전체 데이터의 개수를 제공합니다.
long <code>getLimit()</code>	페이지가 적용된 쿼리에서 제한된 데이터 수를 반환합니다. `limit`는 한 페이지에 표시될 최대 데이터 수를 의미합니다.
long <code>getOffset()</code>	쿼리 결과에서 데이터가 어느 위치에서부터 시작되는지 알려줍니다. 이는 페이지 시에 특정 페이지의 시작점을 나타냅니다.
boolean <code>isEmpty()</code>	결과가 비어 있는지 여부를 확인합니다. 결과 리스트가 비어 있으면 `true`, 그렇지 않으면 `false`를 반환합니다.

주의 사항

`QueryResults`는 성능 상의 이유로 큰 데이터셋을 다룰 때 주의해서 사용해야 합니다.

모든 데이터를 조회하고 나서 페이지를 적용하기 때문에 데이터베이스 쿼리가 복잡해지거나 대용량 데이터를 처리할 경우 성능 저하가 발생할 수 있습니다.

최근에는 Pageable 인터페이스와 함께 Spring Data JPA의 기본 기능을 많이 사용하는 추세이므로, 경우에 따라 `QueryResults` 대신 Spring Data의 Page<T>를 사용하는 것도 고려할 만합니다.

≡ Querydsl 관련 API

≡ QueryResults 객체

`QueryResults` 객체는 페이지징에 대한 정보와 데이터 목록을 가지고 있는 객체입니다.

사용 예시

예시에서는 `offset`과 `limit`을 사용하여 데이터를 페이지 단위로 나누고, `fetchResults()`로 전체 데이터를 가져온 후, 결과와 전체 개수를 각각 얻습니다.

```
QueryResults<User> queryResults = queryFactory
    .selectFrom(user)
    .where(user.age.gt(20))
    .offset(pageable.getOffset())
    .limit(pageable.getPageSize())
    .fetchResults();

List<User> users = queryResults.getResults();
long total = queryResults.getTotal();
```


Querydsl 관련 API

Querydsl 관련 API

`Pageable`은 Spring Data JPA에서 페이징 처리 기능을 제공하는 인터페이스입니다.

페이징 처리는 대규모 데이터를 일정한 크기로 분할하여 필요한 만큼만 조회할 수 있게 해줍니다.

Pageable을 이용하면 쿼리 결과를 페이지 단위로 나누고, 특정 페이지의 데이터를 쉽게 조회할 수 있습니다.

패키지 경로는 `import org.springframework.data.domain.Pageable;` 입니다.

주요 기능

항목	설명
페이지 번호 및 사이즈 설정	페이징 요청 시, 몇 번째 페이지를 가져올 것인지, 그리고 한 페이지에 몇 개의 항목을 포함할지를 설정할 수 있습니다.
정렬 기능	특정 속성을 기준으로 데이터를 정렬할 수 있습니다. 이를 통해 조회된 결과를 원하는 순서대로 나열할 수 있습니다.
페이징 요청 객체 생성	PageRequest 클래스는 Pageable 인터페이스를 구현한 대표적인 구현체입니다. 페이징과 정렬을 함께 설정할 수 있는 객체입니다.

PageRequest의 `of()`와 관련된 기능은 다음과 같은 항목들이 존재합니다,.

메소드	설명
<code>PageRequest.of(int page, int size)</code>	페이지 번호(0 base index), 페이지당 데이터의 개수
<code>PageRequest.of(int page, int size, Sort.Direction direction, String...props)</code>	페이지 번호, 페이지당 데이터의 개수, 정렬 방향, 여러 속성들
<code>PageRequest.of(int page, int size, Sort sort)</code>	페이지 번호, 페이지당 데이터의 개수, 정렬 방향 <code>Sort.by("컬럼이름").ascending() ↔ descending()</code>

☰ Querydsl 관련 API

☰ Querydsl 관련 API

`Pageable`은 Spring Data JPA에서 페이징 처리 기능을 제공하는 인터페이스입니다.

주요 메소드

항목	설명
<code>getPageNumber()</code>	현재 요청된 페이지 번호를 반환합니다. 페이지 번호는 0부터 시작합니다.
<code>getPageSize()</code>	한 페이지에 몇 개의 항목이 포함되는지를 반환합니다.
<code>getOffset()</code>	데이터베이스에서 조회할 때, 해당 페이지의 첫 번째 항목이 전체 목록에서 몇 번째인지(오프셋)를 반환합니다.
<code>getSort()</code>	현재 페이지 요청에 설정된 정렬 조건을 반환합니다. 정렬을 설정하지 않았다면 기본적으로 정렬 조건이 없는 상태로 반환됩니다.
<code>next()</code>	다음 페이지 요청을 반환합니다. 현재 페이지보다 한 페이지 뒤의 `Pageable` 객체를 생성합니다.
<code>previousOrFirst()</code>	이전 페이지를 반환하며, 이전 페이지가 없을 경우 첫 번째 페이지를 반환합니다.
<code>first()</code>	첫 번째 페이지 요청을 반환합니다.
<code>hasPrevious()</code>	이전 페이지가 있는지를 확인합니다. 이전 페이지가 존재하면 `true`, 그렇지 않으면 `false`를 반환합니다.

≡ Querydsl 관련 API

≡ Querydsl 관련 API

`JPAQueryFactory` 객체는 쿼리를 Build 시켜 주는 클래스입니다.

EntityManager와 JPQL를 사용하여 query 문장들을 처리해줍니다.

JPAQueryFactory 객체가 가질수 있는 메소드 목록은 다음과 같습니다.

조건식을 판단하여 결과가 true인 데이터들만 반환해 주는 클래스입니다.

BooleanExpression 추상 클래스는 Predicate 인터페이스를 상속 받고 있습니다.

메소드	설명
selectFrom()	엔터티 이름을 지정합니다.(Qdomain을 사용)
where()	eq(), like() 등의 조건식을 지정합니다. BooleanExpression 클래스를 사용하여 처리할 수 있습니다.
orderBy()	정렬 방식을 지정합니다.
offset()	페이징 처리의 offset 기능으로, 데이터를 가지고 올 시작 인덱스를 지정합니다.
limit()	페이징 처리의 limit 기능으로, 한번에 읽어 올 최대 행수를 지정합니다.

사용 예시

```
JPAQuery<Product> query = queryFactory.selectFrom(qProduct)
    .where(qProduct.productStatus.eq(ProductStatus.SELL))
    .where(qProduct.description.like("%" + "테스트 상품 상세 설명" + "%"))
    .orderBy(qProduct.price.desc());
```

≡ Querydsl 관련 API

≡ Querydsl 관련 API

`BooleanBuilder`는 QueryDSL에서 동적 쿼리를 생성할 때 유용하게 사용할 수 있습니다.

주요 특징

조건이 많아질수록 코드의 가독성과 유지 보수성을 높일 수 있습니다.

`BooleanExpression`을 누적시켜서 조건을 구성합니다.

특정 조건을 동적으로 추가하거나 제외할 수 있습니다.

`WHERE` 절에 포함될 조건을 유연하게 조립할 수 있습니다.

`OR`나 `AND` 조건을 통해 여러 개의 필터링 조건을 동적으로 추가할 수 있게 도와줍니다.

주요 메소드

항목	설명
<code>and()</code>	AND 조건으로 새로운 조건을 추가합니다.
<code>or()</code>	OR 조건으로 새로운 조건을 추가합니다.
<code>not()</code>	조건을 반전시킵니다.
<code>getValue()</code>	누적된 `BooleanExpression`을 반환하여 최종 쿼리에 적용합니다.

Querydsl 관련 API

Querydsl 관련 API

`BooleanExpression` 클래스는 QueryDSL 라이브러리에서 논리 표현식을 다루기 위한 클래스입니다.

조건문 기반 검색, 필터링 또는 복잡한 쿼리 빌드를 위해 매우 유용합니다.

Predicate의 하위 클래스이며, 여러 조건을 결합하여 복잡한 논리식을 구성할 수 있도록 도와줍니다.

주로 QueryDSL의 `Q` 클래스를 사용해 필드에 대한 비교 연산을 수행할 때 생성됩니다.

주요 메소드

대분류	항목	설명
논리 연산	and()	두 개의 조건을 논리적으로 AND 연산으로 결합합니다.
	or()	두 개의 조건을 논리적으로 OR 연산으로 결합합니다.
	not()	조건을 NOT 연산으로 반전시킵니다.
비교 연산	eq()	특정 값과 일치하는지 비교합니다.
	ne()	특정 값과 일치하지 않는지 비교합니다.
	gt(), goe(), lt(), loe()	크기 비교를 위해 사용됩니다.
널 처리 연산	isNull()	필드가 null인지 확인합니다.
	isNotNull()	필드가 null이 아닌지 확인합니다.
문자열 관련	contains(), containsIgnoreCase()	특정 문자열을 포함하는지 확인합니다.
	startsWith(), endsWith()	특정 문자열로 시작하거나 끝나는지 확인합니다.

≡ Querydsl 관련 API

≡ Querydsl 관련 API

`Page<T>` 타입은 list 객체의 sublist를 표현하기 위한 객체입니다.

Page<T> 타입은 Spring MVC와 연동시 상당한 편리함을 제공해 줍니다.

Spring Data JPA에서 목록을 다루기 위하여 List 타입을 사용하기도 하지만, MVC와 연동을 하고자 할 때 Page<T> 타입을 이용하면 편리합니다.

속성	설명
content	페이지에 포함된 엔터티 목록을 나타냅니다.
empty	페이지가 비어 있는지 여부를 나타냅니다.
first	현재 페이지가 첫 번째 페이지인지 여부를 나타냅니다.
last	현재 페이지가 마지막 페이지인지 여부를 나타냅니다.
number	현재 페이지 번호(0부터 시작)를 나타냅니다.
numberOfElements	현재 페이지의 엔터티 수를 나타냅니다.
size	페이지 크기를 나타냅니다.
totalElements	전체 엔터티 수를 나타냅니다.
totalPages	전체 페이지 수를 나타냅니다.

≡ Querydsl 관련 API

≡ Querydsl 관련 API

`Page<T>` 타입은 list 객체의 sublist를 표현하기 위한 객체입니다.

메소드	설명
<code>int getNumber()</code>	현재 페이지 번호를 반환합니다.
<code>int getSize()</code>	한 페이지의 크기를 반환합니다.
<code>int getTotalPages()</code>	전체 페이지 수를 반환합니다.
<code>int getNumberOfElements()</code>	현재 페이지에 나올 데이터 수
<code>long getTotalElements()</code>	전체 데이터 개수를 반환합니다.
<code>boolean hasPreviousPage()</code>	이전 페이지의 존재 유무를 반환합니다.
<code>boolean isFirstPage()</code>	현재 페이지가 첫번째 페이지 인지 여부
<code>boolean hasNextPage()</code>	다음 페이지의 존재 유무를 반환합니다.
<code>boolean isLastPage()</code>	현재 페이지가 마지막 페이지 인지 여부
<code>Pageable nextPageable()</code>	다음 페이지 객체, 존재하지 않으면 null을 반환합니다.
<code>Pageable previousPageable()</code>	다음 페이지 객체, 존재하지 않으면 null을 반환합니다.
<code>List<T> getContent()</code>	조회된 데이터 목록을 반환합니다.
<code>boolean hasContent()</code>	조회된 데이터 존재 유무
<code>Sort getSort()</code>	정렬에 대한 정보를 반환합니다.

≡ Querydsl 관련 API

≡ Querydsl 관련 API

QueryDslPredicateExecutor is Interface to allow execution of QueryDsl Predicate instances.

파일 이름 com.shopping.repository.ProductRepository

Predicate란 '이러한 조건이 맞다'라고 판단하는 근거를 함수로 제공하는 것입니다.

Predicate를 파라미터로 전달하는 방법은 Repository가 QuerydslPredicateExecutor를 상속 받으면 가능합니다.

```
public interface ProductRepository
    extends JpaRepository<Product, Long>, QuerydslPredicateExecutor<Product> {
    ... 코딩 내용
}
```

QuerydslPredicateExecutor 인터페이스는 다음과 같은 메소드들을 구현하고 있습니다.

<https://docs.spring.io/spring-data/commons/docs/current/api/org/springframework/data/querydsl/QuerydslPredicateExecutor.html>

메소드	설명
long count(Predicate)	조건에 맞는 데이터의 총 개수를 반환해줍니다.
boolean exists(Predicate)	조건에 맞는 데이터가 존재하는 지를 파악해줍니다.
Iterable findAll(Predicate)	조건에 맞는 모든 데이터를 반환해줍니다.
Page<T> findAll(Predicate, Pageable)	조건에 맞는 페이지 데이터를 반환해줍니다.
Iterable findAll(Predicate, Sort)	조건에 맞는 정렬된 데이터를 반환해줍니다.
findOne(Predicate)	조건에 맞는 데이터 1개를 반환해줍니다.

Spring Boot

Thymeleaf(타임 리프) 학습하기



Spring



Boot



Security

ServerSide Rendering

server 측에서 미리 정의된 template를 이용하여 html 결과를 동적으로 client에서 전달하는 기술을 말합니다.

서버 사이드 렌더링

Server

template

html 전달

Client

서버 사이드 템플릿 엔진

- Thymeleaf(타임 리프)
- JSP
- FreeMarker
- Groovy
- Mustache 등등

Rendering

HTML, CSS, JavaScript 등을 브라우저에서 해석하여 실제 화면에 그리는 과정을 Rendering이라고 합니다. 이때, Rendering 속도는 웹 페이지의 성능에 영향을 미치는 중요한 요소 중 하나입니다.

Thymeleaf(타임 리프) 소개

컨트롤러가 전달하는 데이터를 이용하여 동적으로 화면을 구현해 주는 역할을 합니다.

요구 사항

관련 의존성 파일에 다음과 같이 주가가 되어 있는 지 확인합니다.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

항목	설명
정의	컨트롤러가 전달해주는 데이터를 보여 주는 View Template(뷰 템플릿)입니다.
특징	진입 장벽이 낮고 쉽게 배울 수 있다는 장점이 있습니다. html 태그를 기반으로 하여 "th:속성"을 이용하여 동적인 View를 제공합니다. 확장자는 html 입니다. 'natural template'이란 브라우저에 파일을 띄우면, 오류가 발생하지 않고 th 문법이 그대로 보인다는 것을 설명해주는 용어입니다.
참조 사이트	https://www.thymeleaf.org/

Thymeleaf(타임 리프) 소개

Thymeleaf 변수식은 다음과 같은 순서대로 코딩합니다.

변수식 사용 순서

<html> 태그에 네임 스페이스를 지정합니다.

```
<html xmlns:th="http://www.thymeleaf.org">
```

변수식을 사용할 위치에서 `${expr}` 등의 데이터 표현 방법을 사용하여 값을 표현하면 됩니다.
여기서, `expr`는 임의의 변수 또는 객체 이름을 의미합니다.

Thymeleaf 변수식

`${expr}`

Thymeleaf(타임 리프) 소개

Thymeleaf 변수식은 다음과 같은 순서대로 코딩합니다.

데이터 표현 방법

항목	설명	비고/예시
<code>\${name}</code>	변수 name의 값을 출력합니다.	<code>\${name}</code> 님~~안녕하세요.
<code>*{age}</code>	특정 객체에 들어 있는 변수(프로퍼티) age의 값을 출력합니다.	<code><input type="text" th:field="**{name}"></code>
<code>#{ }</code>	메시지 혹은 properties 파일에 들어 있는 내용 읽기	
<code>@{ }</code>	하이퍼 링크를 지정합니다.	th:formaction이나 th:href 등에 사용됩니다. <code><a th:href="@{https://www.thymeleaf.org/}">thymeleaf home</code>

변수식 사용 예시

예시	설명
<code><p th:text="\${data}"></p></code>	data라는 데이터를 <code><p></code> 태그에 보여 줍니다.
<code></code>	member 객체의 변수 name라는 정보를 <code></code> 태그에 보여 줍니다.

Model 인터페이스

Controller에서 데이터를 저장할 용도로 사용됩니다.

Thymeleaf를 사용하여 데이터를 저장하고, 뷰에서 해당 데이터를 확인하기 위하여 우선 Model이라는 개념부터 살펴 보겠습니다.

Model 인터페이스는 데이터를 저장(바인딩)하기 위하여 제공되는 인터페이스입니다.

유사한 클래스로 ModelMap 클래스는 Model 인터페이스와 동일한 메소드를 제공하고 있습니다.

다음과 같이 메소드 체이닝 기능을 제공해 줍니다.

```
model.addAttribute("항목1", 값1).addAttribute("항목2", 값2) ;
```

메소드	설명
Model addAttribute(String attrName, Object attrValue)	모델 속성을 추가합니다.
Model addAllAttributes(Map<String, ?> attributes)	맵의 <키, 값> 쌍들을 모델 속성의 이름과 값으로 추가합니다.
boolean containsAttribute(String attrName)	이름이 attrName인 모델 속성이 포함되어 있는 경우 값 true을 리턴합니다.
옵션	required - @Autowired어노테이션을 적용한 프로퍼티에 대해 굳이 설정할 필요가 없는 경우에 false값을 주며 이때 해당 프로퍼티가 존재하지 않더라도 스프링은 예외를 발생시키지 않습니다. 디폴트값은 true입니다.

어노테이션(@RequestMapping)

클라이언트가 요청한 url과 컨트롤러 메소드의 맵핑을 설정해 줍니다.

@RequestMapping 어노테이션은 5.x 버전으로 올라 오면서 @GetMapping, @PostMapping으로 분리되었습니다.

항목	설명
목적	클라이언트가 요청한 url과 컨트롤러 메소드의 맵핑을 설정해 줍니다.
설정 위치	메소드 선언 부
특징	모든 속성은 배열 초기화 기법을 이용하여 여러 개 설정이 가능합니다.
용어	컨트롤러 메소드란 해당 요청이 들어 왔을 때 수행이 되는 메소드를 말합니다.
속성 이름	설명
value	"value=~/getMovie.do"와 같은 형식의 매핑 URL 값입니다. 디폴트 속성이기 때문에 value만 정의하는 경우에는 'value='은 생략할 수 있습니다. @RequestMapping(value={"/addMoviedo", "/updateMoviedo" })와 같은 2개의 URL 모두를 처리합니다.
method	GET, POST, HEAD 등으로 표현 되는 HTTP Request method에 따라 requestMapping을 할 수 있습니다. 'method=RequestMethod.GET' 형식으로 사용합니다. method 값을 정의하지 않는 경우 모든 HTTP Request method에 대해서 처리하겠다는 의미입니다.
params	HTTP Request로 들어오는 파라미터 목록을 의미합니다.

Thymeleaf 예제 진행하기

다음은 Thymeleaf에서 자주 사용되는 문법들에 대한 설명입니다.

항목	설명
th:text	문자열을 출력합니다.
th:each	반복문을 사용하고자 할 때 사용됩니다.
th:if	흐름 분기에 사용됩니다.
th:unless	if와 같이 사용하고, 타 프로그래밍의 else절에 해당합니다.
th:switch	switch...case 구문의 switch 절에 해당합니다.
th:case	switch...case 구문의 case 절에 해당합니다.
th:href	링크를 처리해주는 문법입니다. <code><a th:href="/admin/products/~\${products.id}" \${products.name}"></code>

Thymeleaf 예제 진행하기

Thymeleaf에 대한 몇 가지 예시를 살펴 봅니다.

th:each 구문

th:each 구문은 자바의 확장 for 구문과 유사한 개념으로 Thymeleaf 템플릿 엔진에서 반복문을 처리하는 방법 중 하나입니다.

컬렉션 또는 배열과 같은 객체의 요소를 반복하여 HTML 코드를 생성할 수 있습니다.

status는 현재 반복에 대한 상태 정보를 저장하고 있는 변수로 임의의 이름이어도 상관없으며, optional 값입니다.

status.index에서 index는 0부터 시작합니다.

```
<tr th:each="단수이름, status: ${복수이름}">
  <td th:text="${status.index}"></td>
  <td th:text="${단수이름.변수이름}"></td>
</tr>
```

status 속성	설명
index	순번을 의미하고, 0부터 시작합니다.
count	순번을 의미하고, 1부터 시작합니다.
size	중 요소 개수를 의미합니다.
current	매번 반복시 마다 추출되는 데이터
even	짝수이면 true를 반환해 줍니다.
odd	홀수이면 true를 반환해 줍니다.
first	현재 iteration이 첫 번째 요소이면 true를 반환해 줍니다.
last	현재 iteration이 마지막 요소이면 true를 반환해 줍니다.

Thymeleaf 예제 진행하기

Thymeleaf에 대한 몇 가지 예시를 살펴 봅니다.

th:href 구문

th:href 구문은 링크를 처리해주는 문법에 사용됩니다.

링크의 유형	설명	예시
Absolute Url 절대 주소	'http://' 또는 'https://'와 같이 이동할 서버의 주소를 직접 입력하는 방식입니다.	<a th:href="@{https://www.thymeleaf.org/}">페이지
Relative Url 상대 주소	우리가 실행하는 application 내부를 이동하는 방식입니다.	<a th:href="@{/thymeleaf/ex01}">페이지
-	-	<a th:href="@{/thymeleaf/ex07(param1 = '파라미터01')}">파라미터 전달

Thymeleaf 페이지 레이아웃

웹사이트의 공통 요소를 쉽게 관리해주는 기능입니다.

공통 요소란?

일반적인 웹 사이트에서 모든 페이지에 거의 공통적으로 사용되는 내용들이 있습니다.
예를 들면 header, footer, menu 등이 여기에 해당됩니다.
이러한 요소들을 공통 요소라고 합니다.

maven 의존성 추가

Thymeleaf Layout Dialect를 사용하면 여러 페이지에 사용하는 공통 요소를 만들 수 있습니다.

참조 사이트) <https://ultraq.github.io/thymeleaf-layout-dialect/>

pom.xml 파일에서 다음 코드를 작성하고 Maven Dependencies에서 라이브러리가 추가되었는지 확인합니다.

```
<dependency>
  <groupId>nz.net.ultraq.thymeleaf</groupId>
  <artifactId>thymeleaf-layout-dialect</artifactId>
</dependency>
```

gradle 의존성 추가

```
dependencies {
    implementation 'nz.net.ultraq.thymeleaf:thymeleaf-layout-dialect:3.0.0'
}
```

Thymeleaf 페이지 레이아웃

공통 요소를 만들어 봅니다.

페이지 레이아웃 구현 순서

항목	설명	예시
포함될 파일	th:fragment를 사용한 곳은 다른 페이지에 포함되는 영역이라는 의미입니다. HTML 문서의 일부분을 여러 문서에서 반복적으로 사용해야 할 때 유용합니다.	<pre><div class="footer" th:fragment="qwert"> 푸터 영역 </div></pre>
포함하고자 하는 파일	layout 네임 스페이스를 지정합니다. th:replace를 사용하여 해당 영역을 치환합니다. layout:fragment를 사용하여 변경이 되는 영역을 지정합니다.	<pre><html xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"> <div th:replace="fragments/footer::qwert"></div> <div layout:fragment="content" class="content"></div></pre>

Thymeleaf에 사용될 문법

항목	설명
th:fragment	다른 페이지에 포함이 될 영역을 지정하기 위한 문장입니다.
th:block	블록을 구성하기 위한 태그입니다.
th:replace	해당 태그의 내용을 다른 html 파일 내의 태그로 치환하는 기능입니다.
layout:fragment	각 페이지마다 다르게 구현해야 하는 영역을 지정하는 문법입니다.
layout:decorate	레이아웃 파일의 위치를 지정하기 위하여 사용하는 네임 스페이스입니다.

부트스트랩으로 header, footer 영역 수정하기

웹 사이트를 쉽게 만들기 위한 도와주는 html, css, js 프레임워크입니다.

구현할 내용

레이아웃 파일([mylayout.html](#))에 Bootstrap과 관련된 CDN을 추가합니다.

```
<!-- CSS only -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
<link th:href="@{/css/mylayout.css}" rel="stylesheet">

<!-- JS, Popper.js, and jQuery -->
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

CDN

CDN(Contents Delivery Network)란 물리적으로 멀리 떨어져 있는 개발자들에게 Contents를 조금이라도 더 빨리 제공해주기 위한 서비스를 의미합니다. 일종의 Cache 서버를 이용하여 좀더 빠르게 서비스를 받기 위한 개념입니다.



Spring Boot

회원 가입 및 로그인



Spring



Boot



Security

스프링 boot API

<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>

≡ Spring Security 소개

스프링 기반의 애플리케이션을 위한 보안 솔루션을 제공하는 역할을 합니다.

Spring Security는 로그인 기능, 관리자 권한 등과 기능에 대하여 인증과 인가 등의 기능을 구현하는 기술입니다.

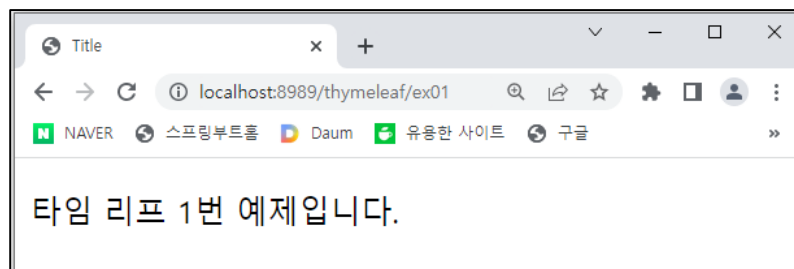
항목	예시	설명
인증 (Authentication)	신원 검증 (by 비번, 생체 인식, 일회용 Pin)	해당 리소스에 대해서 작업을 수행할 수 있는 주체인지 확인하는 절차입니다. 회원 가입 하고 로그인 을 수행 하는 것을 말합니다. 댓글 또는 상품 구매를 위해서는 로그인이라는 인증 절차가 필요합니다. Personal Identification Number은 사용자 식별을 위한 짧은 숫자만으로 이루어진 비밀 번호를 의미합니다.
인가 (Authorization)	자원(페이지/기능) 접근 권한	인증 절차를 거친 다음에 해당 url에 대하여 인가된 회원인지를 검사하는 기능입니다. 인증된 사용자가 실제로 사용할 수 있는 기능이나 서비스를 결정하는 것입니다. 특정 기능에 권한을 부여해주는 능력을 의미합니다.

인증 예시

항목	게시물 등록	댓글 등록
미로그인	0	X
로그인	0	0

요구 사항

Security 기능을 사용하기 전에 비교를 하기 위하여 다음과 같이 웹 브라우저에서
‘http://localhost:8989/thymeleaf/ex01’를 입력하고 엔터 키를 눌러봅니다.



≡ 회원 가입 및 로그인

≡ Spring Security 설정 추가하기

Security 설정에 필요한 의존성 라이브러리는 다음과 같습니다.

Security 의존성 추가

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-security'
}
```

기본 계정

의존성이 추가된 다음 메인 파일인 SpringBootApplication 클래스를 실행해 봅니다.
이때 기본적으로 제공하는 계정 정보의 비밀번호를 콘솔창에서 확인할 수 있습니다.

다음은 메인 프로그램을 실행했을 때 콘솔창에 보이는 비밀번호 정보입니다.

실행시마다 매번 다를 수 있습니다.

Using generated security password: [8ce56232-bd95-4699-97ff-b0b05a5f82df](#)

Dependencies

- > org.springframework.boot:spring-boot-starter-data-jpa:2.6.7
- > org.springframework.boot:spring-boot-starter-thymeleaf:2.6.7
- > org.springframework.boot:spring-boot-starter-web:2.6.7
- > org.springframework.boot:spring-boot-starter-web-services:2.6.7
- > org.springframework.boot:spring-boot-devtools:2.6.7 (runtime)
- com.h2database:h2:1.4.200 (runtime)
- com.oracle.database.jdbc:ojdbc8:21.3.0.0 (runtime)
- org.projectlombok:lombok:1.18.24
- > org.springframework.boot:spring-boot-starter-test:2.6.7
- > com.querydsl:querydsl-jpa:5.0.0
- > com.querydsl:querydsl-apt:5.0.0
- > nz.net.ultraq.thymeleaf:thymeleaf-layout-dialect:3.0.0
- > org.springframework.boot:spring-boot-starter-security:2.6.7

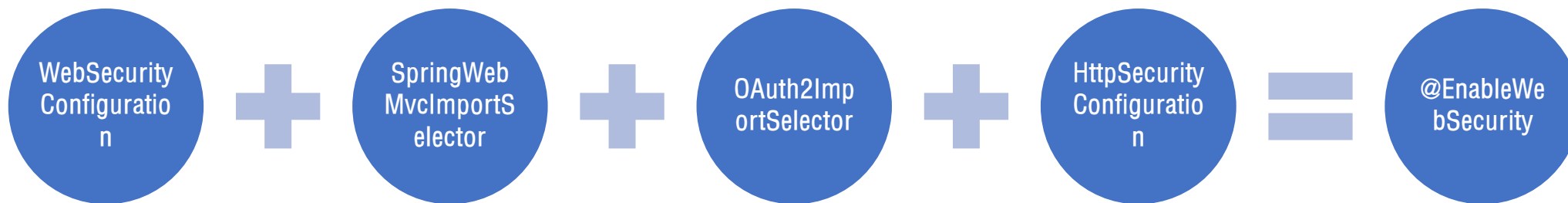
항목	설명
아이디	user
비밀 번호	콘솔창에 출력됩니다.

≡ `@EnableWebSecurity

웹 관련 Security 설정을 활성화해주는 어노테이션입니다.

다음 클래스들을 임포트하여 Spring Security 기능을 사용할 수 있도록 해줍니다.

일반적으로 WebSecurityConfigurerAdapter 클래스를 상속 받아서 클래스를 구현하는 방식을 많이 사용합니다.



일반적으로 WebSecurityConfigurerAdapter 클래스를 상속 받아서 클래스를 구현하는 방식을 많이 사용합니다.

configure() 메소드를 오버라이딩하여 다양한 보안 정책을 정의할 수 있습니다.

항목	설명
특징	SpringSecurityFilterChain이 자동으로 포함됩니다.
설정 위치	클래스 선언 부

@EnableWebSecurity 어노테이션의 효과

Spring Security의 기본 보안 기능이 활성화됩니다.

사용자가 정의한 보안 구성이 Spring의 보안 필터 체인에 추가됩니다.

커스텀 보안 구성을 통해 상세한 권한 제어, 인증 절차 설정 등을 구현할 수 있습니다.

≡ `SecurityFilterChain` 클래스

웹 애플리케이션의 요청을 처리하는 보안 필터 체인을 설정하는 데 사용하는 클래스입니다.

애플리케이션의 인증(authentication)과 권한 부여(authorization)를 설정하고, 로그인 및 로그 아웃 처리 방식 등을 구성할 수 있습니다.

요청이 애플리케이션에 도달하기 전에 다양한 보안 검사를 수행할 수 있도록 합니다.

SecurityFilterChain는 일반적으로 `HttpSecurity` 객체를 사용하여 구성이 됩니다.

이 클래스는 다양한 보안 설정을 구성하는 데 사용되는 빌더 클래스입니다.

기본 개념

항목	설명
Filter	HTTP 요청과 응답을 가로 채어 처리하는 컴포넌트입니다. Spring Security에서는 여러 가지 보안 필터가 있으며, 이들 필터를 순차적으로 실행하여 요청을 검증합니다.
Chain	여러 개의 필터가 연결된 구조입니다. 각각의 필터는 특정 기능을 담당하며, 필터 체인은 요청을 처리하는 데 필요한 일련의 작업을 정의합니다.

주요 필터

항목	설명
UsernamePasswordAuthenticationFilter	사용자 이름과 비밀번호를 사용하여 인증을 처리합니다.
BasicAuthenticationFilter	HTTP Basic Authentication을 처리합니다.
SecurityContextPersistenceFilter	`SecurityContext`를 HTTP 세션에 저장하고 복원합니다.
ExceptionHandler	보안 예외를 처리하고 사용자에게 적절한 응답을 반환합니다.
FilterSecurityInterceptor	요청에 대한 권한을 검사합니다.

≡ `HttpSecurity` 클래스

http 요청에 대한 보안 설정(페이지 권한, 로그인 페이지 설정 등등)을 적용해주는 클래스입니다.

HTTP 요청에 대한 보안 규칙을 정의하고 인증 및 인가를 설정하는 데 사용됩니다.

`HttpSecurity`를 사용하면 다음과 같은 설정을 할 수 있습니다.

인증 전체 흐름에 필요한 Login, Logout 페이지 인증 완료 후 페이지 인증 실패 시 이동 페이지 등등의 설정이 가능합니다.

인증 로직을 커스터م하기 위한 커스텀 필터 설정이 가능합니다.

기타 csrf, 강제 https 호출 등등의 Spring Security 설정이 가능합니다.

항목	설명
인증 및 인가(Authentication and Authorization)	리소스(URL)에 대한 접근 권한을 설정합니다. 예를 들면 어떤 경로는 인증이 필요하고 어떤 경로는 허용될지 설정할 수 있습니다. 예를 들어, 로그인 페이지나 리소스 파일에 대한 접근을 허용하거나 제한할 수 있습니다.
CSRF 보호	기본적으로 CSRF(Cross-Site Request Forgery) 보호가 활성화되어 있으며, 필요에 따라 비활성화할 수 있습니다.
세션 관리	세션 관리를 통해 세션 고정 공격(session fixation)을 방지하거나 세션을 설정할 수 있습니다.
폼 로그인(Form Login)	기본 제공 로그인 페이지를 사용하거나 커스텀 로그인 페이지를 정의할 수 있습니다.
로그 아웃(Logout)	로그 아웃 URL과 이후 리다이렉트 경로 등을 설정할 수 있습니다.
HTTP 기본 인증 (Basic Authentication)	Basic 인증 방식을 활성화하거나 설정할 수 있습니다.

≡ `CSRF

사용자가 인증된 상태에서 의도치 않은 요청을 보내는 공격을 의미합니다.

CSRF(Cross-Site Request Forgery)는 주로 웹 애플리케이션에서 발생합니다.

악의적인 사이트에서 사용자의 인증 정보를 이용해 해당 사용자가 의도하지 않은 작업을 수행하도록 유도합니다.

예를 들어, 사용자가 로그인한 상태에서 돈 이체, 정보 수정 등을 몰래 수행할 수 있습니다.

CSRF 보호

스프링 부트는 기본적으로 CSRF 공격에 대한 보호 기능을 제공합니다.

CSRF 보호는 일반적으로 다음과 같은 방식으로 작동합니다.

항목	설명
CSRF 토큰	스프링 시큐리티는 각 요청에 CSRF 토큰을 포함시켜야 하며, 이 토큰은 서버가 생성하고 클라이언트가 요청 시 함께 보내야 합니다. 서버는 이 토큰을 검증하여 요청이 유효한지를 판단합니다.
폼 기반 보호	HTML 폼에 CSRF 토큰을 포함시키는 것이 일반적입니다. 이를 통해 폼 제출 시 서버가 CSRF 공격을 방어할 수 있습니다.
HTTP 헤더 보호	AJAX 요청을 할 때, CSRF 토큰을 HTTP 헤더에 포함시켜 서버에 요청을 보낼 수 있습니다.

≡ `HttpSecurity` 클래스

http 요청에 대한 보안 설정(페이지 권한, 로그인 페이지 설정 등등)을 적용해주는 클래스입니다.

관련 Api

HttpSecurity 클래스와 관련된 메소드는 다음과 같은 항목들이 존재합니다.

메소드	설명
csrf()	CSRF 보호를 활성화합니다.
loginPage("/member/login")	‘/member/login’를 로그인 페이지 url로 지정합니다.
defaultSuccessUrl("/")	로그인 성공시 이동할 페이지 url을 지정합니다.
usernameParameter("email")	로그인시 사용할 파라미터는 이메일로 체크하도록 하겠습니다.
failureUrl("/member/login/error")	로그인에 실패 했을 때 이동할 url을 지정합니다.
logoutRequestMatcher(new AntPathRequestMatcher("/member/logout"))	로그 아웃 url을 설정합니다.
logoutSuccessUrl("/")	로그 아웃 성공시 이동할 url을 지정합니다.

≡ 회원 가입 및 로그인

≡ 비밀번호 암호화

Spring Security에서 **비밀 번호를 암호화**하고 검증하는 데 사용되는 인터페이스입니다.

회원 가입을 할 때 입력되는 비밀번호는 가능하다면 암호화 처리를 수행하여 다른 사람들이 알아볼 수 없도록 처리하는 것이 좋습니다.

Spring Security는 암호화를 수행하기 위한 도구들을 지원하고 있습니다.

비밀 번호를 저장할 때 해싱(hashing)하고, 이를 통해 저장된 비밀번호와 입력된 비밀번호의 일치 여부를 검사합니다.

관련 Api

Security 설정과 연관된 클래스/인터페이스는 다음과 같은 항목들이 존재합니다.

클래스/인터페이스	설명
PasswordEncoder	비밀 번호를 암호화해주는 인터페이스입니다.
BcryptPasswordEncoder 클래스	PasswordEncoder의 구현체로써, BCrypt라는 해싱 알고리즘을 사용하여 패스워드를 암호화시켜 줍니다. 강력한 보안성을 제공하며, 기본적으로 10번의 해싱 라운드를 수행합니다. 단방향 해시 알고리즘을 사용합니다.

사용 예시

```
PasswordEncoder passwordEncoder = new BcryptPasswordEncoder();
String encodedPassword = passwordEncoder.encode("password123");
boolean isMatch = passwordEncoder.matches("password123", encodedPassword);
```

≡ Spring Security 설정하기

Spring Security에 대한 설정용 파일을 작성합니다.

파일 이름

com.shopping.config.SecurityConfig

백업 파일

SecurityConfig01.java

구현 절차

1. 해당 클래스를 스프링 설정 파일로 사용하기 위하여 @Configuration 어노테이션을 적용합니다.
2. WebSecurityConfigurerAdapter 클래스를 상속 받는 사용자 정의 클래스를 작성합니다.
3. 해당 클래스에 @EnableWebSecurity 어노테이션을 사용합니다.(for security)
그러면, 스프링이 SpringSecurityFiterChain이 자동으로 개입합니다.
4. 메소드 configure(HttpSecurity http)를 오버라이딩하여 취지에 맞는 보안 설정을 커스트 마이징합니다.
5. PasswordEncoder 인터페이스에 대한 bean 객체를 하나 생성합니다.

다음과 같이 메소드를 구현합니다.

변수/메소드	설명
void configure(HttpSecurity http)	WebSecurity 관련 설정을 재작성하기 위해서 오버라이딩해야 하는 메소드입니다. HttpSecurity는 http 요청에 대한 보안 설정(페이지 권한, 로그인 페이지 설정 등등)을 적용해주는 클래스입니다.
PasswordEncoder passwordEncoder()	비밀 번호를 암호화 시켜 주는 PasswordEncoder bean 객체를 생성해 줍니다.

≡ 회원 가입 및 로그인

≡ 회원 가입 기능 구현하기

회원 가입 페이지 검증 기능을 구현합니다.

validation 의존성 추가

```
implementation 'org.springframework.boot:spring-boot-starter-validation'
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

javax.validation 어노테이션

Java Bean validation을 사용하면 객체의 값들을 효율적으로 검증할 수 있습니다.

다음은 Java Bean 검증을 위하여 사용되는 Annotation 항목들입니다.

어노테이션	설명
@NotEmpty	Collection, Map, String, 또는 배열등에서 사용 가능합니다. null이거나 길이가 0인 문자열, 빈 컬렉션, 빈 배열 등을 허용하지 않습니다.
@NotBlank	String에만 적용이 되며, 빈 문자열이거나, 공백으로만 이루어진 문자열(" " 같은 경우)를 허용하지 않습니다.
@Length(min, max)	최대와 최소 길이를 검사합니다.
@Email	이메일 주소 형식인지를 검사합니다.
@Max(number)	지정한 값보다 값이 적은지 검사합니다.
@Min(number)	지정한 값보다 값이 큰지 검사합니다.
@Null	값이 Null인지 검사합니다.
@NotNull	값이 Null이 아닌지를 검사합니다.

≡ command 객체

command 객체(Command Object)에 대하여 살펴 보고, DTO와 연관 관계를 살펴 봅니다.

커맨드 객체란 HttpServletRequest를 통해 들어온 요청 파라미터들을 setter() 메소드를 사용하여 해당 변수(속성)에 바인딩이 되는 객체를 의미합니다.

일반적으로 Java Bean 클래스가 커맨드 객체의 대상이 됩니다.

HttpServletRequest로 받아오는 요청 파라미터의 이름과 동일한 이름의 변수 및 setter() 메소드를 가지고 있어야 합니다.

개발자가 변수와 setter 메소드만 구현해 놓으면 스프링은 내부적으로 HttpServletRequest와 setter 메소드를 이용하여 스스로 바인딩 시켜줍니다.

View 클래스

```
@Getter @Setter
public class MemberView {
    private String name;

    private String email;

    private String password;

    private String address;
}
```

회원 가입 화면

이름

이메일주소

비밀번호

주소

소스 코드

```
@PostMapping(value = "/insert")
public String doPostInsert(MemberView mv){
    ... 코드 생략
}
```

≡ 회원 가입 기능 구현하기

Errors 인터페이스는 유효성 검증 결과를 저장하고자 할때 사용되는 인터페이스입니다.

BindingResult 인터페이스는 Errors의 인터페이스의 하위 인터페이스입니다.

Form에서 넘어온 요청 파라미터들 값을 command 객체에 복사한 결과를 저장하고 있고, 유효성 검사를 통하여 에러 코드로부터 에러 메시지를 가져 옵니다.

관련 메소드	설명
boolean hasErrors()	에러가 존재하는 경우 true를 리턴합니다.
int getErrorCount()	에러 개수를 리턴합니다.
boolean hasGlobalErrors()	reject() 메소드를 이용해서 추가된 글로벌 에러가 존재할 경우 true를 리턴합니다.
int getGlobalErrorCount()	reject() 메소드를 이용해서 추가된 글로벌 에러 개수를 리턴합니다.
boolean hasFieldErrors()	rejectValue() 메소드를 이용해서 추가된 에러가 존재할 경우 true를 리턴합니다.
int getFieldErrorCount()	rejectValue() 메소드를 이용해서 추가된 에러 개수를 리턴합니다.
boolean hasFieldErrors(String field)	rejectValue() 메소드를 이용해서 추가한 특정 필드의 에러가 존재할 경우 true를 리턴합니다.
int getFieldErrorCount(String field)	rejectValue() 메소드를 이용해서 추가한 특정 필드의 에러 개수를 리턴합니다.
List<FieldError> getFieldErrors()	필드와 관련된 에러 내용들을 List 컬렉션 형태로 리턴합니다.

≡ `@Transactional

데이터베이스 트랜잭션 관리를 쉽게 할 수 있도록 지원하는 어노테이션입니다.

트랜잭션은 하나 이상의 작업을 하나의 작업 단위로 묶어, 모두 성공적으로 실행되거나 모두 롤백되어야 한다는 원칙을 따릅니다.

@Transactional을 사용하면 이 작업 단위를 코드에서 간단하게 관리할 수 있습니다.

관련 패키지) `import org.springframework.transaction.annotation.Transactional;`

요구 사항

항목	설명
트랜잭션 시작 및 종료	메소드 실행 시 트랜잭션이 시작되고, 메소드가 정상적으로 끝나면 트랜잭션이 커밋됩니다. 예외가 발생하면 트랜잭션이 롤백됩니다.
롤백 제어	예외 발생 시 자동으로 롤백하지만, 특정 예외에만 롤백하거나, 반대로 롤백하지 않도록 설정할 수 있습니다.
읽기 전용 트랜잭션	<code>`readOnly = true`</code> 로 설정하면 데이터베이스에 변경이 일어나지 않도록 보장해 줍니다. 읽기 전용 작업에서 성능을 최적화할 수 있습니다.

주요 속성

항목	설명
propagation(전파)	현재 실행 중인 트랜잭션이 있을 때, 새 트랜잭션을 생성할지 또는 기존 트랜잭션을 사용할지 설정합니다.
isolation(격리 수준)	동시에 여러 트랜잭션이 실행될 때 발생할 수 있는 문제를 방지하기 위한 설정입니다.
timeout	트랜잭션이 완료될 때까지 대기할 시간을 설정합니다. 설정된 시간 내에 완료되지 않으면 트랜잭션이 롤백됩니다.
readOnly	<code>`true`</code> 로 설정 시 트랜잭션을 읽기 전용으로 설정하여 성능 최적화가 가능합니다.
rollbackFor/noRollbackFor	특정 예외에 대해서만 롤백하거나, 특정 예외에서는 롤백하지 않도록 설정할 수 있습니다.

≡ 로그인/로그 아웃 구현하기

`UserDetailsService`는 Spring Security에서 **사용자의 인증을 처리**하기 위해 제공되는 핵심 인터페이스 중 하나입니다.

사용자 정보를 불러오는 기능을 제공하며, 이를 통해 사용자 인증과 권한 부여를 할 수 있습니다.

일반적으로 데이터베이스나 외부 시스템에서 사용자 정보를 가져오는 데 사용됩니다.

주요 특징

항목	설명
단일 메소드	`loadUserByUsername(String username)`이라는 단일 메소드가 있습니다. 이 메소드는 사용자의 이름(혹은 이메일 등 다른 식별자)을 매개 변수로 받아 해당 사용자에 대한 권한 정보등을 반환합니다.
UserDetails와 연동	보통 커스텀 구현을 통해 데이터베이스에서 사용자 정보를 조회하고, 그 정보를 `UserDetails` 객체로 변환해 줍니다. Spring Security의 인증 프로세스에서 사용할 수 있으며, 비밀번호와 권한 정보도 함께 제공됩니다.
커스터마이징	`UserDetailsService`는 기본적으로 사용자 정보 로드 로직을 개발자가 자유롭게 구현할 수 있어, 다양한 인증 방식(DB, LDAP, 외부 API 등)을 지원하는 데 매우 유연합니다.

≡ 회원 가입 및 로그인

≡ 로그인/로그 아웃 구현하기

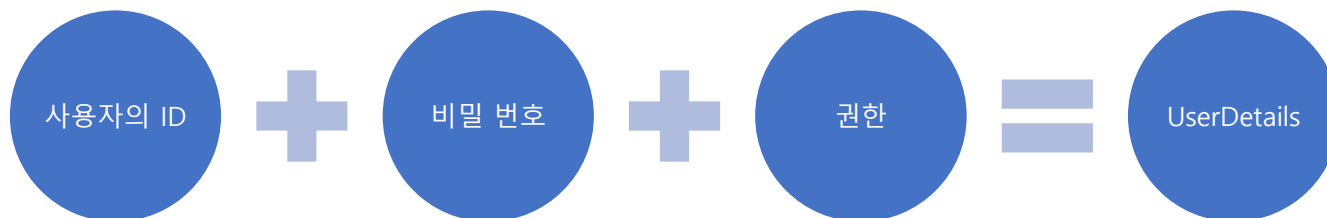
`UserDetails`는 Spring Security에서 **사용자의 인증 및 권한 관리**를 위해 정의된 인터페이스입니다.

인증된 사용자의 정보를 표현하며, 사용자 계정의 기본 속성들을 정의하는 메소드를 제공합니다.

주로 사용자 **로그인**, **인증 절차에서 자주** 사용됩니다.

UserDetailsService 인터페이스와 함께 사용되어 데이터베이스나 다른 데이터 소스에서 사용자의 세부 정보를 로드합니다.

이 인터페이스를 구현해 놓은 구현체 클래스는 User 클래스입니다.



주요 메소드

항목	설명
getAuthorities()	사용자가 가진 권한(역할)을 반환합니다. 이 권한은 보통 GrantedAuthority 객체로 표현됩니다. 예: ROLE_USER, ROLE_ADMIN 등의 역할.
getPassword()	사용자의 암호화된 비밀번호를 반환합니다.
getUsername()	사용자의 고유 아이디(주로 사용자 이름이나 이메일)를 반환합니다.
isAccountNonExpired()	사용자의 계정이 만료되지 않았는지 여부를 반환합니다. true일 경우 계정이 만료되지 않은 상태입니다.
isAccountNonLocked()	사용자의 계정이 잠겨 있지 않은지 여부를 반환합니다. true일 경우 계정이 잠겨 있지 않습니다.
isCredentialsNonExpired()	사용자의 인증 정보(비밀 번호 등)가 만료되지 않았는지 여부를 반환합니다. true일 경우 인증 정보가 만료되지 않았습니다.
isEnabled()	사용자의 계정이 활성화되어 있는지 여부를 반환합니다. true일 경우 계정이 활성화된 상태입니다.

≡ 회원 가입 및 로그인

≡ 로그인/로그 아웃 구현하기

`User` 클래스는 사용자 이름, 비밀번호, 권한 등의 정보를 쉽게 설정하고 인증 작업을 수행할 수 있습니다.

UserDetails를 구현한 대표적인 클래스로, Spring Security에서 제공하는 기본적인 사용자 정보를 쉽게 사용할 수 있도록 만들어져 있습니다.

기본 사용자 정보 이외에 추가적인 정보(예: 이메일, 전화번호 등)를 제공해야 할 경우에는 이 클래스를 상속하거나, 직접 UserDetails 인터페이스를 구현하는 방식으로 확장할 수 있습니다.

authorities 필드는 사용자의 권한을 나타내며, 역할 기반 접근 제어(Role-based Access Control, RBAC)를 구현할 때 자주 사용됩니다.

생성자

항목	설명
생성자	User(String username, String password, Collection<? extends GrantedAuthority> authorities)
생성자	User(String username, String password, boolean enabled, boolean accountNonExpired, boolean credentialsNonExpired, boolean accountNonLocked, Collection<? extends GrantedAuthority> authorities)

주요 메소드

항목	설명
getUsername()	사용자의 이름을 반환합니다.
getPassword()	사용자의 암호를 반환합니다.
getAuthorities()	사용자의 권한 목록을 반환합니다. 이 권한은 GrantedAuthority 객체로 표현됩니다.
isAccountNonExpired()	계정이 만료되지 않았는지 여부를 반환합니다.
isAccountNonLocked()	계정이 잠기지 않았는지 여부를 반환합니다.
isCredentialsNonExpired()	자격 증명이 만료되지 않았는지 여부를 반환합니다.
isEnabled()	계정이 활성화되어 있는지 여부를 반환합니다.

≡ 회원 가입 및 로그인

≡ 로그인/로그 아웃 연동하기

스프링 시큐리티를 사용하여 header 페이지의 풀다운 메뉴를 설정해 봅니다.

파일 이름 pom.xml

thymeleaf-extras-springsecurity5 라이브러리는 Spring Security에서 제공하는 인증, 권한, 로그인 여부 확인 등의 기능을 Thymeleaf 템플릿에서 간편하게 활용할 수 있습니다. 예를 들면 로그인과 로그 아웃시 적절한 메뉴를 설정할 수 있습니다.

pom.xml 파일 또는 build.gradle 파일에 다음과 같이 추가합니다.

스프링 부트 2.x 버전에서는 thymeleaf-extras-springsecurity5는 스프링 시큐리티 5와 연동되도록 설계되었습니다.

스프링 부트 3.x에서는 스프링 시큐리티 6 버전이 사용되므로, `thymeleaf-extras-springsecurity6`으로 작성해야 합니다.

```
<dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
```

```
dependencies {
    implementation 'org.thymeleaf.extras:thymeleaf-extras-springsecurity6'
    implementation 'org.thymeleaf:thymeleaf-spring6:3.1.0.RELEASE'
}
```

≡ 로그인/로그 아웃 연동하기

스프링 시큐리티를 사용하여 header 페이지의 풀다운 메뉴를 다음과 같이 분기 처리합니다.

관련 Api

이 함수들은 `sec` 네임스페이스를 통해 접근할 수 있으며, 로그인 상태, 권한, 인증 등의 조건을 쉽게 확인할 수 있습니다.

이 함수들을 활용하면 로그인 상태나 사용자 권한에 따라 다양한 HTML 요소를 제어할 수 있습니다.

항목	설명
isAuthenticated()	현재 사용자가 인증(로그인)이 되었으면 true 값을 반환해줍니다.
isAnonymous()	현재 사용자가 익명(로그인하지 않은 상태)인 경우 true를 반환해줍니다.
hasAnyAuthority('권한01', '권한02', ...)	열거되어 있는 여러 권한 중 하나라도 충족이 되면 true 값을 반환해줍니다.
hasRole('ROLE_권한')	특정 권한이 있는지 확인합니다. `ROLE_` 접두사는 권한 이름에 추가되어야 합니다.
hasAnyRole('ROLE_권한01', 'ROLE_권한02', ...)	여러 권한 중 하나라도 있으면 조건을 만족합니다.
hasAuthority('권한')	특정 권한을 가진 경우(접두사 없이)를 확인합니다.
principal	현재 인증된 사용자 객체에 접근할 수 있습니다. 주로 사용자 이름이나 이메일을 표시할 때 사용됩니다.
isRememberMe()	현재 인증이 '기억하기' 기능에 의한 것인지 확인합니다.
isFullyAuthenticated()	완전한 인증이 이루어졌는지 확인합니다. (기억하기 기능이 아닌 경우)
denyAll	모든 접근을 차단합니다.
permitAll	모든 사용자가 접근할 수 있도록 허용합니다.

≡ 페이지 권한 설정하기

Spring Security 설정용 파일에 인가와 관련된 코드를 작성합니다.

관련 Api

메소드	설명
authorizeRequests()	Security 처리시 HttpServletRequest를 사용합니다.
mvcMatchers()	지정된 URL 패턴과 일치하는 요청(특정 경로들)에 대해 접근 제어 권한을 설정하는 메소드입니다.
permitAll()	모든 사용자가 인증(로그인)없이 해당 경로를 접근할 수 있도록 하겠습니다. 예를 들어 홈페이지, 회원, 제품, 이미지 경로, Thymeleaf 템플릿 경로 등의 콘텐츠를 누구나 볼 수 있게 설정합니다.
hasRole("ADMIN")	어드민으로 시작하는 경로는 해당 계정이 'ADMIN'일 경우에만 접근 가능하도록 하겠습니다. 이러한 기능은 관리자만 접근해야 하는 기능이나 데이터를 보호하고자 하는 곳에 유용하게 사용됩니다.
anyRequest().authenticated()	위에 열거 되지 않은 URL 요청에 대해서는 로그인 및 인증이 요구됩니다.

Spring Boot

연관 관계 매핑



Spring



Boot



Security

연관 관계 매핑

연관 관계 매핑

엔터티 간의 관계를 정의하고, 이 관계를 데이터베이스의 테이블과 연결하는 과정을 의미합니다.

연관 관계 매핑은 객체 지향 프로그래밍의 개념을 데이터베이스 설계에 잘 반영할 수 있게 도와 줍니다.

JPA를 활용하면 관계형 데이터베이스의 복잡한 관계를 간단하게 표현하고 관리할 수 있습니다.

다양한 매핑 옵션을 이해하고 적절히 활용하면, 데이터베이스 설계를 더욱 효율적으로 수행할 수 있습니다.

요구 사항

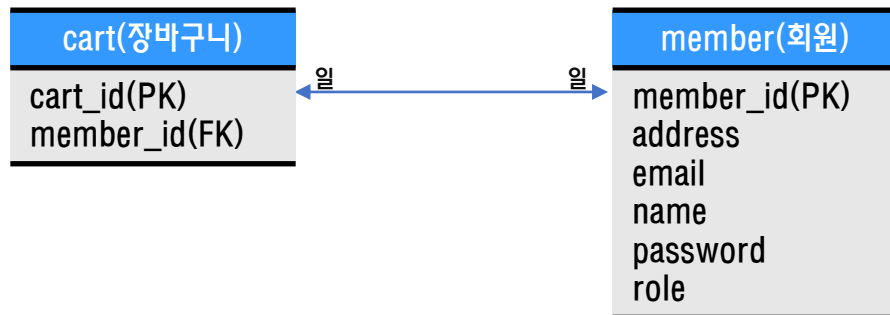
연관 관계 매핑은 Entity 간의 연관 관계를 서술하는 기법을 말합니다.

고려 해야 하는 사항은 (1)매핑의 종류와 (2)방향성입니다.

연관 관계 예시

모든 회원들은 하나의 장바구니를 가지고 있습니다.

이러한 매핑을 '일대일' 매핑이라고 합니다.



연관 관계 매핑

연관 관계 매핑의 종류와 방향성

연관 관계 매핑의 종류와 방향성은 다음과 같습니다.

연관 관계 매핑의 종류

연관 관계 매핑은 4가지 유형이 존재합니다.

이러한 연관 관계 매핑을 database에서는 '참조 무결성 제약 조건'이라고 합니다.

연관 관계에서 fetch 옵션의 기본 값은 **즉시 로딩**입니다.

매핑의 종류	어노테이션	매핑 예시	설명
일대일	@OneToOne	회원(일) ↔ 장바구니(일) User ↔ Profile	회원은 하나의 장바구니를 소유할 수 있습니다. 사용자는 하나의 프로필을 가질 수 있습니다.
일대다	@OneToMany	장바구니(일) ↔ 상품(다) 회원(일) ↔ 주문(다)	1개의 장바구니에는 여러 개의 상품을 담을 수 있습니다. 1명의 회원은 여러 개의 주문이 가능합니다.
다대일	@ManyToOne	주문(다) ↔ 주문상품(일)	1개의 주문에는 여러 개의 상품을 담을 수 있습니다.
다대다	@ManyToMany	회원(다) ↔ 상품(다) Student ↔ Course	여러 명의 회원들이 여러 개의 상품을 구매할 수 있습니다. 중간에 새로운 Entity를 추가하여 보완합니다.

연관 관계 매핑의 방향성

데이터 베이스에서 테이블간의 관계는 항상 양방향입니다

JPA의 Entity에서는 단방향과 양방향이 존재합니다.

연관 관계 매핑

연관 관계 매핑

다대일/일대다 양방향 매핑에 대하여 살펴 보도록 하겠습니다.

파일 이름 com.shopping.entity.Order

주문(Order) Entity와 주문 상품(OrderProduct) Entity를 외래 키(orderId)로 **양방향 매핑**을 하게 되면 다음과 같은 항목들을 확인할 수 있습니다.

1. 주문에 속한 상품에 어떠한 것들이 존재하는 지 파악할 수 있습니다.
2. 주문 상품은 어떠한 주문에 속하는 지를 파악할 수 있습니다.

양방향 매핑시 유의 사항

연관 관계의 주인은 외래키가 있는 곳으로 설정합니다.

연관 관계의 주인이 외래 키를 관리(등록, 수정, 삭제)

주인이 아닌 쪽은 연관 관계 매핑시 mappedBy 속성의 값으로 연관 관계의 주인을 설정해 줍니다.

주인이 아닌 쪽은 읽기만 가능합니다.

... 기존 코드 생략

```
public class Order{
```

... 기존 코드 생략

```
    // "order"는 多 쪽에 있는 해당 변수의 이름
```

```
    @OneToMany(mappedBy = "order") // 하나의 주문은 여러 개의 주문 상품을 가질 수 있습니다.
```

```
    private List<OrderProduct> orderProducts = new ArrayList<>();
```

```
}
```

연관 관계 매핑

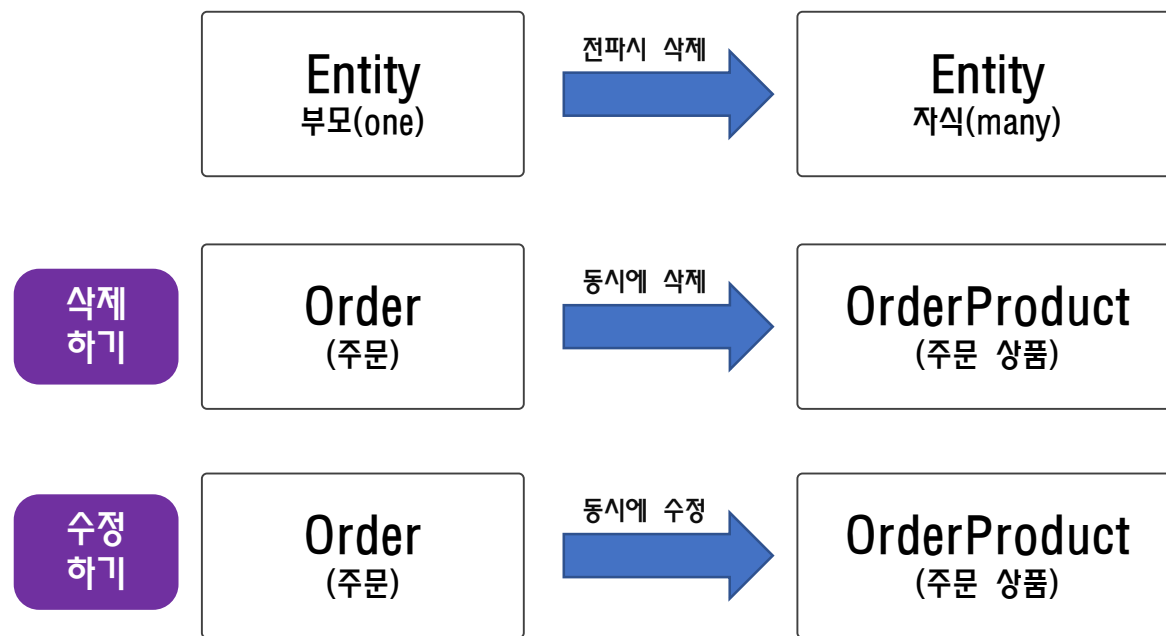
영속성 전이

상태 변경이 이루어진 Entity와 연관 관계를 맺고 있는 Entity에 상태 변화를 전파 시키는 것을 말합니다.

Entity의 상태가 변경이 되었을 때 연관된 Entity의 상태 변화를 전파 시키는 것을 말합니다.

이 기능은 데이터베이스의 관계형 데이터베이스에서 엔티티 간의 관계를 유지하는 데 유용합니다.

데이터 베이스의 `on delete cascade` 옵션과 유사한 개념입니다.



영속성 전이

영속성 전이와 관련된 옵션은 다음과 같은 항목들이 존재합니다.

CASCADE는 Entity의 상태 정보가 변경이 되는 경우, 연관이 되어 있는 다른 엔티티에 어떤 작업을 전파해줄지 설정하는 옵션입니다. 예를 들어, `CascadeType.ALL`은 모든 작업(CREATE, READ, UPDATE, DELETE)을 전파합니다.

옵션

CASCADE와 관련된 옵션은 다음과 같은 항목들이 존재합니다.

CASCADE 종류	설명
ALL	부모 Entity의 영속성 상태 변화를 자식 Entity에게 모두 전이 시킵니다.
PERSIST	부모 Entity가 영속화 될 때까지 자식 Entity도 영속화됩니다.
REMOVE	부모 Entity가 삭제될 때 자식 Entity도 삭제됩니다.
MERGE	부모 Entity가 병합이 될 때 자식 Entity도 병합됩니다.
REFRESH	부모 Entity가 refresh되면 자식 Entity도 refresh됩니다.
DETACH	부모 Entity가 detach되면 자식 Entity도 detach 상태로 변경이 됩니다.

연관 관계 매핑

고아 객체 제거하기

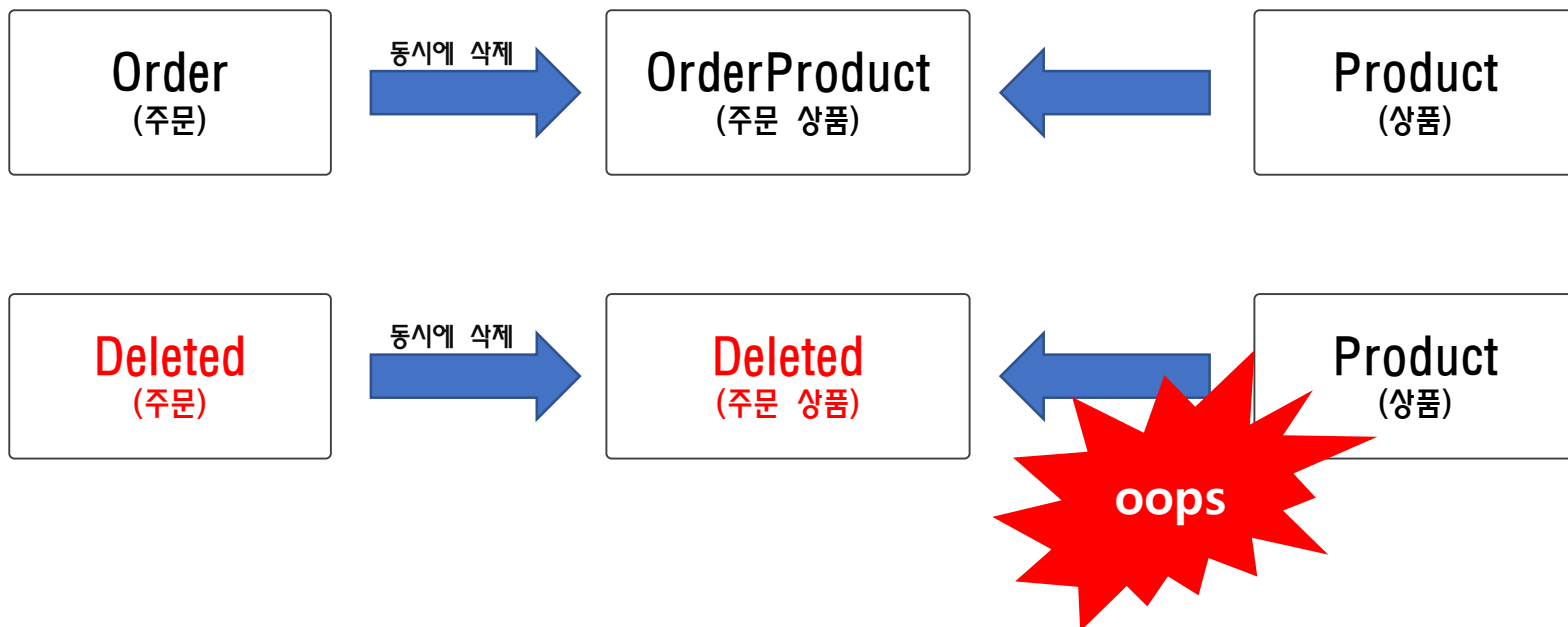
고아(Orphan) 객체는 부모 Entity와 연관 관계가 끊어진 자식 Entity를 의미합니다.

고아 객체를 영속성 전이 기능과 동시에 사용하게 되면 부모의 Entity를 이용하여 자식의 Entity의 생명 주기(life cycle)를 관리할 수 있습니다.

고아 객체 기능은 @OneToOne, @OneToMany 어노테이션에서 옵션으로 사용할 수 있습니다.

고아 객체 기능은 참조하고 있는 여러 곳이면 문제가 발생할 수 있습니다.

예를 들어서, 다음 그림과 같이 OrderProduct Entity가 Order Entity와 Product Entity에서 동시에 참조가 되고 있는 경우 이 기능을 사용하면 안됩니다.



지연 로딩

연관 관계의 Entity에서 지연 로딩과 즉시 로딩을 구분해야 하는 이유를 살펴 봅니다.

JPA에서 엔티티를 데이터베이스에서 가져오는 시점은 크게 두 가지 방식이 있습니다.

로딩 방법	설명
즉시 로딩	특정 Entity를 조회할 때, 연관 관계가 있는 Entity까지 한꺼번에 조회하는 로딩을 말합니다. 관련 데이터를 미리 가져오므로 추가적인 쿼리가 발생하지 않지만, 사용하지 않는 데이터를 가져와 메모리를 낭비할 수 있습니다.
지연 로딩	특정 Entity를 조회할 때, 연관 관계가 있는 Entity를 직접 조회하지 않고, Proxy로 조회합니다. 메모리를 절약하고 성능을 최적화할 수 있습니다. 실제 조회가 이루어질때, 초기화 되면서 데이터를 조회합니다.

≡ 엔터티 공통 속성 공통화

≡ Auditing을 이용한 엔터티 공통 속성 공통화

Auditing은 감사를 위한 기능입니다.

Auditing

일반적으로 Entity가 변경이 되는 경우 자동으로 등록 날짜, 등록한 사람, 수정한 사람 등의 정보가 기록됩니다.

감사(Auditing)는 이러한 Entity의 변경 내용을 감시(audit)하여 자동으로 기록해주는 기능을 의미합니다.

즉, Entity의 생성과 수정을 감시하는 기능입니다.

Spring Data Jpa가 제공하는 엔터티의 변화를 추적하는 Audit 기능을 사용하면 구현이 가능합니다.

사용처

사용자들의 작업 히스토리를 기록합니다.

새로운 변경 작업을 수행하기 위하여 이전 변경 대상을 검색할 때 사용합니다.

≡ `AuditorAware` 인터페이스

JPA에서 엔터티의 생성자 및 수정자를 자동으로 설정하는 데 사용하는 인터페이스입니다.

Entity가 생성되거나 업데이트될 때 해당 작업을 수행한 사용자 정보를 자동으로 주입할 수 있어, 보안 및 감사(Auditing) 목적에 많이 활용됩니다.
`@EnableJpaAuditing` 애너테이션과 함께 사용하면 자동 감사 기능이 활성화됩니다.

AuditorAware의 역할

엔터티가 생성되거나 수정될 때 해당 작업을 수행한 **사용자 정보를 추적**하여 데이터베이스에 기록.

이 정보는 주로 **감사 로그**를 남기거나 보안 요건을 충족하기 위해 필요함.

Spring Security의 인증 정보를 활용하여 현재 **로그인된 사용자 정보**를 가져오는 데 사용.

항목	설명
Optional<T> getCurrentAuditor()	`Optional`로 감싼 현재 감사자(Auditor) 정보를 반환합니다. 예를 들어, Spring Security를 사용하고 있는 경우 인증된 사용자 정보를 가져와 반환할 수 있습니다. T는 반환 타입을 의미하며, 일반적으로 사용자 ID 또는 사용자 엔티티를 반환하는 형태입니다. 참고 사항) Optional 클래스는 "import java.util.Optional;" 패키지입니다.

사용시 장점

항목	설명
자동화된 감사	엔터티가 생성되거나 수정될 때 자동으로 감사를 남길 수 있습니다.
보안 강화	데이터를 수정한 사용자를 추적할 수 있어 데이터 무결성 및 보안이 강화됩니다.
유연성	다양한 방식으로 사용자 정보를 가져올 수 있으므로 다양한 보안 설정에 적용 가능합니다.

≡ `Authentication 인터페이스

보안과 관련 정보를 캡슐화하고 인증된 사용자에 대한 정보를 포함하는 핵심 구성 요소입니다.

현재 인증된 사용자에 대한 정보를 확인하거나 인증 객체를 사용해 권한 관리를 할 때 사용됩니다.

인증 및 권한 정보를 포함하여 애플리케이션에서 사용자의 인증 상태를 관리하는 데 필수적인 역할을 합니다.

유연한 인증 및 권한 부여 기능을 제공하며, 다양한 보안 요구 사항을 충족시킬 수 있습니다.

AuthenticationManager에 의하여 제공됩니다.

연관 사이트

<https://docs.spring.io/spring-data/jpa/docs/1.7.0.DATAJPA-580-SNAPSHOT/reference/html/index.html>

페이지에서 검색 키워드 : AuditorAware 키워드로 검색 요망

주요 메소드

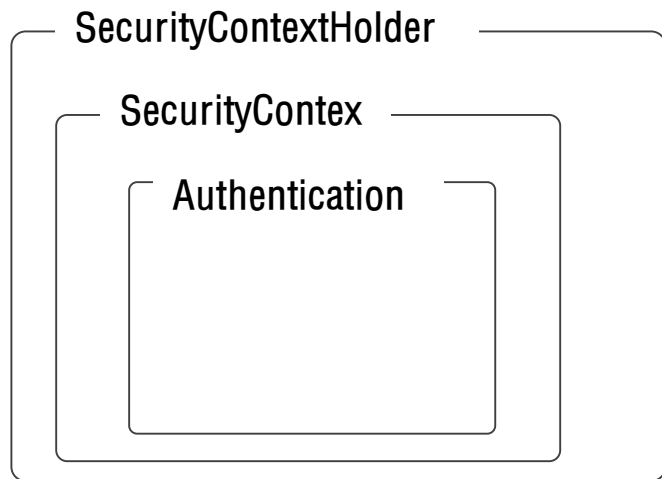
항목	설명
getPrincipal()	Authentication 객체가 보유한 주체(Principal)를 반환합니다. 주로 사용자의 고유 식별 정보를 나타내며, 보통 사용자 이름 또는 `UserDetails` 객체를 반환합니다.
getCredentials()	사용자의 인증 자격 증명을 반환하며, 일반적으로 비밀번호 등의 정보를 나타냅니다. Authentication 객체가 생성된 후, 보안상 이유로 이 자격 증명은 제거되거나 마스킹될 수 있습니다.
getAuthorities()	GrantedAuthority 객체의 목록을 반환하며, 이는 인증된 사용자가 가지고 있는 권한 목록을 나타냅니다. 각 권한은 일반적으로 역할(role) 또는 액세스 권한을 나타냅니다.(예: ROLE_USER, ROLE_ADMIN 등)
isAuthenticated()	현재 Authentication 객체가 인증되었는지를 나타내는 Boolean 값을 반환합니다. 인증되지 않은 경우 false가 되며, 인증된 경우 true가 됩니다.
getDetails()	인증 과정 중에 생성된 추가적인 정보를 제공합니다. 일반적으로 사용자의 IP 주소나 세션 정보가 여기에 저장됩니다. 예를 들어, 세션에 관련된 정보나 사용자의 브라우저 정보를 포함할 수 있습니다.
setAuthenticated(boolean isAuthenticated)	Authentication 객체의 인증 상태를 설정하는 메소드입니다. 이 메소드는 인증을 수정할 수 있는 권한을 가진 클래스에서만 호출해야 하며, 직접 사용하는 경우는 드뭅니다.

≡ `SecurityContextHolder

Spring Security에서 인증 정보를 관리하고 정보들을 제공해주는 핵심적인 클래스입니다.

인증, 권한 관리, 세션 유지 등의 작업에서 핵심적인 역할을 수행합니다.

사용자 요청이 들어왔을 때 현재 사용자의 인증 정보를 유지하여 해당 사용자의 권한을 기반으로 자원에 접근할 수 있도록 합니다.



주요 메소드

항목	설명
getContext()	현재 스레드의 `SecurityContext`를 반환합니다. 만약 `SecurityContext`가 존재하지 않으면 새로운 컨텍스트가 자동으로 생성됩니다.
setContext(SecurityContext context)	현재 스레드의 `SecurityContext`를 지정된 컨텍스트로 설정합니다. 이를 통해 인증 정보를 명시적으로 변경할 수 있습니다.
clearContext()	현재 스레드의 `SecurityContext`를 삭제하여 인증 정보를 비웁니다. 일반적으로 로그아웃 시 사용됩니다.



≡ `@EnableJpaAuditing

JPA의 Auditing 기능을 활성화하기 위한 어노테이션입니다.

해당 어노테이션을 클래스 영역에 명시하면 JPA의 Auditing 기능이 활성화됩니다.

엔터티의 필드에 특정한 어노테이션(`@CreateDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`)을 적용하면 JPA가 자동으로 값을 채워 줍니다. 주로 엔터티(Entity)의 생성 및 수정 시간을 자동으로 기록하는 데 사용되며, 이 기능을 통해 데이터베이스 테이블에 시간 관련 컬럼을 쉽게 관리할 수 있습니다.

연관된 Annotation

Java 8 이상의 날짜 API 사용이 되며, LocalDateTime을 사용하는 것이 일반적입니다.

또한 데이터베이스에서도 지원되므로 시간 관리에 적합합니다.

항목	설명
`@CreatedBy`	엔터티 생성 시 생성한 사람(생성자)를 자동으로 저장해주는 어노테이션입니다.
`@LastModifiedBy`	엔터티의 값을 마지막으로 변경 한 사용자의 정보를 자동으로 저장해주는 어노테이션입니다.
`@CreateDate`	엔터티가 최초로 생성될 때의 시간을 자동으로 저장합니다. 데이터 생성 시점을 추적할 수 있습니다.
`@LastModifiedDate`	엔터티가 마지막으로 수정될 때의 시간을 자동으로 저장합니다. 마지막 수정 시점을 추적하는 데 유용합니다.

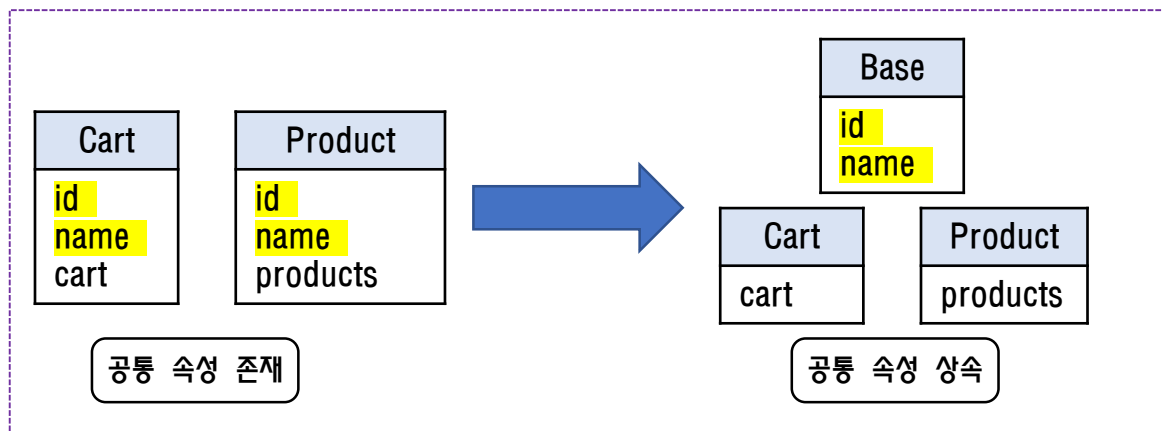
≡ 엔터티 공통 속성 공통화

≡ 날짜 관련 속성 공통화

등록 날짜와 수정 날짜만 구현해 놓은 Entity 클래스를 구현해 보도록 하겠습니다.

항목	설명
@EntityListeners	Entity 리스너를 생성하고자 할 때 사용하는 어노테이션입니다. Entity의 생명 주기(life cycle)에 관여하여 변경 사항을 통지(notification) 받아서 어떤 일(메소드)을 수행합니다. 따라서 엔티티 상태 변화와 관련된 공통 로직을 중앙에서 관리하고 캡슐화하는 데 유용합니다.
@MappedSuperclass	객체의 입장에서 공통적으로 사용할 매핑 정보가 필요할 때 사용하는 어노테이션입니다. 부모 클래스에 정의를 해 놓으면 상속 받는 자식 클래스에 대한 매핑 정보를 제공해 줍니다.
AuditingEntityListener	Entity 영속성 및 업데이트에 대한 Auditing 정보를 캡처해주는 JPA Entity Listener 클래스입니다. 엔티티의 생성 및 수정 시점을 자동으로 기록하기 위해 사용하는 리스너 클래스입니다.

@MappedSuperclass 사용 예시



Spring Boot

상품 등록/수정 및 조회하기



Spring



Boot



Security

☰ 상품 등록/수정 및 조회하기

☰ 상품 등록 화면으로 이동하기

pom.xml 파일에 의존성을 추가합니다.

JDBC 프로그래밍에서 rs(ResultSet) 객체의 getter 메소드를 이용하여 Bean 객체에 setting 해본적이 있을 겁니다.

변수의 개수가 많아지면, 많은 getter와 setter를 구현해야 하는 불편함이 있는 데, 이러한 것을 편리하게 사용할 수 있도록 해주는 라이브러리가 modelmapper입니다.

modelmapper 라이브러리는 스프링 부트에서 상품 등록시 화면에서 전달 받은 DTO 객체를 Entity로 변환을 수행하거나, 상품을 조회하고자 할 때 Entity를 DTO 객체 형태로 변환 작업을 수행하고자 할때 사용할 수 있는 라이브러리입니다.

참조 사이트) <http://modelmapper.org/getting-started/>

항목	자료형	설명
상품 등록	DTO → Entity	화면에서 넘겨진 view(dto 객체)를 JPA에서 사용하기 위하여 Entity로 변경해야 합니다.
상품 조회	Entity → DTO	JPA에서 조회된 Entity를 화면에 보여 주려면 다시 view(dto 객체)로 변환해 주어야 합니다.

modelmapper 관련 의존성을 다음과 같이 추가합니다.

서로 다른 클래스의 필드 이름과 타입이 동일하면 setter, getter를 사용하여 값을 복사하여 데이터를 변경해 줍니다.

```
<dependency>
  <groupId>org.modelmapper</groupId>
  <artifactId>modelmapper</artifactId>
  <version>2.3.9</version>
</dependency>
```

상품 등록/수정 및 조회하기

상품 등록 화면으로 이동하기

상품 등록 화면을 작성하기 전에 thymeleaf와 관련된 태그를 살펴 보도록 합니다.

다음은 상품 등록 화면에 사용될 thymeleaf 관련 태그 목록입니다.

Thymeleaf에 사용될 문법

항목	설명
th:formaction	폼 데이터들이 서버로 제출(submit)될 때 URL을 명시하는 역할로, <form> 태그의 'action' 속성과 동일한 기능을 합니다. @{} 기능을 사용하여 링크 페이지를 만들어 줍니다. <button th:formaction="@{/admin/product/insert}" type="submit" />

☰ 상품 등록/수정 및 조회하기

☰ 상품 등록 화면으로 이동하기

상품 등록 화면을 작성하기 전에 thymeleaf와 관련된 태그를 살펴 보도록 합니다.

Thymeleaf에 사용될 문법

다음은 thymeleaf에서 사용할 수 있는 유틸리티 객체 목록입니다.

공식 문서 페이지는 <https://www.thymeleaf.org/doc/tutorials/3.0/usingthymeleaf.html#appendix-b-expression-utility-objects>입니다.

- #message : 메시지, 국제화 처리
- #uris : URI 이스케이프 지원
- #dates : java.util.Date 서식 지원
- #calendars : java.util.Calendar 서식 지원
- #temporals : 자바8 날짜 서식 지원(날짜는 주로 temporals를 많이 사용)
- #numbers : 숫자 서식 지원
- #strings : 문자 관련 편의 기능
- #objects : 객체 관련 기능
- #booleans : boolean 관련 기능
- #arrays : 배열 관련 기능
- #lists : 컬렉션 관련 기능
- #sets : 컬렉션 관련 기능
- #maps : 컬렉션 관련 기능
- #ids : 아이디 처리 관련 기능

☰ 상품 등록/수정 및 조회하기

☰ 상품 등록 화면으로 이동하기

상품 등록 화면을 작성하기 전에 thymeleaf와 관련된 태그를 살펴 보도록 합니다.

Thymeleaf 문법 예시

항목	설명
<code><div th:each="num: \${#numbers.sequence(1,5)}"></code>	연속된 숫자 5개를 생성합니다.
<code> 원</code>	숫자 형식에 대한 콤마 유형을 지정합니다.
<code>\${#strings.isEmpty(name)}</code>	문자열이 비어 있거나 null인지 확인합니다.
<code>\${#lists.isEmpty(list)}</code>	리스트 컬렉션이 비어 있는 지 확인합니다.
<code>\${#dates.format(regDate, 'yyyy-MM-dd')}</code>	날짜 서식에 대한 포매팅입니다.
<code>\${#lists.toList(object)}</code>	object를 List 컬렉션으로 변환합니다.
<code>\${#lists.size(list)}</code>	List 컬렉션의 요소 개수를 구합니다.
<code>\${#lists.sort(list)}</code>	List 컬렉션의 요소들을 정렬합니다.
<code>\${#arrays.toArray(object)}</code>	object를 배열 타입으로 변경합니다.
<code>\${#arrays.toArrayIntegerArray(object)}</code>	숫자 형식이 들어 있는 object를 정수형 배열 타입으로 변경합니다.
<code>\${#arrays.length(array)}</code>	array 배열의 요소 개수를 구해줍니다.
<code>\${#arrays.isEmpty(array)}</code>	배열 요소가 비어 있는 지 검사합니다.
<code>\${#arrays.contains(array, element)}</code>	배열 array에 요소 element가 존재하는 지 검사합니다.

☰ `@RequestParam` 어노테이션

클라이언트의 HTTP 요청 파라미터를 메소드 파라미터에 바인딩할 때 사용하는 어노테이션입니다.

일반적으로 `GET` 요청의 쿼리 파라미터나 `POST` 요청의 폼 데이터를 받아오는 데 사용됩니다.

이를 활용하면 요청 파라미터의 값을 편리하게 Java 변수에 저장하고 사용할 수 있습니다.

항목	설명
required	요청 파라미터가 필수(required=true)로 지정되어 있습니다. 해당 파라미터가 없으면 예외(MissingServletRequestParameterException)가 발생합니다.
defaultValue	파라미터가 없을 경우 기본값을 지정할 수 있습니다. defaultValue가 설정되면 required 속성은 자동으로 false로 처리됩니다.
다중 파라미터 처리	List, Set 등의 자료구조 타입을 사용하여 여러 개의 같은 이름을 가진 파라미터 값을 받을 수 있습니다. <pre>@GetMapping("/numbers") public String numbers(@RequestParam List<Integer> values) { return "Received numbers: " + values; }</pre> 요청 URL: <code>http://localhost:8080/numbers?values=1&values=2&values=3</code>

☰ 상품 등록/수정 및 조회하기

☰ `@Value` 어노테이션

애플리케이션의 프로퍼티 값을 주입하고자 할 때 사용하는 어노테이션입니다.

기본 사용법

@Value의 관련 패키지는 `import org.springframework.beans.factory.annotation.Value;`입니다.

@Component

```
public class MyComponent {  
    @Value("${my.property}")  
    private String myProperty;  
}
```

위 예제에서 `application.properties` 파일에 다음과 같이 `my.property`를 정의하면
my.property=Hello, World!

`myProperty` 필드는 `Hello, World!` 값으로 초기화됩니다.

☰ `WebMvcConfigurer` 인터페이스

Spring MVC에서 웹 애플리케이션의 설정을 사용자 정의하는 데 사용하는 인터페이스입니다.

Spring Boot는 일반적으로 자동 설정을 제공하지만, `WebMvcConfigurer`를 사용하면 Spring MVC의 설정을 사용자 정의할 수 있어 보다 복잡한 웹 애플리케이션을 만들 때 유용합니다.

사용 용도

인터셉터

CORS 설정

리소스 핸들링

메시지 컨버터

주요 메소드

항목	설명
<code>addInterceptors(InterceptorRegistry registry)</code>	특정 요청에 대하여 인터셉터를 적용할 수 있게 해줍니다. 사용 사례) 로그인 체크, 로깅, API 호출 전후 작업 등을 설정할 수 있습니다.
<code>addCorsMappings(CorsRegistry registry)</code>	CORS(Cross-Origin Resource Sharing) 정책을 설정합니다. 사용 사례) 다른 도메인에서 접근해야 하는 API에 대해 허용할 URL 패턴 및 HTTP 메소드, 헤더 등을 지정할 수 있습니다.
<code>addResourceHandlers(ResourceHandlerRegistry registry)</code>	정적 리소스(이미지, CSS, JS 파일 등)의 경로를 매핑합니다. 사용 사례) 외부 리소스 폴더를 매핑하거나 캐싱 전략을 설정할 때 유용합니다.

☰ `WebMvcConfigurer 인터페이스

Spring MVC에서 웹 애플리케이션의 설정을 사용자 정의하는 데 사용하는 인터페이스입니다.

사용 예시

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/*") // 모든 요청에 CORS 적용
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "DELETE");
    }

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        // /images/** 요청을 받으면 C:\boot\images\ 폴더에서 파일을 찾아 제공하도록 설정합니다.
        registry.addResourceHandler("/images/**") // URL 패턴
            .addResourceLocations("file:///c:/images/"); // 실제 저장 위치
    }
}
```


☰ 상품 등록/수정 및 조회하기

☰ 관리자용 상품 목록 보기

다음은 관리자용 상품 목록 보기 화면에서 새로 추가된 Thymeleaf 관련 문법입니다.

항목	설명	사용 예시
th:action	<form> 태그의 action 속성과 동일한 기능입니다.	<form =@"{/admin/products/' + \${productss.number}}">
th:with	변수를 정의하고 값을 셋팅하고자 할 때 사용됩니다.	<div th:with="start=\${(products.number/pageCount)*pageCount + 1}">
th:classappend	조건식이 true이면 class 속성을 추가합니다.	<li class="page-products" th:classappend="\${products.first}?`disabled`">
th:onclick	onclick 속성에 대한 값을 설정합니다.	<a th:onclick="`javascript:page('10')`">

Spring Boot

주문 하기



Spring



Boot



Security

주문 기능 구현하기

주문시 사용되는 어노테이션 및 클래스 목록입니다.

관련 항목

컨트롤러 메소드의 command 객체를 의미하는 매개 변수에 `@RequestBody` 어노테이션을 붙여 줍니다.

컨트롤러 메소드의 반환 타입은 `ResponseEntity` 클래스를 사용하되 반환 타입 앞에 `@ResponseBody` 어노테이션을 붙여 줍니다.

항목	설명
<code>@RequestBody</code>	command 객체에 붙여 주면 JSON 형태의 문자열을 자바 형태의 객체로 변환해 줍니다.
<code>@ResponseBody</code>	자바 객체를 HTTP 응답 바디로 변환해주는 역할을 합니다.
<code>ResponseEntity</code>	<code>HttpEntity</code> 클래스 = <code>HttpHeader</code> + <code>HttpBody</code> <code>HttpEntity</code> 클래스를 상속 받아 구현한 클래스가 <code>RequestEntity</code> , <code>ResponseEntity</code> 클래스입니다.
<code>Principal</code>	시스템을 사용하려고 하는 사용자 혹은 디바이스 혹은 시스템을 통칭하는 용어입니다. 인증된 사용자에 대한 정보를 구할 수 있습니다. 이 인터페이스는 <code>Authentication</code> 인터페이스의 수퍼 인터페이스입니다.

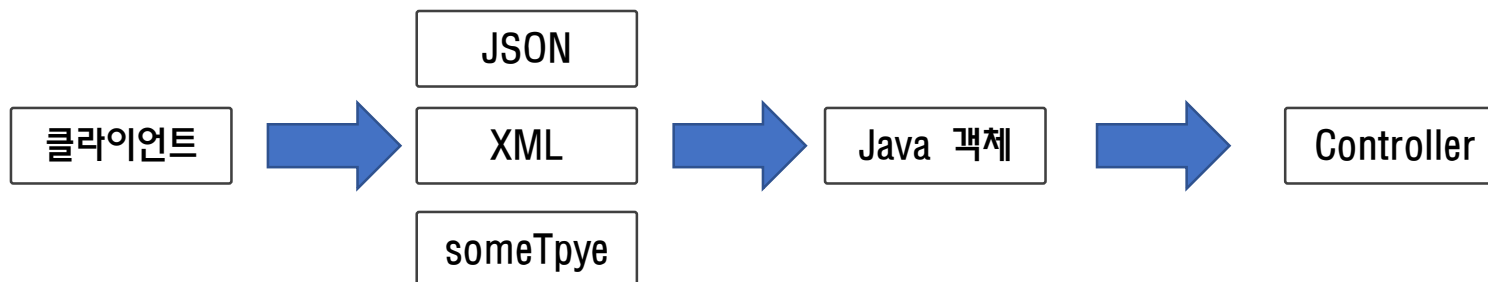
@RequestBody

클라이언트에서 요청한 데이터를 Java 객체로 바인딩해주는 어노테이션입니다.

API에서 JSON, XML과 같은 형식의 데이터를 다룰 때 매우 유용합니다.

`@Valid`와 함께 사용하여 데이터 유효성 검사까지 쉽게 처리할 수 있으며, 올바르게 설정하면 효율적인 RESTful API 개발이 가능합니다.

항목	설명
설정 위치	특정 메소드의 매개 변수



@RequestBody

클라이언트에서 요청한 데이터를 Java 객체로 바인딩해주는 어노테이션입니다.

어노테이션의 특징

항목	설명
요청 데이터 바인딩	@RequestBody는 HTTP 요청의 본문(body)에 있는 데이터를 Java 객체에 직접 바인딩합니다. 클라이언트가 전송한 JSON이나 XML 데이터가 Java 객체에 매핑이 됩니다. Spring은 HttpMessageConverter를 사용해 JSON/XML 데이터를 Java 객체로 자동 변환해 줍니다.
주요 사용 예	주로 POST, PUT, DELETE 요청에서 사용됩니다. 예를 들어, 새로운 리소스를 생성하거나 기존 리소스를 수정할 때, 클라이언트에서 보낸 JSON 데이터를 서버 측에서 Java 객체로 변환해 로직을 수행합니다.
HttpMessageConverter 활용	Spring MVC는 HttpMessageConverter를 통해 본문을 자동 변환합니다. JSON 타입의 처리는 MappingJackson2HttpMessageConverter가 담당합니다. XML 타입의 처리는 Jaxb2RootElementHttpMessageConverter가 담당합니다. 별도의 설정이 없더라도 Spring Boot에서는 기본적으로 Jackson 라이브러리를 이용해 JSON 데이터를 자동으로 매핑합니다.
유효성 검사와 @Valid 어노테이션	@RequestBody는 @Valid와 함께 사용할 수 있습니다. 데이터 유효성을 확인해야 할 경우, @Valid를 @RequestBody 앞에 추가하면 Spring이 자동으로 검증해줍니다. 오류가 있는 경우 MethodArgumentNotValidException 예외를 발생시킵니다.

@ResponseBody

컨트롤러가 반환하는 데이터를 JSON 또는 XML 형식으로 직렬화하여 HTTP 응답 본문으로 전송할 때 사용됩니다.

이 어노테이션을 사용하면 메소드의 반환값을 뷰 페이지가 아닌 HTTP 응답 본문으로 직접 전달할 수 있습니다.

주로 REST API 개발 시 사용되며, HTML 뷰 렌더링을 하지 않고 데이터를 직접 응답할 때 유용합니다.

어노테이션의 특징

항목	설명
데이터 변환	@ResponseBody는 메소드의 반환값을 HTTP 응답 본문으로 변환하기 위해 <code>HttpMessageConverter</code> 를 사용합니다. <code>HttpMessageConverter</code> 는 객체를 JSON, XML 등 다양한 포맷으로 변환하며, 일반적으로 JSON 형태로 반환됩니다. 예를 들어, <code>MappingJackson2HttpMessageConverter</code> 가 JSON 변환을 담당합니다.
자동 직렬화	메소드가 객체를 반환하면, @ResponseBody는 이를 JSON이나 XML과 같은 문자열 형식으로 직렬화합니다. Spring Boot에서는 Jackson 라이브러리가 기본적으로 제공되어 JSON 직렬화가 자동으로 이루어집니다.
REST API에서의 활용	RESTful 서비스를 개발할 때 JSON 형식으로 응답을 보내야 하는 경우가 많습니다. @ResponseBody를 사용하면 추가적인 작업 없이 간단히 JSON 응답을 보낼 수 있어 REST API 개발에 최적화되어 있습니다.
컨트롤러 레벨 사용	클래스 레벨에서 @RestController 어노테이션을 사용하면 클래스 내 모든 메소드에 자동으로 @ResponseBody가 적용됩니다. 이는 @Controller와 @ResponseBody를 함께 사용하는 것과 동일한 효과를 줍니다.

ResponseEntity

HTTP 응답을 세부적으로 제어하기 위해 사용하는 클래스입니다.

HTTP 상태 코드, 헤더, 그리고 응답 본문을 포함한 전체적인 응답을 구성할 수 있습니다.

ResponseEntity는 주로 컨트롤러 메소드의 반환 타입으로 사용되며, RESTful 웹 서비스에서 서버와 클라이언트 간의 커뮤니케이션을 더욱 명확하게 정의할 수 있습니다.

클래스의 특징

항목	설명
HTTP 상태 코드 제어	ResponseEntity는 특정 HTTP 상태 코드를 지정할 수 있습니다. 예시) 성공(200), 요청이 잘못된 경우 400(Bad Request) 또는 404(Not Found) 등등
응답 헤더 설정	HTTP 응답 헤더는 서버와 클라이언트 간의 추가적인 정보를 전달할 때 사용됩니다. ResponseEntity를 통해 특정 헤더를 추가할 수 있으며, 이를 통해 캐싱, 콘텐츠 타입, 쿠키 등의 정보를 클라이언트에게 전달할 수 있습니다.
응답 본문	응답 본문을 직접 설정하여 클라이언트에게 필요한 데이터를 JSON, XML, 문자열 등 다양한 형태로 전달할 수 있습니다. 예를 들어, ResponseEntity<List<User>>와 같이 제네릭 타입을 사용하여 리스트나 특정 객체 형태의 데이터를 응답 본문으로 반환할 수 있습니다.
빌더 패턴 지원	ResponseEntity는 빌더 패턴을 사용하여 응답을 구성할 수 있습니다. 이를 통해 코드의 가독성을 높이고, 응답에 포함될 내용과 형식을 유연하게 설정할 수 있습니다.
예외 상황 처리	컨트롤러 내에서 예외가 발생했을 때, @ExceptionHandler와 함께 ResponseEntity를 사용하여 특정 예외에 대한 응답을 정의할 수 있습니다. 이렇게 하면 각 예외 상황에 맞는 적절한 상태 코드와 메시지를 클라이언트에 전달할 수 있습니다.

Principal

보안 프레임워크에서 인증된 사용자나 개체(entity)를 나타내는 객체로 사용됩니다.

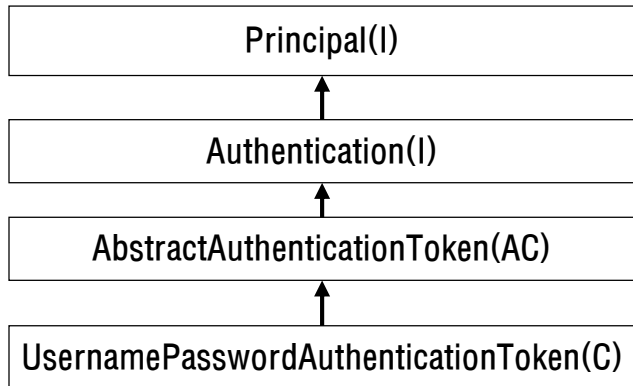
Principal은 자바 보안과 관련된 인터페이스로, 인증된 사용자 또는 엔티티를 나타냅니다.

Spring Security에서 인증된 사용자의 신원 정보를 관리하며 SecurityContext와 Authentication 객체와 함께 사용됩니다.

사용자의 이름을 getName() 메소드로 가져올 수 있으며, OAuth2와 같은 인증 방식에서도 활용됩니다.

Principal을 사용할 때는 null 체크를 통해 인증되지 않은 사용자 접근을 처리해야 합니다.

Principal 관련 계층도



Principal

보안 프레임워크에서 인증된 사용자나 개체(entity)를 나타내는 객체로 사용됩니다.

Principal의 여러 개념

항목	설명
기본 개념	일반적으로 "주체" 또는 "본인"을 의미하며, 시스템에 접근 권한을 가진 사용자를 나타냅니다. 자바에서 Principal은 <code>java.security.Principal</code> 인터페이스를 통해 정의됩니다. 인증이 완료된 사용자를 특정하기 위해 사용되며, 이름 정보를 반환할 수 있는 <code>getName()</code> 메소드를 제공합니다.
주요 기능과 용도	사용자 또는 다른 엔터티의 신원(Identity)을 인증된 상태에서 표현하는 역할을 합니다. 예를 들어, 사용자가 로그인한 후 해당 사용자의 Principal 객체가 생성되고, 이를 통해 사용자 정보를 조회할 수 있습니다. 주로 인증된 사용자의 정보를 확인하거나 세션에서 정보를 가져와 권한을 부여하는 데 사용됩니다.
Principal과 Spring Security	Principal 객체는 세션 내에서 사용자의 신원을 나타내며 Authentication 객체와 함께 사용됩니다.
Principal과 OAuth2	OAuth2 인증 방식을 사용할 때도 Principal 객체가 중요한 역할을 합니다. 특히, OAuth2에서는 인증된 사용자를 나타내는 OAuth2User가 Principal 인터페이스를 구현하여 로그인한 사용자 정보를 제공합니다.
사용 시 주의 사항	인증되지 않은 사용자가 Principal 객체에 접근하려 하면 null을 반환할 수 있으므로, 항상 객체가 null이 아닌지 확인해야 합니다. Principal은 사용자가 누구인지를 나타내는 데 초점을 맞추며, 역할(Role)에 대한 접근 제어는 별도로 GrantedAuthority 인터페이스를 통해 처리됩니다. Principal 자체는 권한 정보를 직접 제공하지 않습니다.

JSON 연관 함수

stringify() 함수는 JSON 객체를 문자열(string)으로 변환해 줍니다.

매개 변수

함수	설명
사용 형식	JSON.stringify(value[, replacer[, space]])
value	필수 매개변수로, JSON 문자열로 변환할 JavaScript 객체입니다.
replacer	선택 사항으로, 객체의 특정 속성만 JSON으로 변환하고자 할 때 사용됩니다. 함수 형태일 때는 변환 과정에서 특정 조건을 지정할 수 있습니다.(화살표 함수 사용 가능) 배열 형태일 때는 변환할 속성들의 이름을 지정할 수 있습니다.
space	가독성을 높이기 위해 들여 쓰기를 설정할 수 있습니다. 숫자 값을 넣으면 해당 숫자만큼 공백을 추가하여 들여 쓰기하며, 문자열 값을 넣으면 문자열이 들여 쓰기에 사용됩니다.

사용 예시

```
const human = { name: "Alice", age: 25 }; /* human는 자바 스크립트 객체 */  
const jsonString = JSON.stringify(human);  
console.log(jsonString); // '{"name":"Alice","age":25}' ← 문자열 출력하기
```

JSON 연관 함수

parse() 함수는 문자열(string)을 JSON 객체로 변환해 줍니다.

매개 변수

함수	설명
사용 형식	JSON.parse(text[, reviver])
text	필수 매개변수로, JavaScript 객체로 변환할 JSON 문자열입니다. 이 문자열은 유효한 JSON 형식이어야 합니다.
reviver	선택 사항으로, 각 키-값 쌍을 변환하는 함수입니다. 특정 조건을 적용하여 결과 객체를 생성할 수 있습니다.

사용 예시

```
const jsonString = '{"name": "Alice", "age": 25}';

// age 속성의 값을 10배로 변환
const human = JSON.parse(jsonString, (key, value) => {
  return key === "age" ? value * 10 : value;
});

console.log(human); // { name: 'Alice', age: 250 }
```

주문 기능 구현하기

주문 기능 페이지를 구현하기 전에 ajax에 대한 내용을 우선 살펴 보도록 하겠습니다.

AJAX

AJAX = Asynchronous JavaScript And XML.

AJAX is not a programming language.

AJAX just uses a combination of:

- A browser built-in XMLHttpRequest object (to request data from a web server)
- JavaScript and HTML DOM (to display or use the data)
- JavaScript와 최근에는 fetch API를 것도 사용됩니다.

misunderstanding

AJAX is a **misleading name**. AJAX applications might use XML to transport data, but it is equally common to transport data as **plain text** or **JSON text**.

AJAX allows web pages to be updated **asynchronously** by exchanging data with a web server behind the scenes. This means that it is possible to update parts of a web page, **without reloading the whole page**.

주문 기능 구현하기

가장 많이 사용되는 함수는 \$.ajax()입니다.

\$.ajax() 함수의 매개 변수는 다음과 같은 항목들이 존재합니다.

항목	설명
url	요청을 보낼 서버의 URL(ASP.NET/WebService, JSP, ASP, PHP, CGI 등등)을 지정합니다.
type	HTTP 메소드, 기본 값은 GET, POST와 GET 중에서 선택합니다.
data	서버로 전송 되는 데이터를 의미합니다.
datatype	응답 결과의 데이터 형식으로, xml, html, json, jsonp, script, text 등등이 있습니다.
timeout	요청에 대한 응답 제한 시간(millisecond)을 지정합니다.
contentType	서버 전송시의 content-type을 타입을 지정합니다. 기본 값은 'application/x-wwwform-urlencoded'입니다.
success	응답이 성공 하는 경우 호출되는 콜백 함수를 지정합니다.
error	에러 발생시 실행되는 콜백 함수를 지정합니다.
complete	요청이 완료되었을 때 호출 되는 콜백 함수를 지정합니다.
cache	요청된 페이지에 대한 캐싱 기능을 사용하지 여부를 지정합니다. 기본 값은 true입니다.
beforeSend	요청을 보내기 전에 실행하고자 하는 콜백 함수를 지정합니다. 예를 들어서, 전송하기 전에 사이트 간 요청 위조(Cross-site Request Forgery) 등을 사전에 막을 수 있습니다.
asyn	기본 값은 true로써 비동기 호출이며, false로 지정하면 동기 호출 전송 방식이 됩니다.

@PostMapping

HTTP POST 요청을 특정 메소드에 매핑하기 위해 사용됩니다.

클라이언트가 POST 요청을 통해 서버에 데이터를 전송할 때 해당 요청을 처리할 메소드를 지정할 수 있습니다.

`@PostMapping`은 RESTful 웹 애플리케이션에서 데이터 생성이나 자원의 업데이트 등의 작업을 수행할 때 유용하게 사용됩니다.

주요 속성

항목	설명
value 또는 path	요청을 처리할 URI 경로를 지정합니다.
consumes	메소드가 허용하는 콘텐츠 타입을 지정합니다. 예시) `consumes = "application/json"`은 요청 본문이 JSON 형식이어야 함을 의미합니다.
produces	메소드가 반환할 콘텐츠 타입을 지정합니다. 예시) `produces = "application/json"`은 응답이 JSON 형식으로 반환됨을 의미합니다.
headers	특정 헤더 조건에 맞는 요청만 매핑하도록 설정합니다. 예시) `headers = "content-type=application/json"`

Spring Boot

MyBatis 사용하기



Spring



Boot



Security

☰ MyBatis 사용하기

☰ 의존성 추가

MyBatis 관련 설정에 대한 의존성을 추가해야 합니다.

MyBatis와 관련된 여러 설정 정보를 다음과 같이 작성합니다.

build.gradle

```
dependencies {  
    // 이전 다른 설정들  
    implementation 'org.mybatis.spring.boot:mybatis-spring-boot-starter:2.3.1'  
}
```

pom.xml

```
<dependency>  
    <groupId>org.mybatis.spring.boot</groupId>  
    <artifactId>mybatis-spring-boot-starter</artifactId>  
    <version>2.3.1</version> <!-- MyBatis 최신 버전으로 설정 -->  
</dependency>
```


☰ Mapper 인터페이스

인터페이스에 어노테이션을 사용하여 SQL을 직접 정의하고, 매핑하는 방식입니다.

인터페이스 방식은 간단한 CRUD 작업에서 자주 사용되며, XML 방식에 비해 설정이 간결하다는 장점이 있습니다.

Mapper 인터페이스를 사용하는 방식은 xml 파일을 사용하는 방식에 비하여 다음과 같은 장단점이 있습니다.

항목	설명
간결함	XML 매핑 파일을 작성할 필요가 없고, 자바 코드 내에서 직접 SQL을 관리할 수 있습니다.
직관적	SQL 쿼리를 직접 명시하므로 SQL 실행 로직이 코드와 밀접하게 연결되어 있어 유지보수가 용이합니다.

주의 사항

복잡한 SQL 쿼리를 직접 작성해야 하므로 쿼리가 길어질 수 있습니다.

이럴 경우 여전히 XML 매핑 파일을 사용하는 것이 더 적합할 수 있습니다.

여러 메소드에서 동일한 SQL을 사용할 경우, 인터페이스 방식에서는 쿼리를 중복해서 작성할 수 있습니다.

이런 상황에서는 XML 매핑 파일을 사용하는 것이 재사용성을 높이는 데 유리할 수 있습니다.

☰ Mapper 인터페이스

인터페이스에 어노테이션을 사용하여 SQL을 직접 정의하고, 매핑하는 방식입니다.

MyBatis에서 SQL을 실행할 메소드를 정의한 인터페이스를 만들고, SQL 쿼리는 어노테이션을 사용해 명시합니다.

사용 예시

```
@Mapper
public interface UserMapper {
    // 단일 사용자 조회
    @Select("SELECT * FROM users WHERE id = #{id}")
    User findById(@Param("id") int id);

    // 모든 사용자 조회
    @Select("SELECT * FROM users")
    List<User> findAll();

    // 사용자 추가
    @Insert("INSERT INTO users(name, email) VALUES(#{name}, #{email})")
    void insertUser(User user);
}
```

☰ MyBatis 사용하기

☰ @Mapper을 사용한 MyBatis 사용하기

인터페이스의 각 메소드에 대해 다음과 같은 SQL 쿼리를 어노테이션으로 정의합니다.

관련 패키지) `import org.apache.ibatis.annotations.*;`

항목	설명
@Select	데이터를 조회하는 SQL을 정의합니다. `#{}`는 파라미터를 바인딩하는 부분입니다.
@Insert	새로운 데이터를 삽입하는 SQL을 정의합니다.
@Update	데이터를 수정하는 SQL을 정의합니다.
@Delete	데이터를 삭제하는 SQL을 정의합니다.
@Param	메소드 파라미터가 SQL 쿼리의 파라미터로 사용될 때 이름을 명시합니다. 이는 SQL 쿼리에서 여러 파라미터를 사용할 때 유용합니다.